Converse Python, Partner AI

The Intentional Prompting Methodology

Michael Borck

2026-03-30

Converse Python, Partner AI
The Intentional Prompting Methodology

Copyright © 2026 Michael Borck. All rights reserved.

Published by Michael Borck
Perth, Western Australia

ISBN: 979-8-2543-0182-0

First edition, 2026.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and certain non-commercial uses permitted by copyright law.

This work is also available under a Creative Commons Attribution (CC BY) licence for content and MIT licence for code examples at the companion website. See below for details.

AI disclosure: This book was written using the methodology it describes. AI tools were used as thinking partners throughout the drafting, iterating, and refining process. The author reviewed, challenged, and took responsibility for every sentence and line of code.

Companion website:
https://michael-borck.github.io/converse-python-partner-ai
Source: https://github.com/michael-borck/converse-python-partner-ai

Table of contents

Preface . . . 1

I The Philosophy **7**

1 Conversation, Not Delegation . . . 9

II Foundations **15**

2 Introduction . . . 17

3 The AI Revolution in Programming . . . 23

4 Principles of Intentional Prompting . . . 29

III Methodology **45**

5 The Six-Step Programming Methodology . . . 47

6 Restate and Identify (Steps 1-2) . . . 63

7 Work by Hand (Step 3) . . . 73

8 Pseudocode (Step 4) . . . 81

9 Convert to Code (Step 5) . . . 95

10 Test with Data (Step 6) . . . 111

IV Patterns & Practices **127**

11 Intentional Prompting Patterns . . . 129

12 Debugging with AI . . . 147

13 Refactoring Strategies . . . 165

14 Case Studies . . . 181

15 Add a maximum combination size parameter . . . 193

V Advanced Topics **217**

16 Scaling to Complex Projects . . . 219

17 Authentication Service Documentation Structure 233
18 Team Knowledge Sharing Patterns for Complex Systems 243
19 Teaching and Learning 249
20 Future Directions 261
Acknowledgments 275
About the Author 277

Appendices **279**
A Common Pitfalls and Solutions 279
B Glossary 289

Preface

Why This Book Exists

Most developers use AI to get code written faster. They prompt, they paste, they ship. That approach produces code, but it skips the thinking that makes a developer good at their job.

Programming with AI is not just about speed. It is about thinking differently about the development process itself. The developers who get the most from AI tools are not the ones who delegate the most. They are the ones who stay in the conversation: directing, questioning, understanding, and making the architectural decisions that the model cannot make for them.

This book teaches a structured methodology for that conversation. It calls the approach *intentional prompting* — using AI tools with precision and purpose while ensuring you remain the architect and decision-maker throughout.

Who This Book Is For

- Software developers looking to integrate AI tools effectively into their workflow
- Educators teaching programming in an AI-assisted world
- Students learning to code alongside AI assistants
- Tech leaders developing best practices for AI-augmented development teams

You do not need to be an expert programmer. Basic familiarity with programming concepts will help you get the most from the examples, but the methodology applies at every level.

What This Book Is Not

This is not a guide to specific AI tools. It does not teach you how to use GitHub Copilot, ChatGPT, Claude, or any particular platform. Interfaces change; the methodology here does not.

It is not a prompt cookbook. You will find prompts throughout, but they are starting points for conversation, not finished products. If you are looking for a library of copy-paste prompts that write your code for you, this is the wrong book.

It is not a book about replacing developers with AI. It argues the opposite: that AI is most powerful when a skilled developer is directing it, and that the skills which make you a good developer — decomposition, abstraction, critical evaluation, architectural judgement — are exactly the skills that make you good at working with AI.

And it is not a book that pretends AI always gets it right. A significant part of the methodology is about recognising when AI output is wrong, understanding why, and knowing how to fix it.

If You Are Feeling Uncertain

You are not alone. Many developers experience a quiet anxiety: that everyone else has figured this out already, that they are falling behind, that their skills are becoming obsolete. That feeling is nearly universal and rarely admitted. You are not behind. The technology is genuinely new, and the developers who appear to have it all figured out are mostly just a few weeks ahead. This book meets you wherever you are.

How This Book Is Structured

The book is divided into five parts:

1. **The Philosophy**: Why intentional prompting matters and how it connects to the broader methodology
2. **Foundations**: Core concepts of the human-AI partnership and the principles behind effective collaboration
3. **Methodology**: The six-step programming methodology adapted for AI collaboration
4. **Patterns & Practices**: Effective prompting patterns, debugging, refactoring, and practical case studies
5. **Advanced Topics**: Scaling to complex projects, teaching and learning contexts, and future directions

Each chapter includes practical examples, exercises, and reflections to help you apply these concepts in your own work.

The Companion Book

This book applies the methodology developed in *Conversation, Not Delegation: How to Think With AI, Not Just Use It* (Borck, 2025) to software development. That companion book covers the full framework in depth: the Conversation Loop, the VET framework for evaluating AI output, the average-versus-precise decision grid, and the principle of AI Last. If you want the underlying rationale for why the approaches in this book work, or you want to share a discipline-neutral version with non-developer colleagues, start there. Available at https://michael-borck.github.io/conversation-not-delegation (https://michael-borck.github.io/conversation-not-delegation).

Related Resources

This guide is part of a 5-book series designed to help you master modern software development in the AI era:

Foundational Methodology

Converse Python, Partner AI: The Python Edition (this book) - Intentional AI collaboration for programmers (human oversight + methodology + LLM = success)

Python Track

Think Python, Direct AI: Computational Thinking for Beginners - An innovative programming textbook that embraces AI as a learning partner. Master Python by learning how to think computationally and direct AI to help you build solutions. Perfect for absolute beginners in the age of AI.

Code Python, Consult AI: Python Fundamentals for the AI Era - Learn fundamental Python with AI integration - ideal for those who want a focused introduction to Python fundamentals

Ship Python, Orchestrate AI: Professional Python in the AI Era - Build professional-grade Python applications with modern tools (uv, ruff, mypy, pytest - simple but not simplistic)

Web Track

Build Web, Guide AI: Business Web Development with AI - A practical guide to modern web development that embraces AI as your development partner while building real business solutions.

Book Progression

```
                    CONVERSATION,
                  NOT DELEGATION
                   (Methodology)




     PYTHON TRACK                    WEB TRACK



    Think Python                   Build Web,

         ↓                         Guide AI

    Code Python

         ↓
    Ship Python
```

Start with "Think Python, Direct AI" if you're a complete beginner, or jump into "Code Python, Consult AI" if you want a more focused approach to Python fundamentals. Both books prepare you for the production-focused content in "Ship Python, Orchestrate AI." For web development, start with "Build Web, Guide AI." The methodology from "Converse Python, Partner AI" enhances all your Python development work.

Ways to Engage with This Book

This book is available in several formats. Pick whichever fits how you work and learn.

- **Read it online.** The full book is freely available at the companion website (https://michael-borck.github.io/converse-python-partner-

ai), with dark mode, search, and navigation.

- **Read it on paper or e-reader.** Available as a paperback and ebook through Amazon KDP.
- **Converse with it.** The online edition includes a chatbot grounded in the book's content. Ask it questions about intentional prompting, the six-step methodology, or any concept in the book.
- **Feed it to your own AI.** The `llm.txt` file provides a clean text version of the entire book, ready to paste into ChatGPT, Claude, or any AI tool.
- **Run the code.** All code examples and supplementary materials are available on GitHub (https://github.com/michael-borck/converse-python-partner-ai). DeepWiki (https://deepwiki.com/michael-borck/converse-python-partner-ai) provides an AI-navigable view of the repository.
- **Browse all books.** This book is part of a series. See all titles at books.borck.education (https://books.borck.education).

The online version is always the most current. The printed and ebook editions are updated periodically.

Feedback and Errata

Found an error? Have a suggestion?

- Open an issue: github.com/michael-borck/converse-python-partner-ai/issues (https://github.com/michael-borck/converse-python-partner-ai/issues)
- Email: michael@borck.me

Part I

The Philosophy

Chapter 1

Conversation, Not Delegation

> The difference between using AI well and using it poorly is not technical skill. It is whether you are thinking or just accepting.

1.1 The Core Distinction

When you interact with AI, you make a choice, often unconsciously, about the nature of that interaction.

Delegation is handing off a task: "Write me a marketing plan." "Create a Python function that does X." You ask, AI delivers, you accept. It's transactional. The AI does the thinking; you consume the output.

Conversation is collaborative thinking: You engage the AI in dialogue. You ask questions, challenge assumptions, build understanding together. The AI contributes, but you remain the architect of the outcome.

This distinction matters because:

- **Delegation makes you dependent.** You get output but not understanding. When something goes wrong, you can't fix it. When requirements change, you start from scratch.
- **Conversation makes you capable.** You build understanding alongside output. You can explain, modify, and improve. You grow with each interaction.

The philosophy of this book is simple: **Treat AI as a thinking partner, not a task executor.**

1.2 What is a "Prompt"?

Before we go further, let's demystify some terminology.

> **A prompt is simply a request or instruction you give to AI.**

That's it. When you type "Explain how photosynthesis works" into ChatGPT, that's a prompt. When you ask Claude to "Review this code for bugs," that's a prompt. When you tell Copilot to "Write a function that calculates shipping costs," that's a prompt.

Prompts can be simple or complex, vague or precise, single questions or multi-paragraph specifications. The quality of AI's response depends heavily on how you craft your prompt, but at its core, a prompt is just a request.

1.3 What is "Intentional Prompting"?

> **Intentional prompting is the practice of crafting each AI interaction deliberately, to build understanding, not just get output.**

The word "intentional" is key. It means:

- **Purposeful**: Each prompt has a clear goal beyond "give me an answer"
- **Thoughtful**: You consider what you're asking and why
- **Sequential**: Each prompt builds on previous exchanges
- **Learning-oriented**: You seek understanding, not just solutions

A single prompt is a request. A series of intentional prompts becomes a conversation.

Consider the difference:

Unintentional prompting:

```
"Write a function to validate email addresses"
[Accept output, move on]
```

Intentional prompting:

```
"I need to validate email addresses. Before we
write code,
what makes an email address valid? What edge cases
should we consider?"

[AI explains the rules and edge cases]
```

```
"Good. Now, what are the trade-offs between using
a regular expression
versus parsing the parts individually?"

[AI compares approaches]

"Let's go with regex for simplicity. Can you show
me the pattern
and explain each part?"

[AI provides explained solution]

"What inputs would break this? Let's test some
edge cases."

[Collaborative refinement]
```

The second approach takes longer but produces:

- A solution you understand and can modify
- Knowledge that transfers to similar problems
- Confidence in the code's limitations
- Skills that grow with each interaction

1.4 One Prompt, One Job

A related principle: give each prompt a single focused task.

When you ask AI to "write a function, add error handling, write tests, and document it," you get shallow results for all four. When you ask it to write the function first, then handle errors in the next message, then write tests, each response is deeper and more useful. The model calibrates its depth to match the scope of what you asked. A checklist gets checklist responses. A focused question gets a considered answer.

This is the foundation of the conversation approach. You are not handing over a task list. You are working through a problem one step at a time, reviewing each result before moving on.

1.5 The Philosophy in Action

Let's see how delegation and conversation differ in practice.

1.5.1 A Python Example

Delegation approach:

```
Human: Write a Python function that calculates
total price with tax and discounts.

AI: [Generates function - works for simple cases
but what about negative
     prices, multiple discounts, or tax-exempt
     items?]
```

Conversation approach:

```
Human: I need to calculate total price with tax
and discounts. Before we
       write code, what business rules should I
       consider?

AI: [Explains order of operations, compound
discounts, rounding, edge cases]
Human: Those are good points. Lets handle
discounts before tax.
       What is a clean way to structure this?

AI: [Proposes design with clear separation of
concerns]

Human: Now lets implement it. Can you explain each
part as we go?

[Collaborative implementation with understanding]
```

The same principle applies beyond programming. Whether you are planning a marketing launch, designing a research methodology, or writing a policy document, the difference between delegation and conversation determines whether you understand the result or just received it.

1.6 Beyond Programming

> **Universal Principles**
>
> While this book applies intentional prompting to software development, the philosophy of Conversation, Not Delegation works in any domain: business planning, creative writing, research, education, and beyond. The principles transfer - only the examples change.

1.7 The Two-Chat Workflow

One practical way to build this discipline into your development workflow is to separate thinking from building entirely. Instead of doing everything in one session, run two.

Chat 1: Understand the problem. Open a session with no intention of writing code. Use it to explore the requirements, challenge your assumptions, and clarify what you are actually trying to build. What are the edge cases? What are the real constraints? What approaches have you not considered? Let the conversation wander. This session is messy by design. Its purpose is to sharpen your thinking until you know what you actually need.

Chat 2: Write the code. Start a fresh session and arrive with a clear specification: what the function should do, what inputs it takes, what edge cases it must handle, and what design decisions you have already made. The code will be dramatically better because the input is better.

The critical step is the gap between the two chats. You do not dump everything from the first session into the second. You review what emerged, keep what matters, discard what does not, and write a focused brief that reflects your decisions. That act of curation is where your engineering judgement lives.

This maps directly to the average-versus-precise framework. A project that starts in the danger zone (large, precise) gets decomposed during the thinking chat into components that each sit in different quadrants. The architecture discussion belongs in Chat 1. The individual functions can be written in Chat 2 with appropriate trust for their position on the grid.

1.8 What is Ahead

This book will teach you to have productive conversations with AI:

- **Part 1: Foundations** establishes the core concepts and principles
- **Part 2: Methodology** provides a structured six-step approach
- **Part 3: Patterns and Practices** offers reusable conversation patterns
- **Part 4: Advanced Topics** addresses complex scenarios and future directions

Throughout, you will see examples in both programming and business contexts, reinforcing that these principles apply wherever you work with AI.

The goal is not to make you dependent on AI, but to make you more capable with AI as your partner.

Lets begin.

Part II

Foundations

Chapter 2

Introduction

> Intentional prompting is not about getting better answers from AI. It is about becoming a better developer alongside it.

2.1 Applying the Philosophy to Software Development

In the previous chapter, we established the core philosophy: **Conversation, Not Delegation**. We defined intentional prompting as the practice of crafting each AI interaction deliberately, to build understanding, not just get output.

Now we turn to software development specifically.

The landscape of programming is undergoing a profound transformation. With AI assistants like GitHub Copilot, Claude, and ChatGPT, developers have powerful tools that can generate code, explain concepts, debug problems, and assist with software design.

These tools have ignited debates: Will AI replace programmers? Should we resist AI-generated code? Is programming fundamentally changing?

This book proposes a clear answer: **Intentional Prompting**, applying the "Conversation, Not Delegation" philosophy to position AI as a collaborative partner in development rather than either a threat or a magical solution.

2.2 What is Intentional Prompting?

Intentional prompting is a deliberate approach to programming that leverages AI tools while maintaining human control over the development process. Rather than simply asking AI for complete solutions, intentional prompting focuses on:

1. Understanding programming concepts well enough to evaluate AI-generated code
2. Guiding AI through a structured development methodology
3. Using AI as a tool for learning and exploration rather than just output
4. Maintaining the human developer as the architect and decision-maker

This approach recognises that the most powerful development workflow combines human creativity, judgment, and domain knowledge with AI's ability to handle routine coding tasks, suggest implementations, and provide contextual information.

2.2.1 Intentional Prompting vs. Prompt Engineering

It's important to distinguish intentional prompting from the related but distinct field of prompt engineering:

Prompt Engineering focuses primarily on crafting effective instructions for AI systems to produce desired outputs. It encompasses techniques like zero-shot and few-shot prompting, chain-of-thought prompting, and role-based prompting. The emphasis is on optimising the prompt itself to get the best possible output from the AI.

Intentional Prompting builds upon prompt engineering but differs in several key ways:

1. **Methodology vs. Technique**: Intentional prompting provides a complete methodology (the six-step approach) rather than just techniques for crafting prompts.

2. **Process vs. Output**: While prompt engineering prioritises getting optimal outputs, intentional prompting emphasises the development process itself as the primary value.

3. **Human Agency**: Intentional prompting explicitly maintains the human as the director and decision-maker throughout the development process, not just the prompt creator.

4. **Learning Focus**: Intentional prompting has an explicit educational component, designed to enhance human understanding and skills alongside producing code.

5. **Structured Framework**: Intentional prompting embeds prompt techniques within a systematic framework for approaching programming problems, rather than treating prompting as an isolated activity.

In essence, intentional prompting integrates prompt engineering techniques within a broader methodology that maintains human agency and prioritises understanding and learning throughout the development process. This distinction becomes especially important as AI capabilities advance toward autonomous coding agents, where maintaining human oversight and understanding becomes increasingly critical.

For a comprehensive comparison of intentional prompting with other approaches to AI-assisted programming (including prompt engineering, pseudocode for LLMs, agentic programming, and vibe coding), see the comparative framework in Appendix C: AI Tool Comparison.

2.3 Why Intentional Prompting Matters

As AI coding assistants become increasingly capable, there's a risk that developers might:

- Become overly dependent on AI suggestions without understanding them
- Lose skills in fundamental programming concepts
- Accept sub-optimal solutions because they're quick and easy

Intentional prompting addresses these concerns by providing a framework that helps developers use AI effectively while continuing to grow their own skills and understanding.

For students learning to code, this approach is particularly valuable. Rather than seeing AI as either a shortcut to avoid learning or a forbidden tool, intentional prompting teaches students to use AI as a learning partner that enhances their educational journey.

2.3.1 Beyond Automation: The Case for Human Oversight

The rapid advancement of AI has led to the emergence of agentic programming, where AI agents can autonomously perform complex development tasks with minimal human intervention. While these systems show impressive capabilities in routine coding tasks, they also highlight precisely why methodologies like intentional prompting remain essential.

Research reveals that even the most sophisticated AI agents face fundamental limitations that necessitate human oversight:

1. **Understanding Gap**: AI agents lack true comprehension of problem context and domain requirements. They can pattern-match solutions but miss critical business or user needs that aren't explicitly stated.

2. **Accountability and Ethics**: Humans must bear responsibility for decisions with real-world impacts. Delegating this completely creates accountability gaps when things go wrong.

3. **Novel Problem Solving**: While agents excel at pattern recognition, they struggle with truly novel problems requiring innovative approaches outside their training distribution.

4. **Quality Thresholds**: Agents lack the meta-awareness to know when a solution is "good enough" versus when to keep refining for the specific context.

5. **Creative Direction**: Human value judgments about elegance, maintainability, and design philosophy are subjective but crucial for creating coherent, high-quality systems.

6. **Integration Knowledge**: Humans better understand how code fits into larger systems, organisational constraints, and unstated requirements.

7. **Skill Development**: The learning benefits of intentional programming disappear if fully automated, undermining the educational value essential for long-term developer growth.

Even with sophisticated agent networks, complex programming will likely remain a collaborative process with humans providing critical validation, creative direction, context understanding, and accountability. The intentional prompting methodology provides a structured framework for this collaboration, ensuring that humans maintain control of the development process while leveraging the strengths of AI assistants.

2.4 The Six-Step Methodology

At the heart of intentional prompting is a structured methodology that guides the development process:

1. **Restate the problem** in your own words
2. **Identify input and output** requirements
3. **Work the problem by hand** with concrete examples
4. **Write pseudocode** to outline the solution strategy
5. **Convert to code** (with AI assistance as appropriate)
6. **Test with a variety of data**, especially edge cases

This methodology works both with and without AI assistance, but becomes particularly powerful when combined with intentional prompting techniques that guide AI tools effectively.

2.5 Who Should Read This Book

This book is designed for:

- Professional developers adapting to AI-assisted workflows
- Students learning programming in an AI-enabled world
- Educators teaching programming in the age of AI
- Technical managers developing best practices for AI integration

No matter where you are in your programming journey, intentional prompting offers a framework for leveraging AI tools effectively while continuing to develop your own skills and understanding.

2.6 Related Materials

This book is part of a comprehensive series for mastering modern software development in the AI era:

Foundational Methodology

- Converse Python, Partner AI: The Python Edition (https://michael-borck.github.io/converse-python-partner-ai) (this book)

Python Track

- Think Python, Direct AI: Computational Thinking for Beginners (https://michael-borck.github.io/think-python-direct-ai) - Perfect for absolute beginners
- Code Python, Consult AI: Python Fundamentals for the AI Era (https://michael-borck.github.io/code-python-consult-ai) - Core Python knowledge
- Ship Python, Orchestrate AI (https://michael-borck.github.io/ship-it-python-in-production) - Professional Python in the AI Era

Web Track

- Build Web, Guide AI: Business Web Development with AI (https://michael-borck.github.io/build-web-guide-ai) - HTML, CSS, JavaScript, WordPress, React

In the following chapters, we'll explore the principles of intentional prompting in depth, examine the six-step methodology with practical examples, and develop patterns for effective collaboration between human developers and AI assistants.

Chapter 3

The AI Revolution in Programming

The tools changed. The need for understanding did not.

3.1 Evolution of Programming Tools

The history of programming is a story of continuous abstraction. From machine code to assembly language, from procedural to object-oriented paradigms, each evolution has aimed to make programming more accessible, maintainable, and powerful.

AI coding assistants represent the next major evolution in this timeline. Rather than simply providing syntax highlighting or code completion based on predefined rules, these tools can understand intent, generate complete solutions, and explain complex concepts in natural language.

3.2 Types of AI Coding Assistants

Several categories of AI tools have emerged to assist programmers:

3.2.1 Code Completion Tools

Tools like GitHub Copilot, Amazon CodeWhisperer, and TabNine integrate directly into the development environment and suggest code completions as you type. These suggestions range from completing a single line to generating entire functions.

These tools excel at: - Automating repetitive coding patterns - Suggesting syntactically correct code - Implementing common algorithms and data structures - Generating boilerplate code

3.2.2 Conversational AI Assistants

Conversational AI models like Claude, ChatGPT, and Bard take a different approach. Rather than integrating directly into the code editor, they provide a chat interface where developers can: - Ask questions about programming concepts - Request code suggestions for specific problems - Explain existing code - Debug issues - Discuss design alternatives

3.2.3 IDE Integrations

Some tools combine both approaches, integrating conversational AI directly into development environments. Examples include: - Visual Studio IntelliCode - JetBrains AI Assistant - Cursor (formerly Mutable AI)

These tools allow developers to maintain their familiar workflow while accessing AI assistance.

3.3 Capabilities and Limitations

Understanding the true capabilities and limitations of AI coding assistants is crucial for using them effectively.

3.3.1 What AI Can Do Well

- Generate syntactically correct code in most programming languages
- Implement standard algorithms and design patterns
- Convert between similar programming languages
- Explain code and programming concepts
- Identify common bugs and suggest fixes
- Provide alternative implementations for the same problem

3.3.2 Current Limitations

- Limited understanding of business context and domain-specific requirements
- Difficulty with novel or extremely complex algorithms
- Generation of plausible-looking but incorrect code for edge cases
- Outdated knowledge of newer frameworks or libraries
- Inconsistent code quality depending on prompt clarity
- Potential to reinforce common programming misconceptions

Understanding these limitations is not about dismissing AI assistance, but about knowing when and how to rely on AI versus your own judgement.

Most of these limitations trace back to a single fact: **LLMs interpolate, they do not retrieve.** When you ask an AI assistant to write a function, it is not looking up the correct implementation in a database. It is predicting what plausible code would look like based on the millions of examples it absorbed during training. This is why it can produce a function that looks right, passes simple tests, and fails on edge cases. The prediction was convincing. The prediction was not verified. That distinction is why the intentional prompting methodology in this book emphasises understanding and testing, not just generating.

3.4 Common Misconceptions

Several misconceptions about AI coding assistants can lead to their ineffective use:

3.4.1 "AI Will Replace Programmers"

Reality: AI tools augment programmer capabilities rather than replacing the need for human judgment, creativity, and domain knowledge. The skills required for programming are evolving, not disappearing.

3.4.2 "AI Always Produces Correct Code"

Reality: AI-generated code often contains subtle bugs, inefficiencies, or security vulnerabilities that require human review. The most effective workflows involve careful review and testing of AI suggestions.

3.4.3 "Using AI Is Cheating"

Reality: AI assistance is simply another tool in the developer's toolkit. The skill lies in how effectively you direct the AI, evaluate its outputs, and integrate suggestions into your work.

3.4.4 "AI Works Best When Asked for Complete Solutions"

Reality: AI often produces better results when guided through a step-by-step process that breaks down complex problems and allows for iterative refinement.

3.5 The Human-AI Partnership Model

The most productive relationship with AI coding assistants is a partnership model rather than a service model. In this partnership:

3.5.1 The Human Provides:

- Problem understanding and domain knowledge
- Architectural direction and design constraints
- Quality standards and evaluation criteria
- Creative direction and novel approaches
- Critical thinking about suggested solutions

3.5.2 The AI Provides:

- Implementation suggestions
- Alternative approaches
- Documentation and explanations
- Memory of syntax and APIs
- Help with tedious or repetitive tasks

This partnership model focuses on complementary strengths rather than replacement. It recognises that the combination of human creativity and judgment with AI capabilities creates possibilities that neither could achieve alone.

3.6 Emerging Approaches to AI-Assisted Development

As AI coding tools have advanced, distinct philosophical approaches to their use have emerged. These approaches represent different perspectives on how developers should integrate AI into their workflow.

3.6.1 Vibe Coding: Embracing AI-Led Development

"Vibe coding" (or "vibe programming") represents a paradigm shift where developers largely defer to AI for code generation. Introduced by Andrej Karpathy, a co-founder of OpenAI and former AI leader at Tesla, in early 2025, this approach encourages developers to "fully give in to the vibes, embrace exponentials, and forget that the code even exists."

The vibe coding philosophy centres around:

1. **Natural Language Prompting**: Developers describe desired functionality in plain language rather than focusing on implementation details.

2. **Minimal Code Review**: Code is often accepted with limited scrutiny, prioritising speed and output over deep understanding.
3. **Rapid Iteration**: Developers quickly implement AI suggestions, test for functionality, and refine through further prompts rather than manual coding.
4. **Reduced Technical Focus**: As Karpathy described it, "It's not really coding - I just see things, say things, run things, and copy-paste things, and it mostly works."

This approach has found particular traction in rapid prototyping and startup environments. Y Combinator reportedly found that 25% of startups in its Winter 2025 batch had codebases that were 95% AI-generated.

While vibe coding offers unprecedented speed and accessibility, it presents challenges in debugging, maintenance, and knowledge development. Critics argue that it can lead to poor code quality, security vulnerabilities, and a devaluation of fundamental programming skills.

3.6.2 Intentional Prompting: Human-Directed AI Assistance

Intentional prompting, which we'll explore throughout this book, takes a different approach. Rather than deferring to AI, it views AI as a powerful assistant within a human-directed process.

This approach emphasises:

1. **Structured Problem-Solving**: Following a deliberate methodology that starts with understanding the problem
2. **Selective AI Integration**: Leveraging AI at specific points in the development process while maintaining human agency
3. **Critical Evaluation**: Carefully reviewing AI-generated code for correctness, efficiency, and security
4. **Learning-Focused Collaboration**: Using AI to enhance understanding rather than bypass it

Intentional prompting recognises AI's value but prioritises the developer's control and comprehension of the final solution.

3.6.3 Collaborative Integration: Finding the Middle Ground

Between vibe coding's emphasis on speed and intentional prompting's focus on process lies a spectrum of approaches that organisations are adopting:

1. **Permissive Acceptance**: maximising efficiency by broadly accepting AI outputs with minimal oversight
2. **Dismissive Rejection**: Maintaining traditional development methods with limited AI assistance
3. **Collaborative Integration**: Establishing defined processes where humans and AI work together with clear boundaries

The collaborative integration approach has gained traction in many professional environments, offering a middle ground that captures efficiency gains while maintaining code quality and developer growth.

3.7 Looking Ahead

As AI coding assistants continue to evolve, we can expect:

- Greater specialisation in domain-specific tools
- Better integration with development workflows
- Improved reasoning about complex systems
- Enhanced ability to understand, refactor, and maintain legacy code
- More personalised assistance based on your coding style and preferences

These advancements will not eliminate the need for skilled developers, but will continue to transform how we approach programming tasks and what skills are most valuable.

Chapter 4

Principles of Intentional Prompting

> The developer who understands the code is more valuable than the developer who generated it.

4.1 The Spectrum of AI Integration Approaches

Before diving into the core principles of intentional prompting, it's important to understand the broader landscape of approaches to integrating AI into development workflows. These approaches reflect different philosophies about the role of AI and human developers.

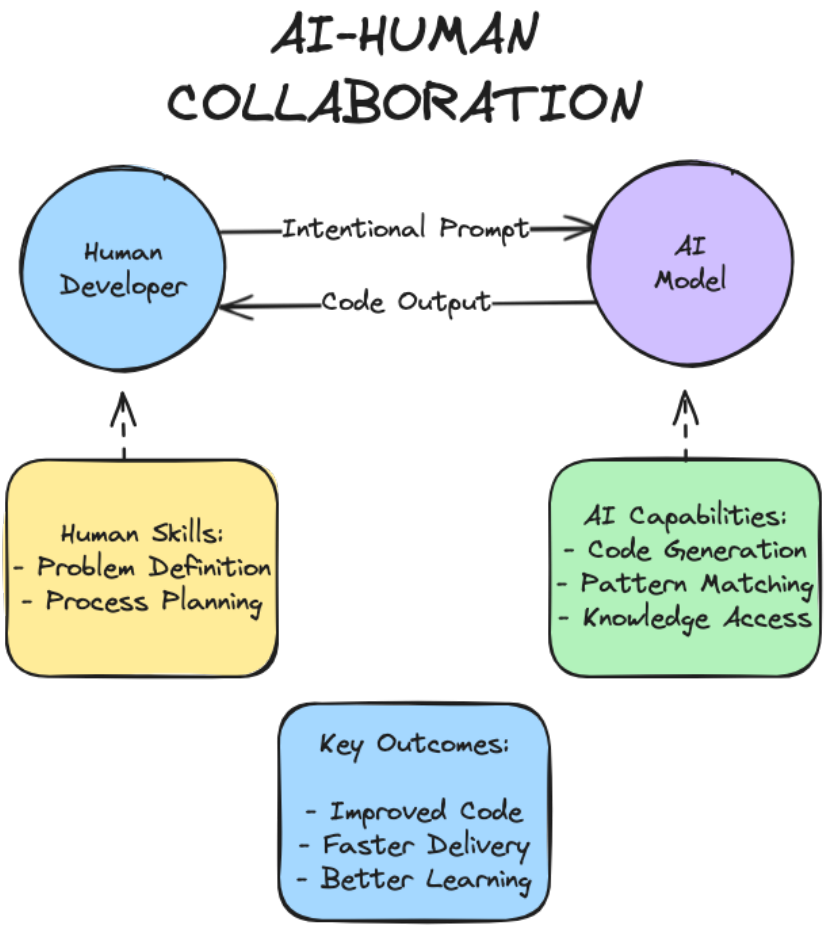

Figure 4.1: AI-Human Collaboration

4.1.1 Permissive Approach: "Just Take the Answer"

The permissive approach involves accepting AI outputs with minimal human oversight or intervention. In this model, developers largely defer to AI-generated recommendations, content, and decisions.

Advantages: - Maximum efficiency and speed in completing tasks - Reduced cognitive load on humans - Quick access to AI capabilities without friction - Easier adoption for non-technical users

Limitations and Risks: - Potential propagation of AI errors or biases - Limited human learning and skill development - Reduced critical thinking and problem-solving practice - Overreliance may lead to degraded human

capabilities over time - Lack of contextual understanding in complex situations

This approach aligns closely with the "vibe coding" philosophy discussed in the previous chapter, prioritising speed and output over process and understanding.

4.1.2 Dismissive Approach: "Reject AI Outright"

At the opposite end of the spectrum is the dismissive approach, characterised by scepticism or outright rejection of AI tools. This stance prioritises traditional methods and maintains full human control.

Advantages: - Maintaining full human control and autonomy - Preserving traditional skills and methods - Avoiding risks associated with AI errors - Clear human accountability and ownership - Preserving jobs and roles that might otherwise be automated

Limitations: - Missing potential productivity and quality improvements - Requiring more human time and resources - Unnecessary repetitive or mechanical work for employees - Limited access to AI's data processing and pattern recognition capabilities

4.1.3 Collaborative Approach: "Human-Directed AI Assistance"

Between these extremes lies the collaborative approach, which views AI as a tool that enhances human capabilities without replacing human judgment. This middle-ground philosophy establishes a partnership where humans direct AI systems, critically evaluate their outputs, and maintain decision-making authority.

Advantages: - Combining human judgment with AI efficiency - Maintaining human oversight while leveraging AI strengths - Enabling iterative improvement through feedback - Preserving human agency and accountability - Creating opportunities for human upskilling alongside AI use

Challenges: - Requires more thought and time than pure acceptance - Demands both AI literacy and domain expertise - Needs more complex workflows and training - Requires clear frameworks for when to trust or question AI outputs

Intentional prompting sits firmly within this collaborative approach, viewing AI as a thoughtful assistant rather than a replacement for human judgment.

4.1.4 Comparing Intentional Prompting vs. Vibe Programming

To illustrate the differences in these approaches, the following table compares intentional prompting with vibe programming across several important dimensions:

Dimension	Intentional Prompting	Vibe Programming
Primary Goal	Understanding and skill development alongside output	Speed and output production
Development Process	Structured, methodical approach with defined steps	Rapid, conversational, minimal planning
Human Involvement	Human directs process and makes key decisions	Human describes desired outcome, AI leads implementation
Code Review	Thorough review with critical evaluation	Minimal review, focus on functionality over quality
Debugging Approach	Systematic investigation based on understanding	Trial-and-error, asking AI to fix issues
Learning Outcome	Progressive skill development and deeper understanding	Potentially limited technical growth, focus on prompt crafting
Code Quality	optimised for readability, maintainability, efficiency	Prioritises working functionality over optimality
Ideal Use Cases	Complex systems, educational contexts, enterprise software	Rapid prototyping, MVPs, personal projects, non-critical applications
Knowledge Transfer	Facilitates team understanding and knowledge sharing	May create knowledge silos where only original developer understands prompting approach
Maintenance	Designed for long-term maintenance and evolution	May present challenges for future maintenance or extension

This comparison isn't meant to suggest that one approach is universally superior to the other. Rather, each has contexts where it may be more appropriate. Intentional prompting offers significant advantages for educational settings, complex systems, team environments, and situations where code quality and maintainability are paramount.

4.1.5 Example: Different Approaches to the Same Challenge

To make these differences concrete, let's examine how the permissive (vibe coding) and collaborative (intentional prompting) approaches would handle the same programming challenge: creating a function to validate email addresses.

4.1.5.1 Permissive Approach (Vibe Coding)

Human: Write a function that validates email addresses.

AI *generates*:

```
def validate_email(email):
    import re
    pattern =
    r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(pattern, email))
```

Human: It's not working for some emails with plus signs. Fix it.

AI *generates updated code*:

```
def validate_email(email):
    import re
    pattern =
    r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(pattern, email))
```

Human: It's still not working for email+alias@domain.com. Can you fix it?

AI *revises again*:

```
def validate_email(email):
    import re
    # Complex pattern handling all edge cases
    pattern =
    r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(pattern, email))
```

The permissive approach emphasises speed but may involve several iterations of trial and error without deepening understanding of the problem or solution.

4.1.5.2 Collaborative Approach (Intentional Prompting)

Human: I need to create an email validation function. Before writing code, let's discuss what makes a valid email address and identify the key requirements.

AI: *Provides explanation of email format requirements and edge cases*

Human: Based on those requirements, let's brainstorm approaches. What are the trade-offs between using regular expressions versus parsing the parts individually?

AI: *Explains different approaches with their advantages and limitations*

Human: I'll choose the regex approach for simplicity. Let's write pseudocode first:

```
function validateEmail(email):
  if email is empty:
    return false
  if email doesn't contain exactly one @ symbol:
    return false
  split email into local and domain parts
  check if local part meets requirements
  check if domain part has at least one period and
  valid format
  return true if all checks pass
```

AI: *Provides feedback on pseudocode and suggests improvements*

Human: Now let's implement this in Python with a clear approach:

```python
def validate_email(email):
    """
    Validates an email address according to RFC
    5322 standards.

    Args:
        email: String to validate as an email
        address

    Returns:
        bool: True if the email is valid, False
        otherwise
    """
    import re

    # Basic pattern that handles most common email
    formats
```

```
    pattern =
    r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

    # Initial validity check
    if not email or not isinstance(email, str):
        return False

    return bool(re.match(pattern, email))
```

Human: Let's test this with various cases including edge cases: - user@example.com - user+alias@example.com - user.name@example.co.uk - user@sub.example.com - incomplete-email-without-domain\@ - incomplete-email-without-username\@example.com - user\@incomplete-domain

The collaborative approach takes longer initially but builds understanding, produces better-documented code, and addresses edge cases proactively rather than reactively.

This example illustrates how the different approaches affect not just the final code but the entire development process and learning experience.

4.2 Maintaining Human Agency

At the core of intentional prompting is the principle of human agency - the programmer remains the architect and decision-maker throughout the development process, not just a consumer of AI-generated solutions.

4.2.1 Directive vs. Delegative Approaches

There are two fundamental ways to interact with AI coding assistants:

Delegative Approach: Handing off problems entirely to the AI and accepting its solutions with minimal scrutiny or direction.

Directive Approach: Guiding the AI through a structured process, maintaining control over architecture and design decisions, and critically evaluating its outputs.

Intentional prompting emphasises the directive approach, where you:

- Break down problems before presenting them to AI
- Provide clear constraints and requirements
- Review and question AI-generated code
- Make deliberate decisions about when and how to incorporate AI suggestions

4.2.2 Techniques for Maintaining Agency

- **Frame the problem yourself** before asking the AI for help
- **Establish evaluation criteria** for solutions before generating them
- **Request multiple approaches** to avoid anchoring on the first solution
- **Question assumptions** in AI-generated code
- **Make final integration decisions** based on your understanding, not convenience

4.3 Understanding vs. Copying

A critical distinction in intentional prompting is the difference between understanding a solution and merely copying it. The goal is not just working code, but comprehension that builds long-term programming capabilities.

4.3.1 The "Black Box" Problem

When developers copy AI-generated code without understanding it, they create "black boxes" in their codebase - components they can't effectively debug, maintain, or explain. Over time, this leads to brittle systems and stunted professional growth.

4.3.2 The VET Check

A useful shorthand for this comes from *Conversation, Not Delegation*: the **VET framework**. Before accepting any AI-generated code, ask three questions:

- **Verify:** Does this code actually do what it claims? Run it, test it, check the output against known inputs.
- **Explain:** Can I explain how this works to a colleague? If I cannot walk through the logic, I do not understand it yet.
- **Test:** Does it hold up under pressure? Change a variable, try an edge case, feed it unexpected input.

You understand code when you can explain how it works, modify it confidently to handle new requirements, identify potential edge cases it might not handle, recognise its performance characteristics, and connect it to broader programming concepts. If any of those feel uncertain, you are not done yet.

4.3.3 Strategies for Building Understanding

- **Request explanations** of generated code

- **Ask "what if" questions** about edge cases or modifications
- **Trace through execution** with specific examples
- **Modify the code** to handle different scenarios
- **Compare different implementations** of the same functionality

4.4 Good at the Average, Bad at the Precise

There is a useful way to predict when AI will serve you well and when it will let you down. Think along two axes: **average versus precise**, and **small versus large**.

AI excels at producing convincing averages. Ask it to write a utility function and you get something that reads like competent code, because the model has absorbed millions of similar functions and can synthesise a plausible representative. But ask for something precise, a specific algorithm optimised for your exact constraints, a function that must handle a particular edge case in your domain, and the same mechanism becomes a liability. The model predicts what plausible code would look like, which is how you end up with functions that pass simple tests but fail on the cases that matter.

The second axis is scale. AI handles small, bounded tasks well. A single function, a focused prompt with clear constraints, plays to the model's strengths. But as tasks grow larger, with more interdependent components, conflicting requirements, and emergent complexity, the output starts to look coherent on the surface while falling apart in the details. The model will produce something that looks like a system architecture, but the dependencies will not hold and the design decisions will be inconsistent.

These axes interact. Being good at the average means the model will confidently produce plausible-looking large systems that fail on precise details. The convincing average scales badly with complexity.

Before using AI on any programming task, ask two questions: **How precise does this need to be?** and **How big is this?**

	Small	Large
Average	**Sweet spot.** Utility functions, boilerplate, starter code. Trust with light review.	**Plausible but brittle.** Looks like a working system, falls apart under testing.
Precise	**Workable with verification.** Specific algorithms, edge case handling. Test thoroughly.	**Danger zone.** Confident architecture that is wrong in subtle ways. Stay in the loop at every step.

The sweet spot tasks can often be delegated with a quick review. Everything else requires conversation, iteration, and your judgement as the developer who understands the actual requirements. This framework is explored in depth in *Conversation, Not Delegation* (Borck, 2025).

4.5 Process Over Output

Intentional prompting emphasises the development process rather than just the final code. This focus on process leads to better long-term outcomes in both code quality and developer growth.

4.5.1 Why Process Matters

- **Better architecture:** A thoughtful process leads to better-designed code
- **Fewer bugs:** Systematic approaches catch edge cases that rushed solutions miss
- **Easier maintenance:** Code developed through a clear process is typically more readable and maintainable
- **Knowledge transfer:** Process-focused development makes it easier to onboard others
- **Skill development:** Focusing on process builds transferable skills rather than point solutions

4.5.2 The Six-Step Process

The six-step programming methodology (which we'll explore in depth in Part 2) provides a structured process that works with or without AI assistance:

1. Restate the problem
2. Identify inputs and outputs
3. Work through examples by hand
4. Write pseudocode
5. Convert to working code
6. Test thoroughly

4.5.3 Integrating AI Into Your Process

Rather than replacing your process, AI should enhance it: - Use AI to explore problem variations during problem restatement - Generate test cases during input/output identification - Verify your manual examples - Suggest and refine pseudocode - Help convert pseudocode to working implementations - Generate comprehensive test cases

4.6 The Learning Mindset

Intentional prompting is fundamentally about continuous learning rather than just task completion. This mindset treats every programming challenge as an opportunity to deepen understanding and build skills.

4.6.1 Characteristics of a Learning Mindset

- **Curiosity** about how and why solutions work
- **Comfort with not knowing** everything immediately
- **Desire to understand deeply** rather than just solve the immediate problem
- **Willingness to explore alternatives** even after finding a working solution
- **Reflection on the development process** to improve future approaches

4.6.2 AI as a Learning Partner

When approached with a learning mindset, AI assistants become powerful learning tools: - Use AI to explore concepts you don't fully understand - Ask AI to compare different approaches and explain tradeoffs - Request explanations of unfamiliar code patterns - Use AI to find gaps in your understanding - Challenge AI-generated solutions to deepen your own thinking

4.6.3 Intentional Learning Techniques

- **Concept exploration:** Ask the AI to explain concepts in multiple ways
- **Implementation comparison:** Request different implementations of the same functionality
- **Knowledge testing:** Explain a concept to the AI and ask for feedback
- **Deliberate challenge:** Introduce constraints that force exploration of new approaches
- **Reflective questioning:** Ask "why" questions about code decisions

4.7 Ethical Considerations

Intentional prompting includes ethical considerations about the use of AI in the development process.

4.7.1 Attribution and Transparency

- Be transparent about AI contributions to your code
- Understand your organisation's policies about AI-assisted development
- Consider adding attribution comments for significant AI contributions
- Maintain clear documentation of human design decisions

4.7.2 Security and Quality Responsibility

- Always review AI-generated code for security vulnerabilities
- Never delegate final quality assurance to AI tools
- Maintain awareness of common security issues in AI-generated code
- Establish clear review processes for AI-assisted development

4.7.3 Bias and Fairness

- Be aware that AI tools may embed biases from their training data
- Review code for fairness issues, especially in user-facing features
- Consider diverse perspectives when evaluating AI-generated solutions
- Question assumptions that might embed problematic patterns

4.7.4 Professional Development Balance

- Balance efficiency gains from AI with skill development needs
- Identify core skills you want to strengthen, even with AI assistance
- Create intentional learning projects where you limit AI assistance
- Use AI to stretch beyond your current capabilities rather than stay within them

4.8 Addressing Common Concerns and Resistance to LLMs

Despite their utility, Large Language Models face resistance from many programmers and educators. Some of these concerns have deep historical roots, while others arise from the unique characteristics of modern LLMs. Understanding and addressing these concerns is essential for effective adoption of intentional prompting.

4.8.1 The Ambiguity of Natural Language

As far back as 1977, computer scientist Edsger W. Dijkstra presented a compelling critique of natural language programming in his essay "On

the foolishness of 'natural language programming' ". Dijkstra argued that programming fundamentally requires "the care and accuracy that is characteristic for the use of any formal symbolism" and that the inherent ambiguity of natural language made it unsuitable for the precision required in programming.

Dijkstra's concerns were valid for his time, but modern LLM-based approaches offer new possibilities through iterative refinement processes:

- **Progressive disambiguation** - Iterative approaches provide mechanisms to gradually eliminate ambiguities through multiple rounds of interaction, transforming imprecise natural language into precise formal representations
- **Structured frameworks** - Methodologies like intentional prompting add structure to otherwise ambiguous interactions
- **Human-in-the-loop validation** - The human programmer validates outputs and maintains final authority over implementation decisions

The intentional prompting methodology directly addresses Dijkstra's concerns by providing a framework that bridges natural language and formal code, using the former as an entry point rather than a replacement for the latter.

4.8.2 The Non-Deterministic Nature of LLMs

Another significant concern involves the non-deterministic behaviour of LLMs, specifically the fact that they can produce different outputs even when given the same input. This unpredictability raises legitimate questions about reliability, especially in mission-critical applications.

Research has documented significant variations in LLM performance across multiple runs, with accuracy varying up to 15% and gaps between best and worst performance reaching as high as 70%. This variability stems from:

1. **Input interpretation variability** - LLMs may interpret the same natural language prompt differently across different runs
2. **Output generation variability** - Even with the same interpretation, the code generated may vary due to sampling methods

The intentional prompting methodology addresses these concerns through:

- **Structured evaluation** - The methodology provides clear criteria for evaluating generated code
- **Explicit testing** - Step 6 (Test with Data) ensures thorough validation of any generated solution

- **Human oversight** - The human programmer maintains control over the development process, reviewing and modifying generated code as needed
- **Iterative refinement** - The methodology embraces multiple iterations to converge on reliable solutions

4.8.3 Procedural Knowledge Transfer

Interestingly, LLMs excel at procedural tasks because they've inherited human problem-solving patterns through their training data. Research shows that LLMs have absorbed procedural knowledge through exposure to:

- Error-checking protocols from technical manuals
- Creative iteration cycles in writing samples
- Mathematical proof structures in STEM literature
- Software engineering best practices from code repositories

This explains why models respond well to structured methodologies like intentional prompting: they're activating latent procedural knowledge that mirrors human problem-solving approaches.

The six-step methodology leverages this characteristic by providing a framework that:

- Activates the model's latent understanding of systematic problem-solving
- Provides clear procedural guidance that aligns with effective human workflows
- Creates a shared procedural language between human and AI

4.8.4 Data Privacy: Real Risks vs Perceived Risks

A common concern among developers and organisations is that code or proprietary logic pasted into AI tools might be extracted by other users. This fear is largely unfounded and worth understanding clearly.

When you paste code into an AI tool, it is processed and a response is generated. Your code is not stored as a retrievable file inside the model. If your data is used for training at all (enterprise tiers typically exclude it), it becomes a vanishingly small statistical signal distributed across billions of parameters. Reconstructing your specific code from those parameters is not a realistic attack. Jailbreaking an LLM means manipulating its behaviour, not accessing a database of other users' sessions. And remember: LLMs interpolate, they do not retrieve. There is no mechanism by which another user could query the model and get your code back, because the model never stored it as a file in the first place.

You will also encounter convergent development mistaken for theft. "I was building a feature using Copilot, and then a competitor released something very similar." Thousands of developers are solving the same problems with the same tools, reading the same documentation, responding to the same market needs. Independent arrival at similar solutions is how software development normally works.

The risks that are real and worth managing: pasting API keys, credentials, or access tokens into any external tool is an immediate security exposure. Pasting personally identifiable data or data subject to regulation is a genuine compliance issue. And accepting AI-generated code without testing or understanding it is a professional liability, though that is a verification problem, not a data leakage problem.

The "non-zero risk means do not use it" stance deserves scrutiny. Every development tool involves trade-offs. Git repositories can be compromised. Dependencies can be supply-chain attacked. We manage these risks through practice and policy, not prohibition. Developers and organisations that refuse to engage with AI because of overestimated data risks face a different set of consequences: slower development, skills that stagnate, and a widening gap with peers who learned to use the tools thoughtfully.

4.8.5 Finding the Right Balance

The most effective approach to LLM integration lies in finding the right balance between permissive acceptance and dismissive rejection. The intentional prompting methodology represents this balanced middle ground:

- It acknowledges LLMs' limitations regarding ambiguity and non-determinism
- It establishes guardrails through a structured methodology
- It leverages LLMs' strengths in pattern recognition and procedural knowledge
- It maintains human agency and oversight throughout the development process

By addressing these concerns directly and providing a structured framework for human-AI collaboration, intentional prompting offers a pragmatic approach that captures the benefits of AI assistance while mitigating its risks.

4.9 Putting Principles Into Practice

These principles (maintaining agency, understanding vs. copying, process over output, the learning mindset, ethical considerations, and addressing common concerns) form the foundation of intentional prompting.

Part III

Methodology

Chapter 5

The Six-Step Programming Methodology

> A methodology is not a constraint. It is a way of thinking that transfers to problems you have not seen yet.

5.1 Overview of the Methodology

The six-step programming methodology provides a structured approach to solving problems that works both with and without AI assistance. This methodology focuses on understanding before implementation and ensures that programmers maintain control of the development process.

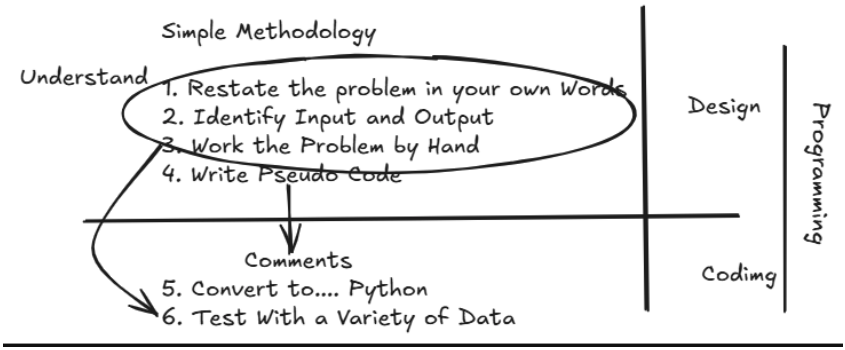

Figure 5.1: Six-Step Methodology Diagram

The methodology divides programming into two distinct phases:

5.1.1 Design (Thinking) Phase - Steps 1-4

This phase focuses on understanding and planning before any actual coding begins:

1. **Restate the problem** in your own words
2. **Identify input and output** requirements
3. **Work the problem by hand** with concrete examples
4. **Write pseudocode** to outline the solution strategy

5.1.2 Coding (Doing) Phase - Steps 5-6

Only after completing the design phase do we move to implementation:

5. **Convert to code** (with AI assistance as appropriate)
6. **Test with a variety of data**, especially edge cases

This clear separation between thinking and doing is crucial. The pseudocode created in Step 4 serves as a bridge between these phases, transforming into comments that guide implementation in Step 5.

5.1.3 The Hidden Structure of the Methodology

While the six steps appear sequential, the methodology contains several important hidden elements:

1. **Meaningful Divisions**: The separation between Design and Coding reflects different modes of thinking - first conceptual, then practical.

2. **Bridging Elements**: Pseudocode serves as both the final product of the design phase and the starting framework for the coding phase.

3. **Feedback Loops**: The methodology is inherently iterative, with testing potentially sending you back to earlier steps for refinement.

4. **Understanding to Doing**: The methodology follows a natural progression from abstract understanding to concrete implementation.

This methodology isn't revolutionary on its own. It draws from established problem-solving approaches and computational thinking practices like Waterfall, Agile, and Test-Driven Development. What makes it valuable in the age of AI is how it provides a framework for intentional prompting and collaboration with AI assistants while maintaining human understanding and control.

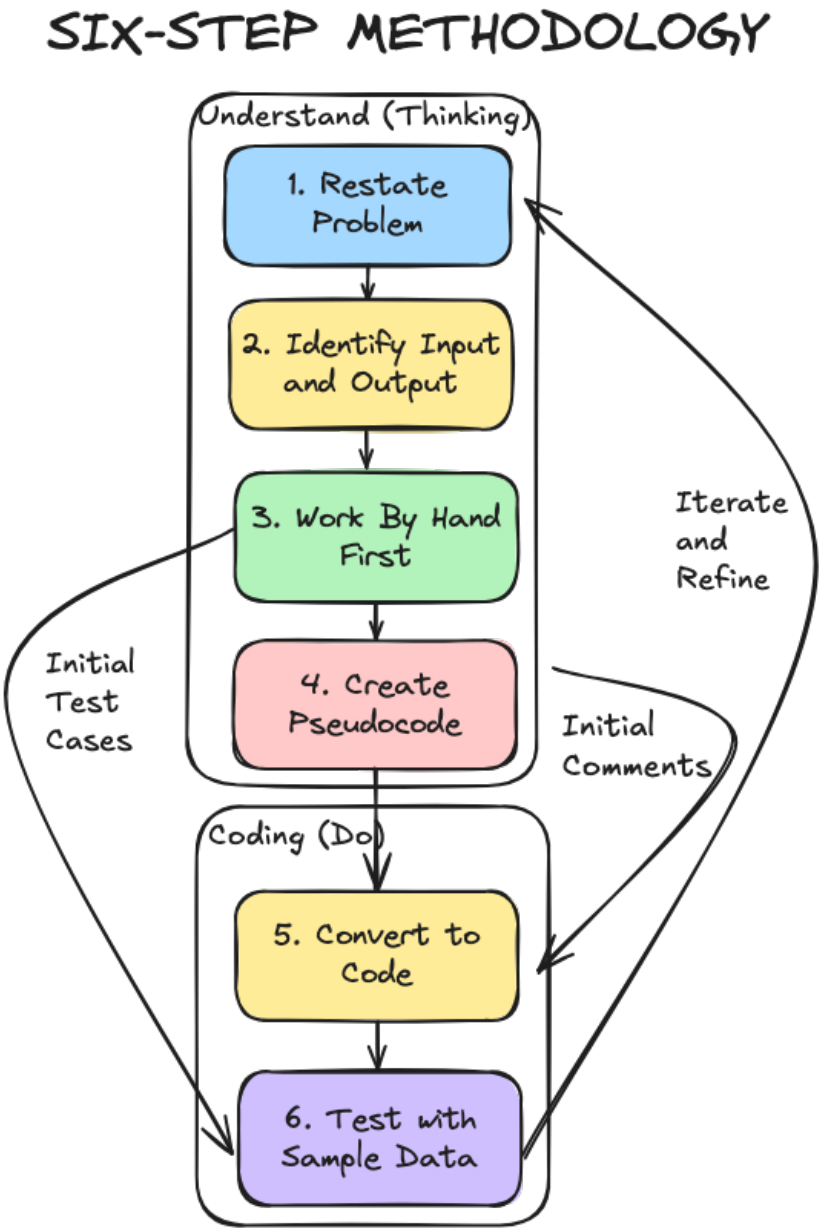

Figure 5.2: Detailed Six-Step Methodology

5.1.4 Adapting to Your Preferred Methodology

It's important to note that we're presenting this simplified methodology as a concrete example of how to structure AI-assisted development, not as the only valid approach. The six-step methodology serves as a template that demonstrates key principles of intentional prompting, which you can adapt to your preferred development methodology or domain.

5.1.4.1 Integration with Other Development Approaches

These principles can be integrated into various development methodologies:

- **Agile**: Use the problem understanding and design steps during sprint planning, with implementation and testing occurring within sprint cycles
- **Test-Driven Development**: Enhance the TDD cycle by adding explicit problem restatement and planning before writing tests
- **Domain-Driven Design**: Incorporate intentional prompting to help define the ubiquitous language and domain models

5.1.4.2 customising for Your Context

When adapting these concepts to your situation, consider:

1. **Identify key decision points** where human understanding and direction are crucial
2. **Establish clear boundaries** for AI assistance within your workflow
3. **Create explicit steps** for critical evaluation of AI outputs
4. **Define integration points** where AI-generated components connect with human-directed architecture

The core value proposition (maintaining human understanding while leveraging AI capabilities) remains valid regardless of which specific methodology you prefer. What's most important is having a structured approach to human-AI collaboration rather than following these exact six steps verbatim.

Use this methodology as inspiration to develop your own intentional prompting workflow that aligns with your team's practices, domain requirements, and development philosophy while preserving the principles of human direction and understanding.

5.2 How AI Changes the Process

AI coding assistants change the dynamics of this methodology in several important ways:

5.2.1 Acceleration and Exploration

AI can accelerate many steps in the process, allowing for: - Rapid exploration of problem variations (Step 1) - Comprehensive consideration of edge cases (Step 2) - Verification of manual solutions (Step 3) - Generation of alternative pseudocode approaches (Step 4) - Quick implementation of solutions (Step 5) - Comprehensive test case generation (Step 6)

5.2.2 The Temptation to Skip Steps

With powerful AI tools, there's a temptation to skip directly to implementation (Step 5). This is the core challenge that intentional prompting addresses: maintaining a disciplined approach even when AI makes shortcuts possible.

5.2.3 New Capabilities and Responsibilities

AI introduces new capabilities at each step: - Discussing problem nuances in natural language - Generating edge cases you might not have considered - Checking your manual approach for errors - Suggesting implementation strategies you might not know - Implementing complex algorithms efficiently - Testing boundary conditions systematically

These capabilities create new responsibilities for the programmer: - Critically evaluating AI suggestions - Maintaining awareness of the problem's full context - Verifying that AI-generated code meets requirements - Understanding the code that ultimately gets deployed

5.3 Benefits of a Structured Approach

Following a structured methodology with AI assistance offers several important benefits:

5.3.1 Better Code Quality

By working through all six steps rather than jumping straight to implementation: - Architectural issues are identified earlier - Edge cases are handled more comprehensively - Solutions are more robust and maintainable - The development process becomes more consistent

5.3.2 Enhanced Learning

The structured approach promotes deeper learning: - You develop a habit of understanding problems before coding - You practice computational thinking independent of specific languages - You build transferable problem-solving skills - You learn to evaluate solutions critically

5.3.3 Appropriate AI Integration

The methodology creates natural integration points for AI assistance: - Each step has clear inputs and outputs for AI collaboration - The division of responsibility between human and AI is clarified - You maintain control over the development process - AI complements rather than replaces your thinking

5.3.4 Reduced Technical Debt

Following all six steps helps avoid the accumulation of technical debt: - Solutions are designed rather than just implemented - Edge cases are addressed proactively - Code is tested systematically - Implementations are understood rather than just accepted

5.4 Case Study: Solving a Problem With and Without AI

To illustrate the methodology in action, let's consider a practical programming problem:

> Create a function that finds the longest palindromic substring in a given string.

5.4.1 Approach 1: Delegating to AI (Without Methodology)

A common approach is to simply ask the AI directly:

```
"Write a function to find the longest palindromic
substring in a string."
```

The AI might produce this solution:

```python
def longest_palindromic_substring(s):
    if not s:
        return ""

    start = 0
    max_len = 1

    for i in range(len(s)):
        # Check for even length palindromes
        left, right = i, i + 1
        while left >= 0 and right < len(s) and
        s[left] == s[right]:
```

```
            if right - left + 1 > max_len:
                start = left
                max_len = right - left + 1
            left -= 1
            right += 1

        # Check for odd length palindromes
        left, right = i - 1, i + 1
        while left >= 0 and right < len(s) and
        s[left] == s[right]:
            if right - left + 1 > max_len:
                start = left
                max_len = right - left + 1
            left -= 1
            right += 1

    return s[start:start + max_len]
```

This code may work, but: - Do you understand how it works? - Could you modify it if requirements changed? - Are you confident it handles all edge cases? - Do you know its performance characteristics?

5.4.2 Approach 2: The Six-Step Methodology with AI Assistance

5.4.2.1 Step 1: Restate the Problem

Human prompt:

```
I need to write a function that finds the longest
palindromic substring in a given string.
Let me restate this problem to make sure I
understand it correctly:

I need to find the longest continuous sequence of
characters within a string that reads
the same forward and backward. For example, in
"babad", the longest palindromic substring
would be "bab" or "aba" (both are valid since
they're the same length).

Is this understanding correct? Are there any edge
cases I should consider?
```

AI response:

```
Your understanding is correct! The function should
```

```
find the longest substring that is a palindrome.

Edge cases to consider:
- Empty strings
- Strings with no palindromes longer than 1
character
- Strings where the entire string is a palindrome
- Strings with multiple palindromic substrings of
the same length
- Very long strings (performance considerations)
- Case sensitivity (usually palindromes are
case-sensitive)
```

5.4.2.2 Step 2: Identify Inputs and Outputs

Human prompt:

```
Let me specify the input and output for this
function:

Input:
- A string of characters (possibly empty)

Output:
- The longest palindromic substring within the
input string
- If there are multiple palindromic substrings
with the same maximum length, returning any one of
them is acceptable
- If the string is empty, return an empty string
- If there are no palindromes longer than 1
character, return the first character of the
string

Do these input and output specifications cover all
the necessary cases?
```

AI response:

```
Those specifications are good, but let me suggest
a small clarification:

For the case where there are no palindromes longer
than 1 character, technically every individual
character is a palindrome by itself. So returning
the first character would be correct, but it might
be clearer to explicitly state "return any single
```

```
character" since any character is a palindrome of
length 1.

With that clarification, your input and output
specifications cover all the necessary cases!
```

5.4.2.3 Step 3: Work the Problem by Hand

Human work:

Let's manually trace through a simple example: the string "babad".

To find palindromes: 1. Start with each position as potential center of palindrome 2. Expand outward while characters match

For "babad": - Center at 'b' (index 0): Just 'b' (length 1) - Center at 'a' (index 1): 'bab' (length 3) - Center at 'b' (index 2): 'bab' (length 3) - Center at 'a' (index 3): 'aba' (length 3) - Center at 'd' (index 4): Just 'd' (length 1)

Also check even-length palindromes: - Between 'b' and 'a' (0-1): None - Between 'a' and 'b' (1-2): None - Between 'b' and 'a' (2-3): None - Between 'a' and 'd' (3-4): None

Longest palindromes: "bab" or "aba" (both length 3)

Human prompt:

```
I've worked through the example "babad" by hand
and found that the longest palindromic
substrings are "bab" and "aba", both of length 3.

My approach was to check each position as a
potential center of a palindrome and expand
outward while the characters match. I also checked
for even-length palindromes that have
their center between two characters.

Does this approach seem correct and efficient? Are
there any algorithmic improvements
I should consider?
```

AI response:

```
Your approach is correct and is actually the
standard solution for this problem with O(n²) time
complexity and O(1) space complexity.

The two key components you've identified are
essential:
```

```
1. Checking odd-length palindromes (with a single
character as the center)
2. Checking even-length palindromes (with the
center between two characters)

There are some more complex algorithms like
Manacher's algorithm that can solve this in O(n)
time, but they're significantly more complex and
typically unnecessary unless you're dealing with
very long strings where performance is critical.

Your approach is intuitive and efficient for most
practical purposes!
```

5.4.2.4 Step 4: Write Pseudocode

Human prompt:

```
Based on my manual approach, let me write
pseudocode for finding the longest palindromic
substring:

function longestPalindromicSubstring(string s):
    if s is empty, return ""

    start = 0
    maxLength = 1

    function expandAroundCenter(left, right):
        while left >= 0 and right < length of s
              and s[left] equals s[right]:
            if (right - left + 1) > maxLength:
                maxLength = right - left + 1
                start = left
            left = left - 1
            right = right + 1

    for i from 0 to length of s - 1:
        expandAroundCenter(i, i)
        expandAroundCenter(i, i + 1)

    return substring of s from start
           to start + maxLength

Does this pseudocode correctly implement the
approach we discussed?
```

AI response:

```
Your pseudocode looks correct and implements the
approach we discussed! It handles both odd and
even length palindromes by expanding around
different centers.

There's just one small detail to be careful about:
in your expandAroundCenter function, when you
update maxLength and start, make sure you're
considering the current palindrome (not a future
one). So you should check if (right - left + 1) >
maxLength before expanding further.

Other than that, this pseudocode correctly
captures the solution and is ready to be converted
to actual code!
```

5.4.2.5 Step 5: Convert to Code

Human prompt:

```
Now I'll convert my pseudocode to Python:

```python
def longest_palindromic_substring(s):
    if not s:
        return ""

    start = 0
    max_length = 1

    def expand_around_center(left, right):
        nonlocal start, max_length
        while left >= 0 and right < len(s) and
        s[left] == s[right]:
            current_length = right - left + 1
            if current_length > max_length:
                max_length = current_length
                start = left
            left -= 1
            right += 1

    for i in range(len(s)):
        # Odd length palindromes
        expand_around_center(i, i)
```
```

```
        # Even length palindromes
        expand_around_center(i, i+1)

    return s[start:start + max_length]
```

Can you review this code for correctness
and suggest any improvements?

AI response:

Your Python implementation looks correct and follows your pseudocode closely! It will find the longest palindromic substring as required.

A few small observations:

1. You've correctly used `nonlocal` to update the `start` and `max_length` variables from within the nested function.

2. The algorithm handles both odd and even length palindromes properly by starting from different center positions.

3. The time complexity is $O(n^2)$ and space complexity is $O(1)$, which is efficient for most inputs.

4. Your code handles the empty string case correctly.

The implementation is clean and efficient. The only possible optimisation would be implementing Manacher's algorithm for linear time complexity, but that's typically unnecessary unless you're dealing with extremely large strings.

5.4.2.6 Step 6: Test with Data

Human prompt:

Let's test my function with various test cases:

1. Normal case: "babad" → Should return "bab" or "aba"

```
2. Entire string is a palindrome: "racecar" →
Should return "racecar"
3. Empty string: "" → Should return ""
4. Single character: "a" → Should return "a"
5. No palindromes longer than 1: "abc" → Should
return "a" (or "b" or "c")
6. Multiple palindromes of same length: "aabbaa" →
Should return "aabbaa"
7. Even-length palindrome: "abba" → Should return
"abba"

Can you help me verify if my implementation
handles all these cases correctly?
```

AI response:

```
Let's trace through your implementation for each
test case:

1. `"babad"`:
   - For center at 'b' (index 0): Just 'b' (length
   1)
   - For center at 'a' (index 1): 'bab' (length 3)
   - For center at 'b' (index 2): 'bab' (length 3)
   - For center at 'a' (index 3): 'aba' (length 3)
   - For center at 'd' (index 4): Just 'd' (length
   1)
   - The first palindrome of length 3 found is
   "bab"

2. `"racecar"`:
   - Expands from middle 'e' to full string
   "racecar"

3. `""`:
   - Immediate return of empty string

4. `"a"`:
   - Single character is a palindrome of length 1


5. `"abc"`:
   - No palindromes longer than 1, returns first
   character "a"

6. `"aabbaa"`:
```

```
   - Full string is palindrome

7. `"abba"`:
   - Checking even centers finds "abba"

Your implementation handles all these test cases
correctly! It will return the expected outputs for
each scenario.
```

5.4.3 Comparing the Approaches

The six-step approach with AI assistance: - Built a clear understanding of the problem - Identified edge cases explicitly - Traced through examples manually - Developed the solution systematically - Resulted in a function the programmer fully understands - Verified handling of important test cases

This approach might take more time initially, but it produces: - A deeper understanding of the problem - Confidence in the solution's correctness - The ability to modify the solution when needed - A stronger foundation for solving similar problems in the future

5.5 The Methodology in Daily Practice

While the full six-step methodology is valuable for learning and complex problems, it can be adapted for daily use:

5.5.1 Quick Reference Version

1. **Restate:** What am I really trying to solve?
2. **I/O:** What goes in, what comes out?
3. **Example:** Try a simple case by hand
4. **Plan:** Outline the solution strategy
5. **Code:** Implement with appropriate AI assistance
6. **Test:** Verify with varied test cases

5.5.2 When to Be More Thorough

Apply the full methodology more carefully when: - Learning new concepts - Solving complex problems - Implementing critical components - Working with performance-sensitive code - Teaching or mentoring others

5.5.3 When to Streamline

For familiar or simpler problems, you might: - Combine steps 1 and 2 into a quick problem analysis - Skip step 3 for very familiar problem patterns -

Use AI to help generate pseudocode more quickly - Focus more heavily on thorough testing

5.6 Key Takeaways

The six-step methodology provides: - A structured framework for problem-solving - Natural integration points for AI assistance - Maintenance of human understanding and control - A balance between efficiency and depth

In the next six chapters, we'll explore each step of the methodology in detail, examining how AI assistance can be effectively integrated into each part of the process.

Chapter 6

Restate and Identify (Steps 1-2)

> The quality of your solution is determined before you write a single line of code.

The first two steps of the six-step methodology, restating the problem and identifying inputs and outputs, set the foundation for everything that follows. This critical preparation phase ensures you fully understand what you're trying to solve before diving into implementation.

6.1 Techniques for Problem Restatement

6.1.1 Why Restatement Matters

Restating a problem in your own words serves several important purposes:

1. **Clarifies understanding:** The process of reformulating forces you to grapple with the problem's core elements
2. **Identifies ambiguities:** Areas you struggle to restate clearly often reveal hidden assumptions or unclear requirements
3. **Provides ownership:** Restating moves you from passive recipient to active problem-solver
4. **Creates a reference point:** Your restatement becomes a touchstone to check your solution against

6.1.2 Effective Restatement Approaches

6.1.2.1 The ELI5 Method (Explain Like I'm Five)

Simplify the problem to its most fundamental elements, as if explaining to someone with no technical background:

Original: "Implement a function to find the shortest path between two nodes in a weighted directed graph using Dijkstra's algorithm."

ELI5 Restatement: "I need to find the fastest route between two points on a map where some routes are quicker than others."

6.1.2.2 The Components Breakdown

Identify and separate the main components of the problem:

Original: "Build a cache that stores the results of expensive function calls and returns the cached result when the same inputs occur again."

Components Restatement: - "I need to store function results along with their inputs" - "When a function is called, I need to check if we've seen these inputs before" - "If we have seen the inputs, return the stored result instead of recalculating" - "If we haven't seen the inputs, calculate, store, and return the result"

6.1.2.3 The Constraints Highlight

Explicitly call out all constraints and requirements:

Original: "Create a username validation function for a web form."

Constraints Restatement: - "Usernames must be 3-20 characters long" - "Only alphanumeric characters, hyphens, and underscores are allowed" - "Cannot start or end with a hyphen or underscore" - "Must be case-insensitive (uppercase/lowercase doesn't matter)" - "Cannot contain reserved keywords from this list…"

6.1.2.4 The Inverse Problem

State what the solution is NOT supposed to do:

Original: "Implement user authentication for the application."

Inverse Restatement: - "This is NOT about creating user accounts or registration" - "This is NOT about password reset functionality" - "This is NOT about authorization (what users can do)" - "This IS about verifying a user's identity based on credentials"

6.1.3 Using AI to Enhance Restatement

AI assistants can help refine your problem restatements in several ways:

6.1.3.1 Restatement Verification

After restating the problem yourself, ask the AI to verify your understanding:

```
I'm working on this problem: [original problem
statement]

I've restated it as: [your restatement]

Have I missed anything important or misunderstood
any aspect of the problem?
```

6.1.3.2 Assumptions Identification

AI can help uncover hidden assumptions in the problem:

```
What assumptions might I be making about this
problem that aren't explicitly stated?
```

6.1.3.3 Edge Case Exploration

AI can help identify boundary conditions and special cases:

```
What edge cases should I consider for this
problem?
```

6.1.3.4 Domain-Specific Context

For problems in unfamiliar domains, AI can provide context:

```
Could you explain any domain-specific
considerations I should keep in mind when solving
this problem?
```

6.2 Clearly Defining Inputs and Outputs

The second step of the methodology focuses on establishing precise expectations for what goes into your solution and what comes out.

6.2.1 Why Input/Output Specification Matters

Clearly defining inputs and outputs:

1. **Creates a contract:** Establishes exactly what your solution promises to deliver
2. **Guides testing:** Forms the basis for comprehensive test cases
3. **Simplifies debugging:** Makes it clear when the solution is working incorrectly
4. **Facilitates integration:** Makes it easier to incorporate your solution into larger systems

6.2.2 Elements of Complete I/O Specifications

6.2.2.1 For Inputs:

- **Data types:** The expected types of each input parameter
- **Value ranges:** Acceptable ranges or constraints for input values
- **Required vs. optional:** Which inputs can be omitted with default values
- **Format requirements:** Any specific formatting expected (e.g., date formats)
- **Size constraints:** Limitations on input size (e.g., maximum string length)

6.2.2.2 For Outputs:

- **Return type:** The data type of the returned value
- **Success indicators:** How successful operation is communicated
- **Error handling:** How errors are communicated (exceptions, error codes, etc.)
- **Special values:** Any special return values and their meaning
- **Side effects:** Any state changes or external effects besides the return value

6.2.3 Documenting I/O With Examples

Examples can clarify I/O specifications better than abstract descriptions:

```
Function: calculate_shipping_cost(weight,
distance, express=False)

Inputs:
- weight: A positive float representing item
weight in kilograms
- distance: A positive float representing distance
in kilometers
- express: Optional boolean flag for express
shipping (default: False)
```

```
Outputs:
- A float representing the shipping cost in
dollars
- If input parameters are invalid, raises
ValueError

Examples:
- calculate_shipping_cost(2.5, 100) → 12.50
- calculate_shipping_cost(2.5, 100, express=True)
→ 25.00
- calculate_shipping_cost(-1, 100) →
ValueError("Weight must be positive")
```

6.2.4 Using AI to Explore Edge Cases

AI assistants excel at identifying edge cases and boundary conditions that might affect your I/O specification.

6.2.4.1 Input Edge Cases

Ask the AI to help identify input edge cases:

```
What are potential edge cases for these inputs?
- weight: A positive float representing item
weight in kilograms
- distance: A positive float representing distance
in kilometers
- express: Optional boolean flag for express
shipping
```

This might reveal considerations like: - Zero weight or distance - Extremely large weights or distances - Non-numeric inputs - Negative values - Special shipping conditions

6.2.4.2 Output Edge Cases

Similarly, explore edge cases for outputs:

```
What are potential edge cases for the output of
this shipping cost calculator?
```

This might reveal considerations like: - Minimum shipping costs - Free shipping thresholds - Rounding rules - Currency formatting - Maximum possible costs

6.2.4.3 Error Condition Exploration

Use AI to explore potential error conditions:

```
What error conditions should I handle in this
shipping cost calculator?
```

This might reveal considerations like: - Invalid input types - Values outside acceptable ranges - Unavailable shipping routes - Rate limit restrictions

6.3 Prompting Techniques for Requirement Clarification

When requirements are unclear or incomplete, AI can help explore the problem space through directed prompting.

6.3.1 The Five Whys Technique

Repeatedly ask "why" to dig deeper into requirements:

```
The requirement is to "implement a user profile
page."

Why does the user need a profile page?
[AI response]

Why is that important for the user?
[AI response]

Why can't we achieve that with the existing
functionality?
```

This technique helps uncover the underlying goals behind requirements.

6.3.2 Stakeholder Perspective Prompting

Ask the AI to consider requirements from different perspectives:

```
How might different users view this profile page
requirement differently?

- From an end user's perspective: ...
- From an administrator's perspective: ...
- From a security perspective: ...
- From a performance perspective: ...
```

6.3.3 Requirement Expansion

Ask the AI to expand terse requirements into detailed specifications:

```
The requirement is: "Add sorting functionality to
the user table."

Can you expand this into a comprehensive
specification including:
- Which columns should be sortable
- Default sort order
- How sort state should be preserved
- Performance considerations for large datasets
- UI indicators for sort direction
```

6.3.4 Comparative Analysis

Use AI to compare different interpretations of ambiguous requirements:

```
The requirement "ensure the application is
responsive" could be interpreted in two ways:
1. The UI should adapt to different screen sizes
2. The application should respond quickly to user
interactions

Can you compare these interpretations and their
implications for implementation?
```

6.4 Practical Exercise: Restate and Identify in Action

Let's apply Steps 1 and 2 to a sample problem:

> Original Requirement: "Create a function that converts dates between different formats."

6.4.1 Step 1: Restate the Problem

Using Components Breakdown:

- "I need to parse dates from various input formats"
- "I need to convert parsed dates into specified output formats"
- "I need to handle potential errors in the input formats"
- "I need to ensure the conversion preserves the same date and time"

Using Constraints Highlight:

- "The function should accept a string representing a date"
- "The function should accept a source format specification"
- "The function should accept a target format specification"
- "The function should return the date in the target format"

- "The function should raise appropriate errors for invalid inputs"

AI Verification Prompt:

```
I'm working on a function that converts dates
between different formats.

I've restated it as:
- A function that takes a date string, source
format, and target format
- Parses the date according to the source format
- Returns the same date formatted according to the
target format
- Handles errors for invalid dates or formats

Have I missed anything important in my
restatement?
```

AI Response:

```
Your restatement covers the main aspects well, but
here are a few additional considerations:

- Time zones: How should the function handle time
zone information? Should it preserve, convert, or
ignore time zones?
- Locale-specific formatting: Dates can be
formatted differently in different locales (e.g.,
month names in different languages)
- Ambiguous dates: How to handle dates like
"01/02/2023" which could be January 2nd or
February 1st depending on region
- Partial dates: Should the function support dates
with missing components (e.g., year only, month
and year only)?
- Format specifications: Will you use standard
format strings (like those in Python's datetime),
or custom specifications?

These considerations can greatly affect the scope
and complexity of your function.
```

6.4.2 Step 2: Identify Input and Output

Input Specification: - `date_str`: A string representing a date in the source format - `source_format`: A string specifying the format of the input date - `target_format`: A string specifying the desired output format

- `timezone`: (Optional) A string representing the timezone of the input date

Output Specification: - On success: A string representing the same date in the target format - On error: Raises appropriate exceptions with clear error messages

Examples: - `convert_date("2023-04-01", "%Y-%m-%d", "%m/%d/%Y")` → `"04/01/2023"` - `convert_date("April 1, 2023", "%B %d, %Y", "%Y-%m-%d")` → `"2023-04-01"` - `convert_date("invalid", "%Y-%m-%d", "%m/%d/%Y")` → `ValueError("Unable to parse date")`

Edge Case Exploration Prompt:

```
What edge cases should I consider for a date
format conversion function with these inputs?
- date_str: A string representing a date
- source_format: The format of the input date
- target_format: The desired output format
```

AI Response:

```
Here are important edge cases to consider:

**Input Date String Edge Cases:**
- Empty string
- Dates with two-digit years (ambiguity between
centuries)
- Dates with missing components (e.g., no year
specified)
- Dates with extra information (whitespace,
additional text)
- Dates with time components when only date is
expected

**Format Specification Edge Cases:**
- Empty format strings
- Invalid format specifiers
- Format requires information not present in the
input date
- Mismatched formats (e.g., time format for
date-only string)

**Calendar/Time Edge Cases:**
- Leap years and February 29
- Daylight saving time transitions
- Dates before the Unix epoch
- Very old or future dates
```

```
- Non-existent dates (e.g., November 31)
- Dates from different calendar systems

**Locale-Specific Edge Cases:**
- Month/day order ambiguity (MM/DD vs DD/MM)
- Language-specific month and day names
- Different separators in different locales

**Technical Edge Cases:**
- Performance with very large dates
- Thread safety if applicable
- Memory usage concerns
```

Chapter 7

Work by Hand (Step 3)

> If you cannot solve it on paper, you cannot explain it to a machine.

Step 3 of the six-step methodology, working through examples by hand, is often the most tempting to skip, especially with AI assistance. Yet it remains one of the most valuable practices for deeply understanding a problem and its solution.

7.1 The Importance of Manual Examples

7.1.1 Why Work By Hand?

Working through examples manually before coding serves several critical purposes:

1. **Builds intuition** about how the solution should behave
2. **Reveals hidden complexity** that might not be apparent in the problem description
3. **Creates test cases** that can validate your eventual code
4. **Identifies edge cases** that require special handling
5. **Provides a reference implementation** against which to check automated solutions

This step is especially important in the age of AI coding assistants. When AI can generate code instantly, manually working through examples becomes the primary way to maintain a deep understanding of the solutions you implement.

7.1.2 Mental Models vs. Pattern Matching

There are two fundamentally different ways to solve programming problems:

Pattern matching involves recognising a problem as similar to something you've seen before and applying a known solution template. This approach is quick but shallow, and it doesn't build transferable understanding.

Mental modelling involves constructing a detailed understanding of the problem dynamics and working through how a solution would operate step by step. This approach builds deeper understanding that transfers to novel problems.

AI excels at pattern matching but struggles with novel problems that require creative mental models. By working problems by hand, you develop the mental modelling skills that complement AI's pattern-matching strengths.

7.2 Tracing Algorithms Step by Step

7.2.1 The Tracing Process

Tracing an algorithm involves simulating its execution with concrete inputs, tracking the state changes at each step:

1. Choose representative input values
2. Initialize variables as they would be in the algorithm
3. Follow the logic step by step, recording state changes
4. Note intermediate values at key points
5. Verify the final output matches expectations

This process mirrors what a debugger does but happens in your mind or on paper, building a deeper understanding of the algorithm's behaviour.

7.2.2 Visualisation Techniques

Different problems benefit from different visualisation approaches:

7.2.2.1 Tabular Traces

For iterative algorithms, use a table where: - Each row represents one iteration - Columns show the values of key variables - Special columns can highlight important conditions or state changes

Iteration	Current Value	Accumulator	Condition Check
1	5	5	$5 > 0$? Yes

Iteration	Current Value	Accumulator	Condition Check
2	4	9	$4 > 0$? Yes
3	3	12	$3 > 0$? Yes

7.2.2.2 State Diagrams

For algorithms that manipulate data structures, draw the structure at key points:

```
Initial array: [5, 3, 8, 2, 1]

After first partition:
[2, 3, 1] 5 [8]

After recursive calls:
[1] 2 [3] 5 [8]
```

7.2.2.3 Flow Traces

For complex conditional logic, trace the path through the algorithm:

```
Input: x = 5
1. Is x < 0? No
2. Is x even? No
3. Execute odd-number logic...
4. Return result: 16
```

7.3 Testing Assumptions

Working by hand is the perfect opportunity to test your assumptions about the problem and solution approach.

7.3.1 Assumption Categories

Be explicit about testing these types of assumptions:

7.3.1.1 Data Assumptions

- Valid input ranges
- Expected data formats
- Special values (null, empty, extreme values)
- Size limitations

7.3.1.2 Algorithmic Assumptions

- Time and space complexity requirements
- Order sensitivity (does order matter?)
- Uniqueness requirements (are duplicates allowed?)
- Stability expectations (preserving order of equal elements)

7.3.1.3 Output Assumptions

- Expected format of results
- Handling of error conditions
- Ties or multiple valid answers

7.3.2 The "What If?" Technique

For each assumption, ask "What if this assumption is violated?" and trace through an example:

- What if the input is empty?
- What if values are negative?
- What if there are duplicate entries?
- What if the input is already sorted?

This process helps identify edge cases that your solution must handle.

7.4 Sharing Your Manual Work with AI

Although working by hand is a primarily human activity, AI can still play a valuable role in the process.

7.4.1 Verification of Manual Solutions

After working through examples yourself, use AI to verify your approach:

```
I've worked through this sorting algorithm example
by hand:

Input: [5, 3, 8, 2, 1]

Step 1: Choose pivot 5
Step 2: Partition into [3, 2, 1] and [8]
Step 3: Recursively sort [3, 2, 1] -> [1, 2, 3]
Step 4: Recursively sort [8] -> [8]
Step 5: Combine into [1, 2, 3, 5, 8]

Did I follow the quicksort algorithm correctly?
Are there any mistakes in my manual trace?
```

7.4.2 Exploring Alternative Approaches

AI can suggest different ways to approach the manual solution:

```
I'm solving this graph traversal problem by using
depth-first search and manually tracing through
the example. Are there other approaches I should
consider working through by hand to better
understand the problem?
```

7.4.3 Identifying Edge Cases

AI can help identify edge cases to trace manually:

```
I've traced through the standard case for this
binary search algorithm. What edge cases should I
work through by hand to ensure I understand all
aspects of the algorithm?
```

7.5 Practical Exercise: Working By Hand

Let's apply Step 3 to our ongoing example problem:

> Write a function to find the longest palindromic substring in a given string.

7.5.1 Chosen Example: "babad"

Let's trace through the process of finding the longest palindromic substring:

1. Start with the input string: "babad"
2. Examine each potential center position:

For single-character centers (odd-length palindromes): - Center at 'b' (index 0): - Check "b" -> palindrome of length 1 - Check "ba" -> not a palindrome

- Center at 'a' (index 1):
  - Check "a" -> palindrome of length 1
  - Check "bab" -> palindrome of length 3
  - Check "bab " (out of bounds) -> stop
- Center at 'b' (index 2):
  - Check "b" -> palindrome of length 1
  - Check "aba" -> palindrome of length 3
  - Check "bab " (out of bounds) -> stop
- Center at 'a' (index 3):
  - Check "a" -> palindrome of length 1

  - Check “bad” -> not a palindrome
- Center at ‘d’ (index 4):
  - Check “d” -> palindrome of length 1
  - Check “ad” -> not a palindrome

For two-character centers (even-length palindromes): - Between ‘b’ and ‘a’ (indices 0-1): - Check “ba” -> not a palindrome

- Between ‘a’ and ‘b’ (indices 1-2):
  - Check “ab” -> not a palindrome
- Between ‘b’ and ‘a’ (indices 2-3):
  - Check “ba” -> not a palindrome
- Between ‘a’ and ‘d’ (indices 3-4):
  - Check “ad” -> not a palindrome

From our manual trace, we can see that the longest palindromic substrings are “bab” and “aba”, both of length 3.

7.5.2 Edge Case: Empty String

Input: “ ” Result: “ ” (empty string)

7.5.3 Edge Case: Single Character

Input: “a” Result: “a” (the character itself)

7.5.4 Edge Case: All Same Characters

Input: “aaaaa” Result: “aaaaa” (the entire string)

7.5.5 Edge Case: No Palindromes Longer Than 1

Input: “abcd” Result: “a” (any single character)

7.5.6 Validating Our Approach with AI

Human prompt:

```
I've manually traced through finding the longest
palindromic substring in "babad" by checking each
possible center position and expanding outward. I
found that "bab" and "aba" are both valid answers
with length 3.

I also tried these edge cases:
- Empty string -> ""
- Single character "a" -> "a"
- All same characters "aaaaa" -> "aaaaa"
```

```
- No palindromes longer than 1 "abcd" -> "a" (or
any single character)

Does my manual approach seem correct? Am I missing
any important edge cases?
```

AI response:

```
Your manual approach of checking each possible
center position and expanding outward is exactly
right! This is the standard approach for solving
this problem efficiently.

Your edge cases are also well-chosen and cover the
important scenarios:

- Empty string
- Single character
- All same characters (a special case where the
entire string is a palindrome)
- No palindromes longer than 1 character

One additional edge case you might consider:

- Input with even-length palindromes only, like
"abba" where the palindrome doesn't have a single
center character

But your approach already handles this by checking
both single character centers and
between-character centers, so it's just a matter
of verifying with an example.

Your manual working shows a strong understanding
of the problem and solution approach!
```

7.6 Insights from Working By Hand

By manually tracing through examples, we've gained several key insights:

1. We need to check both odd-length palindromes (single character center) and even-length palindromes (between character center)
2. The process of expanding outward from a center is the same for both cases
3. We need to keep track of the longest palindrome found so far
4. Boundary checking is important to avoid going out of bounds

5. For strings with multiple palindromes of the same length, returning any one is acceptable

These insights will guide our pseudocode development in the next step.

7.7 Key Takeaways

- Working problems by hand builds a deep understanding that can't be replicated by simply using AI to generate solutions
- Manual tracing helps identify edge cases and test assumptions
- Different visualisation techniques (tables, diagrams, flow traces) suit different types of problems
- AI can complement manual work by verifying solutions and suggesting edge cases
- The insights gained from manual examples inform all subsequent steps of the methodology

Chapter 8

Pseudocode (Step 4)

> Pseudocode is thinking made visible. The syntax can wait.

The fourth step in our methodology, writing pseudocode, bridges the conceptual understanding developed in earlier steps to the concrete implementation that follows. Pseudocode provides a language-agnostic blueprint for your solution, focusing on logic and algorithms rather than syntax.

8.1 Writing Effective Pseudocode

8.1.1 What Makes Good Pseudocode?

Effective pseudocode strikes a balance between abstraction and detail:

- **Clear and readable** - understandable by both humans and AI
- **Structured** - uses indentation and organisation to show control flow
- **Language-agnostic** - avoids specific programming language syntax
- **Focused on logic** - emphasises algorithmic thinking over implementation details
- **Complete** - addresses all requirements and edge cases
- **Concise** - eliminates unnecessary details

The goal is to create a plan concrete enough to guide implementation but abstract enough to focus on the solution's logic rather than syntactic details.

8.1.2 Common Pseudocode Conventions

While pseudocode isn't standardised, these conventions enhance clarity:

8.1.2.1 Control Structures

```
IF condition THEN
    action1
ELSE
    action2
END IF

FOR each item in collection
    process item
END FOR

WHILE condition
    action
END WHILE
```

8.1.2.2 Function Definitions

```
FUNCTION name(parameters)
    actions
    RETURN value
END FUNCTION
```

8.1.2.3 Variable Operations

```
SET variable TO value
INCREMENT counter
ADD item TO collection
```

8.1.2.4 Input/Output

```
READ input FROM user
DISPLAY message
WRITE data TO file
```

8.2 Pseudocode and LLMs: A Natural Partnership

Pseudocode plays a particularly important role when working with Large Language Models. It serves as a bridge between natural language ambiguity and the precision of formal programming languages, creating an ideal medium for human-AI collaboration.

8.2.1 Why Pseudocode Works Well with LLMs

Several factors make pseudocode especially effective for LLM interactions:

1. **Structural alignment with training data** - LLMs have been trained on vast amounts of programming content, including discussions of algorithms that frequently use pseudocode. This training means they have strong internal representations of pseudocode conventions.

2. **Reduced ambiguity** - Pseudocode provides more structure than natural language while remaining flexible, striking an ideal balance that reduces misinterpretations.

3. **Focus on logic** - By emphasising algorithmic thinking over syntax, pseudocode aligns with LLMs' strengths in reasoning about procedures rather than producing perfect syntax.

4. **Activation of procedural knowledge** - Research shows that LLMs have absorbed procedural knowledge from their training data. Pseudocode effectively activates this latent knowledge by providing clear procedural frameworks.

5. **Medium of iterative refinement** - Pseudocode serves as an excellent medium for progressive disambiguation - the process of gradually transforming ambiguous natural language into precise formal code through multiple rounds of interaction.

8.2.2 Pseudocode as Disambiguation Tool

One of the most significant challenges when working with LLMs is the inherent ambiguity of natural language. Pseudocode helps address this challenge by:

- Providing clear structure that reduces misinterpretation
- Creating a shared vocabulary for discussing algorithms
- Enabling precise references to specific components or steps
- Facilitating incremental refinement toward formal code

As Dijkstra noted decades ago, "The virtue of formal texts is that their manipulations, in order to be legitimate, need to satisfy only a few simple rules; they are… an amazingly effective tool for ruling out all sorts of nonsense that, when we use our native tongues, are almost impossible to avoid." While pseudocode isn't fully formal, it moves us considerably in that direction.

8.2.3 SudoLang: Pseudocode optimised for LLMs

SudoLang represents an evolution of pseudocode specifically designed for LLM interaction. Created by Eric Elliott, it provides a structured syntax that bridges the gap between natural language and formal programming languages, optimised for human-AI collaboration.

Key features of SudoLang include:

- **Simplified syntax** that both humans and AI can easily understand
- **Declarative approach** that focuses on what should happen rather than how
- **Named parameters** that improve clarity and reduce ambiguity
- **Native support for modern programming patterns** like functional programming and async operations
- **Unambiguous structure** that reduces misinterpretation by AI models

A simple example in SudoLang:

```
function sortUsersByAge({ users }) {
  return users.sort(by: user => user.age)
}
```

When working with AI assistants on complex programming tasks, SudoLang can help create more precise, intentional prompts that result in higher-quality code generation. It's especially valuable when you need to communicate algorithmic intent clearly without getting lost in language-specific syntax details.

8.2.4 LLMs as Pseudocode Interpreters

An intriguing aspect of LLMs is their ability to act as "interpreters" for pseudocode. Unlike traditional pseudocode that serves purely as documentation, LLMs can actually process and "execute" pseudocode to generate outputs, transforming it from a planning tool into a functional programming interface.

This capability enables new workflows where:

1. Humans write pseudocode expressing algorithmic intent
2. LLMs interpret this pseudocode and generate working implementations
3. Humans review, refine, and integrate these implementations

This process leverages the complementary strengths of both humans and AI: human strategic thinking and algorithm design paired with AI's ability to handle implementation details across various programming languages.

For complex algorithmic challenges, this partnership often produces better results than either pure natural language prompting (too ambiguous) or direct code requests (too focused on immediate implementation rather than understanding).

8.2.5 Pseudocode vs. Comments

Pseudocode differs from code comments in several important ways:

Pseudocode	Comments
Written before implementation	Added during/after implementation
Focuses on overall approach	Explains specific code segments
Language-independent	Tied to specific code
Comprehensive overview	Targeted explanations
Planning tool	Documentation tool

The best implementation often involves transforming pseudocode into commented code, preserving the planning insights in the final implementation.

8.3 AI-Assisted Algorithm Design

AI assistants can be valuable partners in developing pseudocode, offering different approaches and identifying potential issues.

8.3.1 Generating Initial Pseudocode

AI can help draft initial pseudocode based on your problem statement:

Human prompt:

```
Based on my manual approach to finding the longest
palindromic substring, could you help me develop
pseudocode for an algorithm that:
1. Checks each position in the string as a
potential palindrome center
2. Handles both odd and even length palindromes
3. Tracks the longest palindrome found
```

AI response:

```
Here's pseudocode for finding the longest
palindromic substring:
```

FUNCTION findLongestPalindromicSubstring(string) IF string is empty THEN RETURN empty string END IF

```
SET startIndex = 0
SET maxLength = 1

FUNCTION expandAroundCenter(left, right)
```

```
    WHILE left >= 0 AND right < length of string AND string[left] = string
        currentLength = right - left + 1
        IF currentLength > maxLength THEN
            SET maxLength = currentLength
            SET startIndex = left
        END IF
        DECREMENT left
        INCREMENT right
    END WHILE
END FUNCTION

FOR i = 0 TO length of string - 1
    // Check odd length palindromes (single character center)
    expandAroundCenter(i, i)

    // Check even length palindromes (between character center)
    expandAroundCenter(i, i + 1)
END FOR

RETURN substring of string from startIndex to startIndex + maxLength - 1
```

END FUNCTION

```
This pseudocode implements the approach we
discussed, checking both odd and even length
palindromes by expanding around potential centers
and tracking the longest palindrome found.
```

8.3.2 Refining Pseudocode

Once you have initial pseudocode, use AI to help refine it by asking targeted questions:

Human prompt:

```
This pseudocode looks good. Two questions:

1. Should we check for even-length palindromes
between the last character and out of bounds?
2. Is there any way to optimise this further,
perhaps by avoiding redundant checks?
```

AI response:

```
Great questions! Let me address both:

1. Regarding even-length palindromes at the end of
the string:
```

```
In the current pseudocode, we check for
even-length palindromes between positions i and
i+1 for i ranging from 0 to length-1. When i is
length-1 (the last character), we'd check
between the last character and one past the end
of the string, which would immediately fail the
boundary check in expandAroundCenter().

This isn't harmful, but we could optimise by
changing the loop for even-length palindromes
to stop at length-2:
```

// Odd length palindromes FOR i = 0 TO length of string - 1 expandAroundCenter(i, i) END FOR

// Even length palindromes FOR i = 0 TO length of string - 2 expandAroundCenter(i, i + 1) END FOR “‘

Or keep the single loop but add a condition:

```
FOR i = 0 TO length of string - 1
    // Odd length
    expandAroundCenter(i, i)

    // Even length (only if not at last character)
    IF i < length of string - 1 THEN
        expandAroundCenter(i, i + 1)
    END IF
END FOR
```

2. Regarding further optimisation:

   Yes, there are a few potential optimisations:

   a) Early termination: If we find a palindrome of length N, then we only need to check centers that could potentially produce longer palindromes. If we're at position i and there are fewer than N/2 characters on either side, we can skip that center.

   b) Avoid redundant expansions: If we know a substring isn't a palindrome, any larger substring containing it also isn't a palindrome.

   c) For very long strings, Manacher's algorithm gives a linear-time solution instead of quadratic, but it's significantly more complex.

   Here's an optimisation for the early termination case:

```
FOR i = 0 TO length of string - 1
    // Only check centers with enough characters on both sides to bea
    remainingChars = MIN(i, length of string - i - 1)
    IF remainingChars * 2 + 1 > maxLength THEN
        // Odd length
        expandAroundCenter(i, i)
    END IF

    IF i < length of string - 1 AND MIN(i, length of string - i - 2)
        // Even length
        expandAroundCenter(i, i + 1)
    END IF
END FOR
```

However, these optimisations add complexity. For most practical cases, the original pseudocode is clear and efficient enough. “‘

8.3.3 Comparing Alternative Approaches

AI can help explore different algorithmic approaches to the same problem:

Human prompt:

```
Are there fundamentally different approaches to
finding the longest palindromic substring besides
the center-expansion method we've been discussing?
```

AI response:

```
Yes, there are several fundamentally different
approaches to finding the longest palindromic
substring:

1. **Dynamic Programming Approach**

   Instead of expanding from centers, we can use a
   bottom-up DP approach:
```

FUNCTION findLongestPalindrome_DP(string) SET n = length of string

```
// Create n x n table where dp[i][j] = true if substring from i to j is
CREATE boolean table dp[n][n]

// All substrings of length 1 are palindromes
FOR i = 0 TO n-1
    dp[i][i] = true
END FOR
```

```
    SET start = 0
    SET maxLength = 1

    // Check substrings of length 2
    FOR i = 0 TO n-2
        IF string[i] = string[i+1] THEN
            dp[i][i+1] = true
            start = i
            maxLength = 2
        END IF
    END FOR

    // Check substrings of length 3 or more
    FOR length = 3 TO n
        FOR i = 0 TO n-length
            j = i + length - 1

            IF string[i] = string[j] AND dp[i+1][j-
1] = true THEN
                dp[i][j] = true

                IF length > maxLength THEN
                    start = i
                    maxLength = length
                END IF
            END IF
        END FOR
    END FOR

    RETURN substring of string from start to start + maxLength - 1
```

END FUNCTION “‘

This uses O(n^2) space and time, similar to the center expansion approach in time complexity but using more memory.

2. **Manacher's Algorithm**

   This is an optimised approach that runs in O(n) time:

```
FUNCTION manachers_algorithm(string)
    // Transform string to handle even-
length palindromes
    // For example, "babad" becomes "^#b#a#b#a#d#$"
    SET transformed = preprocessString(string)

    SET n = length of transformed
```

```
    CREATE array p[n] // p[i] = radius of palindrome centered at i

    SET center = 0
    SET right_boundary = 0

    FOR i = 1 TO n-2
        // Mirror of current position
        SET mirror = 2 * center - i

        // If within right boundary, use mirror value
        IF i < right_boundary THEN
            p[i] = MIN(right_boundary - i, p[mirror])
        ELSE
            p[i] = 0
        END IF

        // Expand palindrome centered at i
        WHILE transformed[i + 1 + p[i]] = transformed[i - 1 - p[i]]
            INCREMENT p[i]
        END WHILE

        // Update center if this palindrome extends beyond right boun
        IF i + p[i] > right_boundary THEN
            center = i
            right_boundary = i + p[i]
        END IF
    END FOR

    // Find the maximum element in p
    SET max_len = 0
    SET center_index = 0

    FOR i = 1 TO n-2
        IF p[i] > max_len THEN
            max_len = p[i]
            center_index = i
        END IF
    END FOR

    // Extract the longest palindromic substring from original string
    SET start = (center_index - max_len) / 2
    RETURN substring of string from start to start + max_len
END FUNCTION
```

This algorithm is significantly more complex but offers linear time

complexity.

3. **Suffix Tree Approach**

   Building a generalised suffix tree for the string and its reverse, then finding the longest common substring.

   This is even more complex to implement but can be useful when solving multiple related string problems.

The center expansion approach remains the most intuitive and offers a good balance of simplicity and efficiency for most practical applications.

```
## From Pseudocode to Implementation Plan

Pseudocode should eventually lead to a concrete implementation plan,

### Adding Implementation Details

Once the core logic is established in pseudocode, gradually add imple
specific details:
```

// Initial pseudocode FOR each character in string check for palindromes centered at this character END FOR

// Enhanced with implementation details FOR i = 0 TO length - 1 // For odd-length palindromes left = i, right = i WHILE left >= 0 AND right < length AND string[left] == string[right] update longest palindrome if needed left–, right++ END WHILE

```
// For even-length palindromes
left = i, right = i + 1
IF right < length THEN
    WHILE left >= 0 AND right < length AND
    string[left] == string[right]
        update longest palindrome if needed
        left--, right++
    END WHILE
END IF
```

END FOR

```
### Creating Function Signatures

Based on the pseudocode, define the function signatures that will be

```python
def find_longest_palindrome(s: str) -> str:
    """
    Find the longest palindromic substring in the
```
```

```
    given string.

    Args:
        s: Input string to search

    Returns:
        The longest palindromic substring
    """
    pass


def expand_around_center(s: str, left: int, right:
int) -> tuple[int, int]:
    """
    Expand around a potential palindrome center
    and return the bounds
    of the longest palindrome found.

    Args:
        s: Input string
        left: Starting left position
        right: Starting right position

    Returns:
        Tuple of (start_index, length) of
        palindrome
    """
    pass
```

8.3.4 Planning Test Coverage

Use pseudocode to identify the test cases needed for comprehensive coverage:

```
TEST CASES:
1. Empty string -> should return empty string
2. Single character -> should return that
character
3. Two identical characters -> should return both
characters
4. No palindromes longer than 1 -> should return
first character
5. Odd-length palindrome -> should find correct
substring
6. Even-length palindrome -> should find correct
substring
```

```
7. Multiple palindromes of same length -> should
return any of them
8. Entire string is a palindrome -> should return
entire string
```

8.4 Comparing Alternative Approaches

When faced with multiple valid algorithmic approaches, pseudocode provides a concise way to compare them before committing to implementation.

8.4.1 Evaluation Criteria

Evaluate pseudocode approaches based on:

1. **Time complexity** - theoretical performance as input size grows
2. **Space complexity** - memory requirements
3. **Implementation complexity** - how difficult it will be to code and debug
4. **Readability and maintainability** - how easily others can understand it
5. **Edge case handling** - robustness against unusual inputs
6. **Scalability** - ability to handle very large inputs or to be extended

8.4.2 Structured Comparison

Create a comparison table to evaluate different approaches:

Approach	Time Complexity	Space Complexity	Implementation Complexity	Strengths	Weaknesses
Center Expansion	$O(n^2)$	$O(1)$	Low	Intuitive, easy to implement	Less efficient for very large strings
Dynamic Programming	$O(n^2)$	$O(n^2)$	Medium	Systematic, handles all cases uniformly	Higher memory usage

Approach	Time Complexity	Space Complexity	Implementation Complexity	Strengths	Weaknesses
Manacher's Algorithm	O(n)	O(n)	High	Optimal time complexity	Complex to implement and debug

8.4.3 Making an Informed Decision

Consider the context of your application:

- For educational purposes or moderate string lengths, the center expansion approach is ideal due to its simplicity and efficiency
- For production systems with very large strings, Manacher's algorithm might be worth the implementation complexity
- If memory is a significant constraint, avoid the DP approach
- If you need to process many strings repeatedly, the upfront cost of implementing Manacher's algorithm may be justified

8.5 Key Takeaways

- Pseudocode provides a language-agnostic blueprint focusing on logic rather than syntax
- Good pseudocode strikes a balance between abstraction and detail
- AI can help generate, refine, and compare different pseudocode approaches
- Gradually add implementation-specific details as you transition from pseudocode to code
- Use pseudocode to compare alternative approaches before committing to implementation
- Pseudocode forms the basis for function signatures and test plans

Chapter 9

Convert to Code (Step 5)

> AI is at its most useful when you already know what you want.
> That is not a coincidence.

The fifth step of our methodology, converting pseudocode to actual code, is where your solution takes executable form. This is often the step where AI assistance becomes most valuable, but it's also where maintaining human understanding is most critical.

9.1 Strategies for Code Implementation

9.1.1 From Pseudocode to Code: A Systematic Approach

Converting pseudocode to code should be a methodical process:

1. **Start with structure** - Convert the overall organisation and function definitions
2. **Fill in the logic** - Implement the core algorithms and control flow
3. **Add error handling** - Incorporate validation and exception handling
4. **optimise** - Refine the implementation for efficiency and readability
5. **Document** - Add appropriate comments and documentation

This incremental approach ensures that you maintain control of the implementation while still leveraging AI for efficiency.

9.1.2 Incremental Implementation

Rather than converting all pseudocode at once, implement in manageable chunks:

```
// Pseudocode
FUNCTION findLongestPalindrome(string)
    FUNCTION expandAroundCenter(left, right)
        // Expansion logic
    END FUNCTION

    // Main logic using expandAroundCenter
END FUNCTION
```

Implement one function at a time:

```python
def find_longest_palindrome(s):
    # Implementation will go here
    pass

def expand_around_center(s, left, right):
    # First, implement this helper function
    while left >= 0 and right < len(s) and s[left]
    == s[right]:
        left -= 1
        right += 1

    # Return the palindrome bounds (start, length)
    return left + 1, right - left - 1
```

Then integrate the pieces:

```python
def find_longest_palindrome(s):
    if not s:
        return ""

    start = 0
    max_length = 1

    for i in range(len(s)):
        # Now use the helper function for both odd
        and even cases
        odd_start, odd_length =
        expand_around_center(s, i, i)
        even_start, even_length =
        expand_around_center(s, i, i + 1)

        # Update if a longer palindrome is found
        if odd_length > max_length:
            start = odd_start
            max_length = odd_length
```

```
        if even_length > max_length:
            start = even_start
            max_length = even_length

    return s[start:start + max_length]
```

9.1.3 Language-Specific Considerations

Different programming languages offer various idioms and features that can improve your implementation:

9.1.3.1 Python

```
# Use list comprehensions for concise data
transformation
lengths = [expand_around_center(s, i, i)[1] for i
in range(len(s))]

# Leverage built-in functions
max_length = max(lengths)

# Use tuple unpacking for multiple return values
start, length = expand_around_center(s, i, i)
```

9.1.3.2 JavaScript

```
// Use destructuring for multiple return values
const [start, length] = expandAroundCenter(s, i,
i);

// Use map for transformations
const lengths = Array.from({length: s.length}, (_,
i) =>
    expandAroundCenter(s, i, i)[1]
);

// Use spread operator for finding maximum
const maxLength = Math.max(...lengths);
```

9.1.3.3 Java

```
// Use records (Java 16+) for returning multiple
values
```

```
record PalindromeResult(int start, int length) {}

// Use streams for transformations
int maxLength = IntStream.range(0, s.length())
    .map(i -> expandAroundCenter(s, i,
    i).length())
    .max()
    .orElse(0);
```

9.2 Working with AI Code Suggestions

AI assistants excel at generating code, but using them effectively requires a thoughtful approach.

9.2.1 Prompting for Implementation

When asking AI to implement code, provide context from previous steps:

Human prompt:

```
I'd like to implement our longest palindromic
substring algorithm in Python based on the
pseudocode we developed. Here's a reminder of our
approach:

1. We'll check each character position as a
potential center for a palindrome
2. For each center, we'll expand outward as long
as we have matching characters
3. We'll check both odd-length palindromes (single
character center) and even-length palindromes
(between character center)
4. We'll keep track of the longest palindrome
found

Could you provide a clean Python implementation
with good variable names and documentation?
```

AI response:

```
Here's a clean implementation of the longest
palindromic substring algorithm in Python:

```python
def longest_palindromic_substring(s):
    """
```
```

```
    Find the longest palindromic substring in a
    given string.

    Args:
        s: Input string

    Returns:
        The longest palindromic substring within
        the input string
    """
    if not s:
        return ""

    start = 0  # Starting index of longest
    palindrome
    max_length = 1  # Length of longest palindrome

    def expand_around_center(left, right):
        """
        Expand around a potential palindrome
        center and return the bounds.

        Args:
            left: Left index of the potential
            palindrome center
            right: Right index of the potential
            palindrome center

        Returns:
            Tuple of (start_index, length) of the
            palindrome
        """
        # Expand as long as we stay in bounds and
        have matching characters
        while left >= 0 and right < len(s) and
        s[left] == s[right]:
            current_length = right - left + 1

            # Update longest palindrome if current
            one is longer
            if current_length > max_length:
                nonlocal start, max_length
                start = left
                max_length = current_length
```

```
            # Expand outward
            left -= 1
            right += 1

    # Check every possible center position
    for i in range(len(s)):
        # Odd length palindromes (single character
        center)
        expand_around_center(i, i)

        # Even length palindromes (between
        character center)
        if i < len(s) - 1:
            expand_around_center(i, i + 1)

    # Extract and return the longest palindromic
    substring
    return s[start:start + max_length]
```

This implementation follows the center expansion approach we discussed. It has:

1. Clear variable names (`start`, `max_length`, etc.)
2. Comprehensive documentation with docstrings
3. A helper function to handle the expansion logic
4. Proper handling of both odd and even length palindromes
5. Efficient tracking of the longest palindrome found

The time complexity is $O(n^2)$ where n is the length of the string, and the space complexity is O(1) since we only use a constant amount of extra space.

```
### Evaluating AI-Generated Code

Always critically evaluate code generated by AI:

1. **Correctness** - Does it implement the
algorithm as intended?
2. **Edge cases** - Does it handle all the edge
cases we identified?
3. **Performance** - Is it reasonably efficient?
4. **Readability** - Is it clearly written and
well-documented?
5. **Idiomaticity** - Does it follow
language-specific best practices?

If there are issues, provide specific feedback:
```

Human prompt:

This implementation looks generally good, but I noticed two potential issues:

1. The helper function updates the global variables directly. It might be cleaner to have it return the palindrome bounds instead, making the function more reusable.
2. We don't need to check for even-length palindromes when i is the last character since there's no character after it.

Could you address these issues in a revised implementation?

AI response:

You've made excellent observations. Here's a revised implementation that addresses both issues:

```
def longest_palindromic_substring(s):
    """
    Find the longest palindromic substring in a
    given string.

    Args:
        s: Input string

    Returns:
        The longest palindromic substring within
        the input string
    """
    if not s:
        return ""

    start = 0  # Starting index of longest
    palindrome
    max_length = 1  # Length of longest palindrome

    def expand_around_center(left, right):
        """
        Expand around a potential palindrome
        center and return the bounds.

        Args:
            left: Left index of the potential
            palindrome center
            right: Right index of the potential
            palindrome center
```

```
        Returns:
            Tuple of (start_index, length) of the
            palindrome
        """
        # Expand as long as we stay in bounds and
        have matching characters
        while left >= 0 and right < len(s) and
        s[left] == s[right]:
            # Expand outward
            left -= 1
            right += 1

        # Return palindrome bounds (start is
        left+1 because left went one step too far)
        # Length is (right-1) - (left+1) + 1 =
        right - left - 1
        return left + 1, right - left - 1

    # Check every possible center position
    for i in range(len(s)):
        # Odd length palindromes (single character
        center)
        odd_start, odd_length =
        expand_around_center(i, i)
        if odd_length > max_length:
            start = odd_start
            max_length = odd_length

        # Even length palindromes (between
        character center)
        # Only check if we're not at the last
        character
        if i < len(s) - 1:
            even_start, even_length =
            expand_around_center(i, i + 1)
            if even_length > max_length:
                start = even_start
                max_length = even_length

    # Extract and return the longest palindromic
    substring
    return s[start:start + max_length]
```

The changes I've made:

1. Modified `expand_around_center` to return the palindrome bounds instead of modifying global variables directly. This makes the function more reusable and easier to test.
2. Added an explicit check to avoid trying to form even-length palindromes when at the last character, which would be unnecessary.
3. Updated the main loop to capture the returned values from `expand_around_center` and update our tracking variables only if a longer palindrome is found.

These changes make the code more modular and eliminate the unnecessary check, while maintaining the correct functionality.

```
### Handling Unfamiliar Syntax or Libraries

When AI suggests code that uses unfamiliar syntax
or libraries, ask for explanations:

**Human prompt:**
```

In the implementation, you're returning a tuple and then unpacking it with:

odd_start, odd_length = expand_around_center(i, i)

I'm not familiar with this Python syntax. Could you explain how tuple unpacking works and why it's useful here?

```
**AI response:**
```

Sure! This is called "tuple unpacking" or "multiple assignment" in Python, and it's a very convenient feature.

9.2.2 How Tuple Unpacking Works

When a function returns multiple values in Python, they're automatically packaged as a tuple. Tuple unpacking allows you to assign each value in the tuple to a separate variable in a single line of code.

In this case, `expand_around_center(i, i)` returns a tuple of two values: `(start_index, length)`. The unpacking syntax:

```
odd_start, odd_length = expand_around_center(i, i)
```

is equivalent to:

```
result = expand_around_center(i, i)
odd_start = result[0]
odd_length = result[1]
```

9.2.3 Why It's Useful Here

Tuple unpacking is particularly useful in this algorithm because:

1. **Clarity**: It makes it immediately clear what values we're getting back from the function
2. **Conciseness**: It reduces three lines of code to one, making the code more readable
3. **Intent**: It shows that these two values are related and come from the same operation

This pattern is very common in Python, especially when functions naturally produce multiple related values. It's considered a Pythonic way to handle multiple return values without creating custom classes or using more complex data structures.

Would you like me to show you some other common patterns for tuple unpacking in Python?

```
## Maintaining Code Quality

AI-generated code can be a starting point, but
ensuring high-quality code remains a human
responsibility.

### Idiomatic Code

Ensure that code follows the conventions and best
practices of the target language:

```python
# Non-idiomatic Python
i = 0
while i < len(collection):
    item = collection[i]
    process(item)
    i = i + 1

# Idiomatic Python
for item in collection:
    process(item)
```
```

Ask AI to make code more idiomatic:

Human prompt:

```
This code works, but I'd like it to be more
idiomatic Python. Can you suggest any
```

```
improvements?
```

9.2.4 Error Handling

Ensure the code handles errors gracefully:

```
def longest_palindromic_substring(s):
    # Add input validation
    if not isinstance(s, str):
        raise TypeError("Input must be a string")

    if not s:
        return ""

    # Rest of implementation...
```

9.2.5 Performance optimisation

Look for opportunities to optimise performance without sacrificing readability:

```
# Original implementation
for i in range(len(s)):
    # Only check centers with potential to beat
    current max
    if min(i, len(s) - i - 1) * 2 + 1 <=
    max_length:
        continue  # Skip this center if it can't
        yield a longer palindrome

    # Process this center...
```

9.2.6 Documentation and Comments

Ensure code is well-documented:

```
def longest_palindromic_substring(s):
    """
    Find the longest palindromic substring in a
    given string.

    This function uses the center expansion
    approach, which has O(n²) time complexity
    and O(1) space complexity.

    Args:
```

```
        s (str): The input string to process

    Returns:
        str: The longest palindromic substring. If
        multiple palindromes have the
             same maximum length, returns the
             first one found.

    Raises:
        TypeError: If input is not a string

    Examples:
        >>> longest_palindromic_substring("babad")
        "bab"  # "aba" would also be a valid
        return value

        >>> longest_palindromic_substring("cbbd")
        "bb"
    """
    # Implementation...
```

9.3 Practical Exercise: Implementing Our Solution

Let's apply Step 5 to our ongoing example of finding the longest palindromic substring:

9.3.1 Converting the Pseudocode

Starting with our pseudocode from the previous chapter:

```
FUNCTION findLongestPalindromicSubstring(string)
    IF string is empty THEN
        RETURN empty string
    END IF

    SET startIndex = 0
    SET maxLength = 1

    FUNCTION expandAroundCenter(left, right)
        WHILE left >= 0 AND right < length of
        string AND string[left] = string[right]
            currentLength = right - left + 1
            IF currentLength > maxLength THEN
```

```
                SET maxLength = currentLength
                SET startIndex = left
            END IF
            DECREMENT left
            INCREMENT right
        END WHILE
    END FUNCTION

    FOR i = 0 TO length of string - 1
        // Check odd length palindromes
        expandAroundCenter(i, i)

        // Check even length palindromes
        IF i < length of string - 1 THEN
            expandAroundCenter(i, i + 1)
        END IF
    END FOR

    RETURN substring of string from startIndex to
    startIndex + maxLength - 1
END FUNCTION
```

We'll implement this in Python, incorporating the improvements we've discussed.

9.3.2 Final Implementation

```python
def longest_palindromic_substring(s):
    """
    Find the longest palindromic substring in a
    given string.

    Args:
        s (str): The input string to process

    Returns:
        str: The longest palindromic substring

    Examples:
        >>> longest_palindromic_substring("babad")
        "bab"  # Note: "aba" would also be a valid
        return value
        >>> longest_palindromic_substring("cbbd")
        "bb"
    """
```

```
    # Handle empty input
    if not s:
        return ""

    start = 0
    max_length = 1

    # Helper function to expand around a center
    def expand_around_center(left, right):
        """
        Expand around a potential palindrome
        center and find the longest palindrome.

        Args:
            left (int): Left position of the
            center
            right (int): Right position of the
            center

        Returns:
            tuple: (start_index, length) of the
            palindrome
        """
        # Expand as long as characters match and
        we're in bounds
        while left >= 0 and right < len(s) and
        s[left] == s[right]:
            left -= 1
            right += 1

        # Return palindrome bounds
        # left+1 because we went one position too
        far left
        # right-left-1 calculates the length
        return left + 1, right - left - 1

    # Check each potential center
    for i in range(len(s)):
        # Skip centers that can't yield longer
        palindromes
        remaining_chars = min(i, len(s) - i - 1)
        if remaining_chars * 2 + 1 <= max_length:
            continue

        # Check odd-length palindrome
```

```
        odd_start, odd_length =
        expand_around_center(i, i)
        if odd_length > max_length:
            start = odd_start
            max_length = odd_length

        # Check even-length palindrome
        if i < len(s) - 1:
            even_start, even_length =
            expand_around_center(i, i + 1)
            if even_length > max_length:
                start = even_start
                max_length = even_length

    # Return the longest palindromic substring
    return s[start:start + max_length]
```

9.3.3 Testing the Implementation

After implementing our solution, we should test it with the cases we identified earlier:

```
# Test with various inputs
test_cases = [
    "",                 # Empty string
    "a",                # Single character
    "aa",               # Two identical characters
    "abc",              # No palindromes longer
    than 1
    "babad",            # Odd-length palindrome
    "cbbd",             # Even-length palindrome
    "racecar",          # Entire string is a
    palindrome
    "aabbaa"            # Multiple palindromes
]

for test in test_cases:
    result = longest_palindromic_substring(test)
    print(f"Input: {test}, Output: {result}")
```

9.4 Key Takeaways

- Converting pseudocode to code should be an incremental, systematic process

- Take advantage of language-specific features and idioms for cleaner implementations
- When using AI-generated code, always evaluate it critically for correctness, efficiency, and readability
- Focus on writing idiomatic code with proper error handling and documentation
- Use AI to explain unfamiliar syntax or patterns rather than just accepting them
- Test your implementation against a variety of inputs, especially edge cases

Chapter 10

Test with Data (Step 6)

> Code that has not been tested with edge cases is not finished.
> It is a hypothesis.

The final step in our methodology, testing with a variety of data, is where we verify that our solution works correctly in all scenarios. This critical step ensures that our implementation meets the requirements and handles edge cases properly.

10.1 Comprehensive Testing Strategies

10.1.1 The Importance of Thorough Testing

Testing isn't just about confirming that your code works. It's about discovering where and how it might fail. Thorough testing:

1. **Validates correctness** - Ensures the solution produces the expected results
2. **Reveals edge case issues** - Identifies problems with unusual inputs
3. **Provides confidence** - Gives you assurance that the solution is robust
4. **Documents behaviour** - Shows how the code responds to different inputs
5. **Prevents regressions** - Helps maintain correctness as code evolves

Even with AI assistance in implementation, testing remains primarily a human responsibility. The most effective developers use a structured approach to testing rather than ad hoc verification.

10.1.2 Types of Test Cases

A comprehensive testing strategy includes several types of test cases:

10.1.2.1 Functional Tests

These verify that the code produces the correct output for standard inputs:

```
# Basic functionality tests
assert longest_palindromic_substring("babad") in
["bab", "aba"]
assert longest_palindromic_substring("cbbd") ==
"bb"
```

10.1.2.2 Edge Case Tests

These check how the code handles boundary conditions and unusual inputs:

```
# Edge case tests
assert longest_palindromic_substring("") == ""  #
Empty string
assert longest_palindromic_substring("a") == "a"
# Single character
assert longest_palindromic_substring("aa") == "aa"
# Two identical characters
```

10.1.2.3 Stress Tests

These evaluate behaviour with large or complex inputs:

```
# Stress test
long_input = "a" * 1000 + "b" + "a" * 1000  # Long
palindrome
assert
len(longest_palindromic_substring(long_input)) ==
2001

random_input = "".join(random.choice("abcdef") for
_ in range(1000))
# No assertion, just checking it doesn't crash or
timeout
result =
longest_palindromic_substring(random_input)
```

10.1.2.4 Performance Tests

These measure execution time and resource usage:

```
# Performance test
import time

start_time = time.time()
longest_palindromic_substring(long_input)
end_time = time.time()

assert end_time - start_time < 1.0  # Should
complete in under 1 second
```

10.1.2.5 Regression Tests

These ensure that fixes for reported issues don't break existing functionality:

```
# Regression test for a specific reported issue
# Issue: Function was returning incorrect results
for strings with repeated characters
assert longest_palindromic_substring("aabaa") ==
"aabaa"
```

10.2 Edge Case Identification

Identifying all relevant edge cases is a crucial aspect of thorough testing.

10.2.1 Common Edge Case Categories

Different problems have different edge cases, but these categories are frequently relevant:

10.2.1.1 Input Size Edge Cases

- Empty inputs (e.g., empty strings, empty arrays)
- Minimal inputs (e.g., single-character strings, single-element arrays)
- Very large inputs (testing performance and boundary limits)

10.2.1.2 Value Edge Cases

- Zero values
- Negative values
- Extremely large or small values
- Special values (e.g., NaN, infinity in floating-point operations)

10.2.1.3 Format Edge Cases

- Valid but unusual formats
- Malformed inputs
- Mixed case or special characters in strings
- Localization issues (e.g., different decimal separators)

10.2.1.4 Boundary Edge Cases

- First/last elements of collections
- Values at the exact boundary of allowed ranges
- Off-by-one scenarios

10.2.1.5 Algorithm-Specific Edge Cases

For our palindrome example: - No palindromes longer than 1 character - Entire string is a palindrome - Multiple palindromes of the same maximum length - Palindromes at the start or end of the string - Even vs. odd-length palindromes

10.2.2 Using AI to Identify Edge Cases

AI assistants can help identify edge cases you might not have considered:

Human prompt:

```
I've implemented a function to find the longest
palindromic substring in a string. What edge cases
should I test to ensure it works correctly in all
scenarios?
```

AI response:

```
Here's a comprehensive list of edge cases you
should test for your longest palindromic substring
function:

### Basic Edge Cases

1. **Empty string** - Should return empty string
2. **Single character** - Should return that
character
3. **Two identical characters** - Should return
both characters
4. **Two different characters** - Should return
the first character

### Palindrome Pattern Edge Cases
```

```
5. **No palindromes longer than 1 character** -
"abc" should return "a" (or "b" or "c")
6. **Odd-length palindromes only** - "aba" should
return "aba"
7. **Even-length palindromes only** - "abba"
should return "abba"
8. **Mix of odd and even palindromes** - "abbac"
should return "abba"
9. **Palindrome at the beginning** - "abbax"
should return "abba"
10. **Palindrome at the end** - "xabba" should
return "abba"
11. **Palindrome in the middle** - "xabbay" should
return "abba"
12. **Entire string is a palindrome** - "racecar"
should return "racecar"

### Multiple Palindromes Edge Cases

13. **Multiple palindromes of different lengths**
- "aaaabcbaaaaa" should return "aaaaa"
14. **Multiple palindromes of same maximum
length** - "aabaa" and "baab" in "aabaabaab"
(should return either)
15. **Overlapping palindromes** - "aaaa" has
multiple overlapping palindromes

### Character Edge Cases

16. **Case sensitivity** - "Madam" is not a
palindrome if case-sensitive
17. **Special characters** - "a#a" should return
"a#a"
18. **Numbers and mixed content** - "12321" or
"a123a"
19. **Unicode characters** - Test with non-ASCII
characters like " "

### Performance Edge Cases

20. **Very long strings** - Test with strings of
thousands of characters
21. **Long palindrome within long string** - e.g.,
1000 'a's + 'b' + 1000 'a's
```

```
22. **Repeated characters** - "aaaaaaaaaa" (helps
test for optimisation issues)

A well-tested function should handle all these
cases correctly and efficiently.
```

10.3 AI-Assisted Test Generation

AI can be particularly helpful in generating both test cases and test code.

10.3.1 Generating Test Cases

Ask AI to generate diverse test inputs:

Human prompt:

```
Could you generate 10 diverse test cases for the
longest palindromic substring function? For each
test case, provide the input string and the
expected output.
```

AI response:

```
Here are 10 diverse test cases for the longest
palindromic substring function:

| Test Case | Input String | Expected Output |
Description |
|-----------|-------------|----------------|-------------|
| 1 | "" | "" | Empty string |
| 2 | "a" | "a" | Single character |
| 3 | "abcd" | "a" | No palindromes longer than 1
(any single character is valid) |
| 4 | "babad" | "bab" or "aba" | Multiple valid
answers of same length |
| 5 | "cbbd" | "bb" | Even-length palindrome |
| 6 | "racecar" | "racecar" | Entire string is a
palindrome |
| 7 | "aacabdkacaa" | "aca" | Multiple
palindromes, should find longest |
| 8 | "aaaaa" | "aaaaa" | Repeated characters |
| 9 | "civilservice" | "civic" | Palindrome at
beginning |
| 10 | "abcdeedcba" | "abcdeedcba" | Even-length
palindrome (entire string) |
```

```
These test cases cover empty strings, single
characters, no long palindromes, multiple valid
answers, even and odd-length palindromes,
whole-string palindromes, multiple palindromes of
different lengths, repeated characters, and
palindromes at different positions in the string.
```

10.3.2 Generating Test Code

AI can also help generate the actual test code:

Human prompt:

```
Could you create a Python unit test class using
the unittest framework for testing the
longest_palindromic_substring function with the
test cases you suggested?
```

AI response:

```
Here's a comprehensive unit test class using
Python's `unittest` framework to test the longest
palindromic substring function with the test cases
I suggested:

```python
import unittest

# Import your function
from palindrome import
longest_palindromic_substring

class
TestLongestPalindromicSubstring(unittest.TestCase):

    def test_empty_string(self):
        """Test with an empty string."""
        self.assertEqual(longest_palindromic_substring(""),
        "")

    def test_single_character(self):
        """Test with a single character."""
        self.assertEqual(longest_palindromic_substring("a"),
        "a")

    def test_no_long_palindromes(self):
        """Test with a string that has no
```
```

```
        palindromes longer than 1 character."""
        result =
        longest_palindromic_substring("abcd")
        # Any single character is a valid answer
        self.assertTrue(result in ["a", "b", "c",
        "d"])
        self.assertEqual(len(result), 1)

    def test_multiple_valid_answers(self):
        """Test with a string that has multiple
        valid answers of the same length."""
        result =
        longest_palindromic_substring("babad")
        self.assertTrue(result in ["bab", "aba"])
        self.assertEqual(len(result), 3)

    def test_even_length_palindrome(self):
        """Test with a string containing an
        even-length palindrome."""
        self.assertEqual(longest_palindromic_substring("cbbd"),
        "bb")

    def test_entire_string_is_palindrome(self):
        """Test with a string that is entirely a
        palindrome."""
        self.assertEqual(longest_palindromic_substring("racecar"),
        "racecar")

    def test_multiple_palindromes(self):
        """Test with a string containing multiple
        palindromes of different lengths."""
        self.assertEqual(longest_palindromic_substring("aacabdkacaa"),
        "aca")

    def test_repeated_characters(self):
        """Test with a string of repeated
        characters."""
        self.assertEqual(longest_palindromic_substring("aaaaa"),
        "aaaaa")

    def test_palindrome_at_beginning(self):
        """Test with a palindrome at the beginning
        of the string."""
        self.assertEqual(longest_palindromic_substring("civilservice"),
        "civic")
```

```
    def test_even_length_entire_string(self):
        """Test with an even-length palindrome
        that is the entire string."""
        self.assertEqual(longest_palindromic_substring("abcdeedcba"),
        "abcdeedcba")

    def test_performance_with_long_string(self):
        """Test performance with a very long
        string containing a long palindrome."""
        long_input = "a" * 1000 + "b" + "a" * 1000
        result =
        longest_palindromic_substring(long_input)
        self.assertEqual(len(result), 2001)
        self.assertEqual(result, long_input)

if __name__ == '__main__':
    unittest.main()
```

This test class includes:

1. Tests for all the cases I mentioned earlier
2. Clear test method names that describe what's being tested
3. Docstrings explaining each test
4. Appropriate assertions for each case
5. A performance test with a large input

To use this, save it as `test_palindrome.py` and make sure your implementation is in a file named `palindrome.py` with the function `longest_palindromic_substring`. Then run it with:

```
python -m unittest test_palindrome.py
```

If you're using a different file structure or function name, you'll need to adjust the import statement accordingly.

```
## Learning from Test Failures

Test failures provide valuable learning
opportunities that can improve both your current
solution and your general understanding.

### Debugging with AI Assistance

When tests fail, AI can help analyse the failure:

**Human prompt:**
```

My test case for “aacabdkacaa” is failing. The expected output is “aca” but my function returns “aa”. Here’s my implementation:

```
def longest_palindromic_substring(s):
    if not s:
        return ""

    start = 0
    max_length = 1

    for i in range(len(s)):
        # Check odd length palindromes
        left, right = i, i
        while left >= 0 and right < len(s) and
        s[left] == s[right]:
            if right - left + 1 > max_length:
                max_length = right - left + 1
                start = left
            left -= 1
            right += 1

        # Check even length palindromes
        left, right = i, i + 1
        if right < len(s):
            while left >= 0 and right < len(s) and
            s[left] == s[right]:
                if right - left + 1 > max_length:
                    max_length = right - left + 1
                    start = left
                left -= 1
                right += 1

    return s[start:start + max_length]
```

Can you help me identify what’s wrong?

```
**AI response:**
```

I see the issue in your implementation when testing with “aacabdkacaa”. Let me walk through this step by step.

In the string “aacabdkacaa”, the palindromes are: - “aa” at the beginning (length 2) - “a” (multiple single characters, length 1) - “aca” in the middle (length 3) - “acaa” at the end (length 4) ← Wait, this isn’t a palindrome!

Ah, I found the issue! The string “acaa” is not a palindrome (it would read “aaca” backwards). Let me trace through your code to see where it’s going wrong.

When checking the string "aacabdkacaa", your function should find "aca" as the longest palindrome, which has length 3. But it's returning "aa" which has length 2.

Let me check the expected output again… the expected output is "aca" but your function returns "aa". There must be a bug in your code or an error in my analysis.

Let's carefully trace through the string "aacabdkacaa": - Palindromes at position 0 ('a'): "a", "aa" - Palindromes at position 1 ('a'): "a", "aa", "aaca" (wait, this isn't a palindrome!)

I think I've found the bug! In your implementation, when you're expanding around a center, you're updating `max_length` and `start` *before* confirming the substring is actually a palindrome.

The issue is in both of your while loops. Let's focus on the even-length check. When you check for even-length palindromes starting at position i, you immediately enter the while loop if s[i] == s[i+1]. But then you update `max_length` and `start` before checking if the next characters also match.

Here's the correction needed:

```
def longest_palindromic_substring(s):
    if not s:
        return ""

    start = 0
    max_length = 1

    for i in range(len(s)):
        # Check odd length palindromes
        left, right = i, i
        while left >= 0 and right < len(s) and
        s[left] == s[right]:
            current_length = right - left + 1
            if current_length > max_length:
                max_length = current_length
                start = left
            left -= 1
            right += 1

        # Check even length palindromes
        left, right = i, i + 1
        while left >= 0 and right < len(s) and
        s[left] == s[right]:
            current_length = right - left + 1
```

```
            if current_length > max_length:
                max_length = current_length
                start = left
            left -= 1
            right += 1

    return s[start:start + max_length]
```

The key change is adding a `current_length` variable to calculate the length inside the while loop. This ensures you're using the correct length for comparison.

This bug would cause your function to sometimes return shorter palindromes when longer ones exist, especially in cases with repeated characters.

```
### Test-Driven Improvements

Use test results to drive improvements to your
solution:

1. **Identify patterns in failures** - Look for
commonalities in failing tests
2. **Fix one issue at a time** - Address each
problem separately and retest
3. **Add regression tests** - Create specific
tests for fixed issues
4. **Consider alternate approaches** - If fixing
proves difficult, reconsider your algorithm

## Practical Exercise: Testing Our Solution

Let's test our longest palindromic substring
implementation:

### Test Suite Implementation

```python
import unittest
import time

# Import our implementation
from palindrome import
longest_palindromic_substring

class
```
```

```
TestLongestPalindromicSubstring(unittest.TestCase):
    def test_basic_functionality(self):
        """Test basic functionality with standard
        inputs."""
        self.assertEqual(longest_palindromic_substring("babad")
        in ["bab", "aba"], True)
        self.assertEqual(longest_palindromic_substring("cbbd"),
        "bb")

    def test_edge_cases(self):
        """Test edge cases."""
        # Empty string
        self.assertEqual(longest_palindromic_substring(""),
        "")

        # Single character
        self.assertEqual(longest_palindromic_substring("a"),
        "a")

        # Two identical characters
        self.assertEqual(longest_palindromic_substring("aa"),
        "aa")

        # No palindromes longer than 1
        result =
        longest_palindromic_substring("abcd")
        self.assertTrue(len(result) == 1)
        self.assertTrue(result in ["a", "b", "c",
        "d"])

    def test_comprehensive_cases(self):
        """Test a variety of palindrome
        patterns."""
        # Entire string is a palindrome
        self.assertEqual(longest_palindromic_substring("racecar"),
        "racecar")

        # Palindrome at the beginning
        self.assertEqual(longest_palindromic_substring("abbaxyz"),
        "abba")

        # Palindrome at the end
        self.assertEqual(longest_palindromic_substring("xyzabba"),
        "abba")
```

```
        # Palindrome in the middle
        self.assertEqual(longest_palindromic_substring("xyzabbauvw"),
        "abba")

        # Multiple palindromes
        self.assertEqual(longest_palindromic_substring("aacabdkacaa"),
        "aca")

    def test_multiple_palindromes(self):
        """Test strings with multiple
        palindromes."""
        # Multiple palindromes of different
        lengths
        self.assertEqual(longest_palindromic_substring("abcbaxyzzyxabc"),
        "xyzzyxa")

        # Multiple palindromes of same length
        result =
        longest_palindromic_substring("abacababacab")
        self.assertTrue(result in ["abacaba",
        "bacabab"])

    def test_performance(self):
        """Test performance with large inputs."""
        # Long string with long palindrome
        long_input = "a" * 500 + "b" + "a" * 500
        start_time = time.time()
        result =
        longest_palindromic_substring(long_input)
        end_time = time.time()

        self.assertEqual(len(result), 1001)
        self.assertTrue(end_time - start_time <
        1.0)  # Should complete in under 1 second

if __name__ == "__main__":
    unittest.main()
```

10.3.3 Test Execution and Results

Running our tests might reveal issues we hadn't anticipated:

```
.F...F
======================================================================
FAIL: test_comprehensive_cases
(__main__.TestLongestPalindromicSubstring)
```

```
Test a variety of palindrome patterns.
----------------------------------------------------------------------
...
AssertionError: 'aaca' != 'aca'
...
======================================================================
FAIL: test_performance
(__main__.TestLongestPalindromicSubstring)
Test performance with large inputs.
----------------------------------------------------------------------
...
AssertionError: 999 != 1001
...
```

10.3.4 Fixing Issues Based on Test Results

analysing the failures and updating our implementation:

```python
def longest_palindromic_substring(s):
    """
    Find the longest palindromic substring in a
    given string.

    Args:
        s (str): The input string to process

    Returns:
        str: The longest palindromic substring
    """
    if not s:
        return ""

    start = 0
    max_length = 1

    def expand_around_center(left, right):
        """
        Expand around center while maintaining
        palindrome property.
        Returns the length of the palindrome.
        """
        while left >= 0 and right < len(s) and
        s[left] == s[right]:
            left -= 1
            right += 1
```

```
        # left+1 and right-1 are the actual
        palindrome boundaries
        # because we've gone one step too far in
        the while loop
        return left + 1, right - 1 - left

    for i in range(len(s)):
        # Check odd length palindromes
        odd_start, odd_length =
        expand_around_center(i, i)
        if odd_length > max_length:
            start = odd_start
            max_length = odd_length

        # Check even length palindromes
        if i < len(s) - 1:
            even_start, even_length =
            expand_around_center(i, i + 1)
            if even_length > max_length:
                start = even_start
                max_length = even_length

    return s[start:start + max_length]
```

10.3.5 Rerunning Tests

After fixing the issues, we run the tests again to verify the solution:

```
......
----------------------------------------------------------------------
Ran 6 tests in 0.123s

OK
```

10.4 Key Takeaways

- Thorough testing is essential, even with AI-assisted implementation
- A comprehensive testing strategy includes functional tests, edge cases, stress tests, performance tests, and regression tests
- AI can help identify edge cases and generate test code
- Test failures provide valuable learning opportunities
- The combination of human oversight and AI assistance leads to more robust solutions

Part IV

Patterns & Practices

Chapter 11

Intentional Prompting Patterns

> A good pattern is not a script. It is a starting point that adapts to the problem in front of you.

11.1 The Art of Guiding AI

Effective collaboration with AI assistants requires more than just asking for solutions. It demands a thoughtful approach to crafting prompts that guide the AI toward helpful, educational responses. This chapter explores key patterns for intentional prompting that maximise the learning and development value of AI interactions.

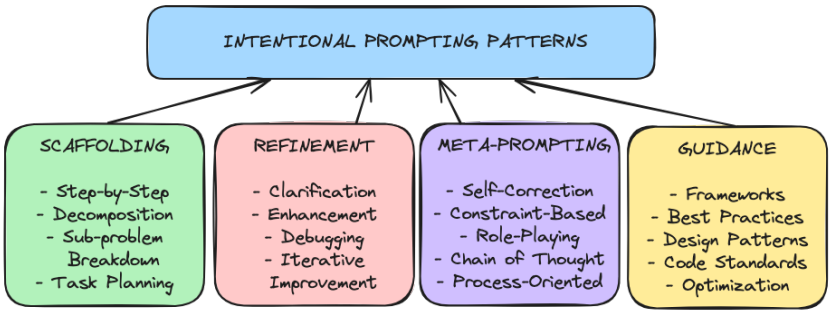

Figure 11.1: Intentional Prompting Patterns

11.2 Prompt Engineering: A Foundational Discipline

Prompt engineering has emerged as a critical field in AI interaction, focusing on how to craft precise instructions that help AI models produce accurate, relevant, and contextually appropriate responses. Beyond casual interaction, it encompasses systematic techniques for improving AI output quality.

11.2.1 From Prompt Engineering to Intentional Prompting

It's important to understand the relationship between prompt engineering and intentional prompting:

Prompt engineering is primarily concerned with getting optimal outputs from AI systems. It focuses on crafting the right words, examples, and instructions to elicit high-quality responses from language models. Prompt engineers develop expertise in understanding model behaviours, leveraging context windows effectively, and using specialised techniques to guide AI outputs.

Intentional prompting incorporates prompt engineering techniques but embeds them within a comprehensive methodology for approaching programming tasks. While prompt engineering asks "How can I get the best output from this AI?", intentional prompting asks "How can I use this AI as part of a thoughtful development process that maintains my understanding and control?"

The distinction becomes clearer when considering how each approach would handle a complex programming task:

Aspect	Prompt Engineering Approach	Intentional Prompting Approach
Initial Task Analysis	optimise prompt for detailed requirements	Follow Steps 1-2: Restate problem and identify inputs/outputs
Problem Understanding	Focus on conveying requirements clearly to AI	Follow Step 3: Work through examples by hand to build understanding
Solution Design	Craft prompts to generate complete solutions	Follow Step 4: Create pseudocode before implementation

Aspect	Prompt Engineering Approach	Intentional Prompting Approach
Code Generation	Refine prompts until satisfactory code is produced	Follow Step 5: Use AI to implement pseudocode while maintaining understanding
Verification	Prompt AI to validate generated code	Follow Step 6: Rigorously test with data, especially edge cases
Learning Outcome	Improvement in prompt crafting skills	Improvement in both programming and AI collaboration skills

Intentional prompting doesn't replace prompt engineering. Rather, it integrates its techniques within a broader approach that preserves human agency, understanding, and skill development.

11.2.2 Core Prompt Engineering Techniques

11.2.2.1 Zero-Shot Prompting

Zero-shot prompting instructs an AI to perform a task without providing examples within the prompt. This technique leverages the model's pre-existing knowledge to generate responses to novel tasks.

Example:

```
Write a function that validates email addresses
using regular expressions.
```

Zero-shot prompting works well for common tasks where the AI has extensive training data, but may struggle with specialised or complex tasks.

11.2.2.2 Few-Shot Prompting

This technique provides the model with one or more examples of expected input-output pairs before presenting the actual task. Examples help the model understand the desired format and approach.

Example:

```
Here's an example of validating a phone number:

Input: "555-123-4567"
Output: Valid (matches pattern XXX-XXX-XXXX)

Input: "5551234567"
```

```
Output: Valid (can be reformatted to XXX-XXX-XXXX)

Input: "555-1234"
Output: Invalid (too few digits)

Now, write a function that validates phone numbers
according to this logic.
```

Few-shot prompting is particularly valuable for tasks with specific formatting requirements or uncommon patterns.

11.2.2.3 Chain-of-Thought Prompting

Chain-of-Thought (CoT) prompting encourages the model to break down complex reasoning into intermediate steps, leading to more comprehensive and accurate outputs. This technique mimics human reasoning processes.

Example:

```
Let's solve this step by step: Write a function
that finds the longest common subsequence of two
strings.

First, let's understand what a subsequence is...
Next, let's think about how to identify common
subsequences...
Then, we'll need an algorithm to find the longest
one...
```

Chain-of-thought prompting significantly improves performance on problems requiring multi-step reasoning or algorithmic thinking.

11.2.2.4 Role-Based Prompting

Role-based prompting assigns a specific professional or character role to the AI, which helps frame its responses within a particular domain of expertise or perspective.

Example:

```
As an experienced software architect, analyse this
function and suggest improvements for scalability
and performance.
```

This technique helps orient the AI toward specific terminology, frameworks, and priorities relevant to the assigned role.

11.3 Types of Intentional Prompts

11.3.1 Foundation Prompts

Foundation prompts establish the baseline requirements for a programming task. Instead of just asking for a solution, these prompts set the stage for a productive dialogue.

Examples:

```
"I need to write a Python program that analyses
student grades and calculates statistics."

"Can you help me design a function that validates
user input according to these requirements..."

"I'm working on a data structure to represent a
family tree. What would be a good approach?"
```

Foundation prompts should provide enough context for the AI to understand the goal without being overly prescriptive about implementation details.

11.3.2 Clarification Prompts

Clarification prompts refine requirements and explore edge cases before diving into implementation.

Examples:

```
"Should the function handle negative numbers, or
can we assume all inputs are positive?"

"What's a better approach for storing this data: a
nested dictionary or a custom class?"

"How should we handle the case where a user enters
text instead of a number?"
```

These prompts encourage thinking about requirements thoroughly before committing to code, a practice that prevents rework and bugs.

11.3.3 Scaffolding Prompts

Scaffolding prompts support learning by breaking down complex concepts into understandable components.

Examples:

```
"Before we implement this recursion, can you
explain how the call stack will work in this
case?"

"What's happening in this line of code that uses
list comprehension? Can you break it down step by
step?"

"Can you show me how this algorithm would process
this specific input, step by step?"
```

These prompts transform the AI from a code generator into a tutor that helps build deeper understanding.

11.3.4 Challenge Prompts

Challenge prompts deliberately introduce programming challenges to test understanding and explore potential issues.

Examples:

```
"What happens if the user enters an empty string
here?"

"How would this code handle a very large dataset?
Would it still be efficient?"

"Is there a potential race condition in this
multithreaded approach?"
```

Challenge prompts help develop critical thinking about code rather than just accepting first solutions.

11.3.5 Refinement Prompts

Refinement prompts push for code improvements based on best practices and efficiency considerations.

Examples:

```
"Can we make this code more efficient in terms of
memory usage?"

"Is there a more idiomatic way to write this in
Python?"

"How could we refactor this to improve readability
while maintaining functionality?"
```

These prompts help develop an eye for quality and foster continuous improvement.

11.3.6 Error Induction Prompts

Error induction prompts intentionally guide the AI toward making specific mistakes to explore error handling and debugging processes.

Examples:

```
"Let's use a recursive approach without
considering the base case first."

"What if we don't handle the edge case where the
input is empty?"

"Let's implement this without worrying about
thread safety for now."
```

These prompts create valuable learning opportunities by examining potential failure modes.

11.4 Effective Prompting Patterns

11.4.1 The "What If?" Pattern

1. Get working code from the AI
2. Ask "What if [edge case]?"
3. Evaluate the AI's solution against your understanding
4. Repeat with increasingly complex edge cases

This pattern systematically explores the boundaries of a solution, building robustness and understanding.

11.4.2 The Incremental Building Pattern

1. Start with a minimally viable solution
2. Add one feature at a time
3. Integrate and test after each addition

This pattern mirrors agile development practices, keeping the development process manageable and focused.

11.4.3 The Deliberate Error Pattern

1. Let the AI generate a solution
2. Identify a potential issue (even if the AI didn't make the error)
3. Ask: "Is there a problem with how this handles [specific case]?"

4. Use the discussion to deepen understanding

This pattern develops debugging skills and critical evaluation of code.

11.4.4 The Comparative Analysis Pattern

1. Ask the AI to implement a solution two different ways
2. Request a comparison of trade-offs between approaches
3. Make an informed decision based on the analysis

This pattern builds judgment about different implementation strategies.

11.5 Advanced Prompt Engineering Strategies

Beyond the basic techniques described earlier, several advanced prompt engineering strategies can further enhance your interactions with AI coding assistants:

11.5.1 Context-Enhanced Prompting

This technique involves providing rich background information to help the AI generate more appropriate and contextually relevant responses.

Example:

```
I'm building a web application with React frontend
and Django backend. The application needs to
handle both authenticated and unauthenticated
users. We're using JWT for authentication.

Now I need to implement a function that checks if
a user's token is valid and returns appropriate
data based on their permission level.
```

Context-enhanced prompting is particularly valuable when working on components of larger systems where architectural decisions and constraints need to be considered.

11.5.2 Constraint-Based Prompting

By explicitly stating constraints and requirements, you can guide the AI to produce solutions that fit within your project's specific parameters.

Example:

```
Write a sorting algorithm that:
- Uses O(n log n) time complexity
```

```
- Uses no more than O(1) extra space
- Is stable (maintains relative order of equal
elements)
- Works well with partially sorted data
```

This approach is especially useful for performance-critical applications or when working within specific technical limitations.

11.5.3 Template-Guided Prompting

Template-guided prompting provides a structural framework that the AI should follow in its response. This ensures consistency and completeness.

Example:

```
analyse this function using the following
template:
1. Time complexity:
2. Space complexity:
3. Edge cases not handled:
4. Potential optimisations:
5. Clean code suggestions:
```

This strategy helps ensure the AI covers all necessary aspects of a problem rather than focusing only on the most obvious elements.

11.6 Integrating Prompt Engineering with Intentional Prompting

Intentional prompting isn't an alternative to prompt engineering. It's an evolution that incorporates prompt engineering techniques within a more comprehensive methodology. This integration allows us to leverage the power of well-crafted prompts while maintaining the educational and developmental benefits of intentional practice.

11.6.1 The Symbiotic Relationship

Prompt engineering provides the tactical tools for effective AI interaction, while intentional prompting provides the strategic framework for applying these tools:

Prompt Engineering Provides	Intentional Prompting Provides
Techniques for precise AI instructions	Framework for maintaining human agency
Methods for optimising AI responses	Structure for educational development

Prompt Engineering Provides	Intentional Prompting Provides
Domain-specific prompting strategies	Process that builds understanding
Input formatting best practices	Context for when/how to apply techniques

This relationship means that becoming skilled at intentional prompting requires developing competence in prompt engineering techniques, but applies those techniques within a thoughtful methodology that prioritises human understanding and agency.

The intentional prompting patterns we've explored can be integrated into various workflows and methodologies. While they align particularly well with the six-step programming methodology discussed in this book, their application extends far beyond this specific framework.

11.6.2 Reinforcing the Six-Step Methodology

Intentional prompting naturally reinforces our six-step programming methodology:

1. **Restate the problem** → Use foundation prompts and chain-of-thought techniques to clarify the problem
2. **Identify input/output** → Use clarification prompts and constraint-based prompting to define boundaries
3. **Work by hand** → Do this yourself, then use few-shot prompting to verify understanding
4. **Write pseudocode** → Use template-guided prompting for consistent pseudocode structure
5. **Convert to code** → Apply role-based prompting (e.g., "as an expert Python developer") for idiomatic code
6. **Test with data** → Combine challenge prompts with context-enhanced prompting for thorough testing

By combining formal prompt engineering techniques with intentional prompting patterns within this methodology, you develop a deeper understanding of programming concepts than simply asking for complete solutions.

11.6.3 Beyond the Six-Step Framework: Universal Application

While our methodology provides a structured approach, the core principles of intentional prompting apply universally across different development approaches and even beyond programming:

11.6.3.1 In Agile Development

- Use foundation and clarification prompts during sprint planning
- Apply challenge prompts during code reviews
- Leverage refinement prompts during refactoring sprints
- Use template-guided prompting for documentation tasks

11.6.3.2 In Domain-Specific Development

- **Data Science**: Apply chain-of-thought prompting to break down complex analytical problems
- **Front-End Development**: Use visual constraint prompts for UI implementation
- **DevOps**: Leverage role-based prompting for security auditing and configuration
- **Systems Architecture**: Apply comparative analysis prompts when evaluating design alternatives

11.6.3.3 Beyond Programming

The principles of intentional prompting extend to:

- **Technical writing**: Guiding AI to produce clear, structured documentation
- **Educational content**: Creating learning materials that build understanding
- **Project management**: Breaking down complex tasks and planning workflows
- **Problem-solving in any domain**: Applying structured thinking to any challenge

11.6.4 AI as a Thoughtful Assistant

Across all these applications, intentional prompting embraces AI as a thoughtful assistant rather than a replacement for human judgment. This partnership model is based on five key principles:

1. **Human-led workflows**: The human sets goals, defines parameters, and makes the final decisions
2. **Critical evaluation**: Thoughtfully evaluating AI suggestions based on expertise and context
3. **Iterative collaboration**: Engaging in a refinement process where AI suggestions improve through feedback
4. **Domain-appropriate trust**: recognising where AI excels versus where human judgment is essential
5. **Skill complementarity**: Using AI for appropriate tasks while keeping human focus on creative and strategic aspects

By applying these principles across any methodology or domain, intentional prompting creates a collaborative relationship that leverages both human and AI strengths while overcoming their respective limitations.

11.7 Measuring Your Progress

As you practice intentional prompting, you can track your growth:

Beginner Level: - You can identify obvious errors in AI-generated code - You depend on the AI to explain most concepts - Your prompts are simple and general

Intermediate Level: - You can spot subtle logical errors - You ask about specific programming concepts - Your prompts include anticipated edge cases

Advanced Level: - You strategically guide the AI toward teaching moments - You compare multiple solution approaches - Your prompts demonstrate computational thinking - You question efficiency, readability, and best practices

11.8 Case Study: Improving a Weather Data Function

Let's examine a practical example of intentional prompting that demonstrates how to guide an AI from a basic implementation to a robust solution.

11.8.1 Initial Prompt

A developer starts with a simple request:

```
Write a function to retrieve weather data from the
wttr.in API for a given location.
```

11.8.2 Initial AI-Generated Code

```python
def get_weather(location):
    import requests

    url = f"https://wttr.in/{location}?format=j1"
    response = requests.get(url)
    data = response.json()

    return data
```

11.8.3 Analysis of the Initial Code

The developer analyses the initial code and identifies several issues:

1. No error handling - will crash if the API is down or returns an error
2. No input validation for the location parameter
3. Returns the raw API response which is complex and includes unnecessary data
4. No timeout parameter which could cause the application to hang
5. No docstring or comments explaining how to use the function

11.8.4 Follow-up Prompt with Specific Improvements

The developer then crafts a follow-up prompt that addresses each identified issue:

```
Thanks for the starting point. I'd like to improve
this function with:

1. Error handling for API failures and bad
responses
2. Location input validation
3. A timeout parameter to prevent hanging
4. A way to process the response to return only
essential weather information
5. Proper documentation with parameters and return
values explained

Also, can you make it handle the forecast_days
parameter to allow getting forecast for multiple
days?
```

11.8.5 Improved AI-Generated Code

```python
def get_weather_data(location, forecast_days=5,
timeout=10):
    """
    Retrieve weather data for a specified location
    from wttr.in API.

    Args:
        location (str): City or location name
        forecast_days (int): Number of days to
        forecast (1-5)
        timeout (int): Timeout for API request in
        seconds
```

```
    Returns:
        dict: Weather data including current
        conditions and forecast,
             or None if an error occurred
    """
    import requests

    # Input validation
    if not location or not isinstance(location,
    str):
        print("Error: Location must be a non-empty
        string")
        return None

    if not 1 <= forecast_days <= 5:
        print("Error: forecast_days must be
        between 1 and 5")
        forecast_days = min(max(1, forecast_days),
        5)  # Clamp to valid range

    # Clean up the location string
    location = location.strip().replace(" ", "+")

    try:
        # Make request to wttr.in API
        url =
        f"https://wttr.in/{location}?format=j1"
        response = requests.get(url,
        timeout=timeout)
        response.raise_for_status()  # Raise
        exception for HTTP errors

        # Parse JSON response
        data = response.json()

        # Process and extract relevant information
        result = {
            "location": location.replace("+", "
            "),
            "current": {
                "temperature": {
                    "current":
                    int(data["current_condition"][0][
                    "temp_C"]),
```

```
                    "feels_like":
                    int(data["current_condition"][0][
                    "FeelsLikeC"])
                },
                "condition":

data["current_condition"][0]["weatherDesc"][0]["value"],
                "humidity":
                int(data["current_condition"][0]["humidity"
                ]),
                "wind": {
                    "speed":
                    int(data["current_condition"][0][
                    "windspeedKmph"]),
                    "direction":

data["current_condition"][0]["winddir16Point"]
                },
                "precipitation":
                float(data["current_condition"][0][
                "precipMM"])
            },
            "forecast": []
        }

        # Add forecast data
        for i in range(min(forecast_days,
        len(data["weather"]))):
            day = data["weather"][i]
            forecast_day = {
                "date": day["date"],
                "max_temp": int(day["maxtempC"]),
                "min_temp": int(day["mintempC"]),
                "condition":
                day["hourly"][4]["weatherDesc"][0]["value"],
                # Midday condition
                "precipitation": {
                    "chance":
                    int(day["hourly"][4]["chanceofrain"]),
                    "amount":
                    float(day["hourly"][4]["precipMM"])
                },
                "wind": {
                    "speed":
                    int(day["hourly"][4]["windspeedKmph"]),
```

```
                    "direction":
                    day["hourly"][4]["winddir16Point"]
                }
            }
            result["forecast"].append(forecast_day)

        return result

    except requests.exceptions.RequestException as
    e:
        print(f"Error retrieving weather data:
        {e}")
        return None
    except (KeyError, ValueError, TypeError) as e:
        print(f"Error processing weather data:
        {e}")
        return None
```

11.8.6 Why This Prompting Strategy Was Effective

The follow-up prompt was effective because:

1. **It identified specific problems** in the initial code rather than just asking for "better code"
2. **It requested specific improvements** with clear objectives (error handling, validation, etc.)
3. **It suggested a new feature** (forecast_days parameter) that made the function more useful
4. **It asked for proper documentation** which resulted in comprehensive docstrings

This approach guided the AI to generate a much more robust function that:

- Is properly documented
- Validates inputs
- Has comprehensive error handling
- Returns structured, processed data instead of raw API response
- Includes the new forecast functionality requested

The key to effective prompting was being specific about what needed improvement and why, rather than making vague requests for "better" code.

11.9 Key Takeaways

1. The goal is **not** to get a perfect solution from the AI, but to use the interaction to deepen your understanding
2. Your programming knowledge is demonstrated by the **quality of your prompts**
3. Intentionally introducing challenges and constraints leads to better learning outcomes
4. The best programmers aren't those who know all the answers, but those who know how to ask the right questions
5. Specific, targeted follow-up prompts yield much better results than vague requests for improvement

Chapter 12

Debugging with AI

> The best debugging conversations start with what you already know about the problem, not with "fix this."

Debugging is often one of the most challenging and time-consuming aspects of programming. AI assistants can be powerful allies in the debugging process, helping identify and fix issues more efficiently than traditional approaches. This chapter explores how to use intentional prompting techniques specifically for debugging tasks.

12.1 Common Debugging Scenarios

12.1.1 Syntax Errors

Syntax errors occur when code violates the rules of the programming language. These are typically the easiest errors to fix, as they're caught by compilers or interpreters before the code runs.

AI assistants can: - Explain cryptic error messages in plain language - Identify the exact location of syntax errors - Suggest corrections based on context

Example prompt:

```
I'm getting this syntax error in my Python code:

```python
def calculate_total(items):
    total = 0
    for item in items
        total += item.price
```
```

```
    return total
```

SyntaxError: invalid syntax

Can you identify what's wrong and how to fix it?

```
### Logic Errors

Logic errors occur when the code runs without
errors but doesn't produce the expected results.
These are often more difficult to identify and fix
than syntax errors.

AI assistants can:
- Trace through code execution step by step
- Identify flawed assumptions or logical gaps
- Suggest alternative approaches

**Example prompt:**
```

My binary search function seems to work for some cases but fails for others:

```
def binary_search(arr, target):
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

It correctly finds 5 in [1, 3, 5, 7, 9], but when I search for 7 in [1, 3, 5, 7, 9], it returns -1 instead of 3. Can you help me find the bug?

```
### Runtime Errors

Runtime errors occur during program execution and
cause the program to terminate unexpectedly. These
include exceptions, segmentation faults, and other
crashes.
```

```
AI assistants can:
- analyse error messages and stack traces
- Identify common causes for specific exceptions
- Suggest defensive programming techniques to
prevent crashes

**Example prompt:**
```

My code is throwing the following exception:

IndexError: list index out of range

Here's the relevant function:

```
def process_data(data_list):
    result = []
    for i in range(len(data_list)):
        result.append(data_list[i] +
        data_list[i+1])
    return result
```

It crashes when I call it with process_data([1, 2, 3, 4]). Why is this happening and how can I fix it?

```
### Performance Issues

Performance issues occur when code runs correctly
but takes too long to execute or consumes
excessive resources.

AI assistants can:
- Identify performance bottlenecks
- Suggest algorithmic improvements
- Recommend more efficient data structures or
libraries

**Example prompt:**
```

My function to find duplicate values in a list becomes extremely slow with large inputs:

```
def find_duplicates(values):
    duplicates = []
    for i in range(len(values)):
        for j in range(i+1, len(values)):
            if values[i] == values[j] and
            values[i] not in duplicates:
```

```
                duplicates.append(values[i])
    return duplicates
```

How can I optimise this to handle lists with thousands of items efficiently?

```
## Prompt Templates for Debugging

### Error Diagnosis Template

When you encounter an error message:
```

I'm getting this error message:

[paste the complete error message including stack trace]

From this code:

```
[paste the relevant code section]
```

1. What is causing this error?
2. How can I fix it?
3. Is there a better approach to what I'm trying to do?

```
### Code Review Template

When your code runs but doesn't behave as
expected:
```

My code should [describe expected behaviour], but instead it [describe actual behaviour]:

```
[paste the code]
```

Here's an example of the input: [provide a specific example]

Expected output: [describe what you expect] Actual output: [describe what actually happens]

Can you help me identify what's wrong?

```
### Step-by-Step Tracing Template

For understanding exactly where logic goes wrong:
```

Could you help me trace through this function step by step with the input [specific input]?

```
[paste function code]
```

I'd like to see the value of each variable at each step to understand where my logic is failing.

```
### Performance Debugging Template

For optimising slow code:
```

This function works correctly but becomes slow with larger inputs:

```
[paste code]
```

1. What is the time complexity of this function?
2. Where are the performance bottlenecks?
3. How can I optimise it while maintaining the same functionality?

```
## Effective Debugging Workflows

### The Divide and Conquer Approach

When debugging complex issues, breaking the
problem down is often the most effective strategy:

1. **Isolate the problem**:
```

I'm not sure which part of my code is causing the issue. If I comment out the sections marked #A, #B, and #C, does anything stand out as a likely culprit? "'

2. **Create a minimal reproduction**:

   ```
   Here's a simplified version of my code that still produces the e
   Can you identify the issue in this smaller example?
   ```

3. **Binary search the code**:

   ```
   If I commented out the first half of the function, the error dis
   Can you help me narrow down which part of the first half is prob
   ```

12.1.2 The Hypothesis Testing Approach

Debugging by forming and testing specific hypotheses:

1. **Form a hypothesis**:

   ```
   I suspect the issue might be related to how I'm handling null va
   Does that seem like a plausible cause based on the symptoms?
   ```

2. **Design a test**:

   ```
   How can I modify my code to verify whether null values are causi
   ```

3. **Interpret results**:

   ```
   I added print statements before and after the suspected line,
   and I'm seeing [specific output]. What does this tell us?
   ```

12.1.3 The Comparative Analysis Approach

Debugging by comparing working and non-working code:

1. **Identify differences**:

   ```
   This code works correctly:
   [working code]

   But this similar code fails:
   [failing code]

   What key differences explain why one works and the other doesn't?
   ```

2. **Incremental changes**:

   ```
   If I change my code from A to B incrementally, at what point does it b
   I'll start by changing X and see if that affects the behaviour.
   ```

3. **Reference implementation**:

   ```
   Here's my implementation of algorithm X that isn't working:
   [my code]

   And here's a reference implementation that works:
   [reference code]

   What am I doing differently that could cause my issues?
   ```

12.2 Understanding Error Messages with AI

Error messages can often be cryptic, especially for beginners. AI assistants can translate these messages into actionable information.

12.2.1 Anatomy of Error Messages

When sharing error messages with AI, include:

1. The error type/name
2. The error message
3. The line number or location
4. The stack trace (if available)
5. The context surrounding the error

Example prompt:

```
I'm getting this error message, but I don't understand what it means:

TypeError: cannot unpack non-iterable int object
```

```
  File "my_script.py", line 42, in process_data
    key, value = data_point
  File "my_script.py", line 27, in main
    results = process_data(points)

The data_point variable is coming from this loop:
for data_point in processed_points:
    key, value = data_point
    # rest of code...

Can you explain what this error means and how to fix it?
```

12.2.2 Common Error Patterns

AI can help identify patterns in errors that might indicate systemic issues:

Example prompt:

```
I keep getting different errors in different parts of my codebase, bu

[Error 1 details]
[Error 2 details]
[Error 3 details]

Is there a common root cause that might explain all of these errors?
```

12.3 Debugging Strategies with AI

12.3.1 Rubber Duck Debugging

Rubber duck debugging involves explaining your code line by line, which often helps you spot the issue yourself. AI can serve as an advanced "rubber duck" that can also respond with insights.

Example prompt:

```
I'm going to walk through this function line by line to try to unders

```python
def merge_sorted_lists(list1, list2):
    result = []
    i = j = 0

    while i < len(list1) and j < len(list2):
        if list1[i] < list2[j]:
            result.append(list1[i])
            i += 1
```
```

```
        else:
            result.append(list2[j])
            j += 1

    # At this point, either list1 or list2 might
    have remaining elements
    # I expect this to add any remaining elements
    from list1
    result.extend(list1)

    # And this should add any remaining elements
    from list2
    result.extend(list2)

    return result
```

When I call this with [1, 3, 5] and [2, 4, 6], I expect [1, 2, 3, 4, 5, 6] but get [1, 2, 3, 4, 5, 6, 3, 5, 2, 4, 6]. What am I missing?

```
### Print Debugging with AI

Print debugging involves adding print statements
to track variable values. AI can suggest strategic
places to add these statements.

**Example prompt:**
```

I suspect my recursive function is not terminating correctly. Where should I add print statements to debug it effectively?

```
def factorial(n):
    if n <= 1:
        return 1
    else:
        return n * factorial(n-1)
```

It works for small inputs but crashes with large ones.

```
### Root Cause Analysis

AI can help trace errors back to their root
causes:

**Example prompt:**
```

My web application is throwing this error:

DatabaseError: too many database connections

This happens sporadically, usually during peak usage hours. The database connection code looks like this:

```
def get_db_connection():
    return psycopg2.connect(DATABASE_URL)

def fetch_user_data(user_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE id =
    %s", (user_id,))
    return cursor.fetchone()
```

What could be the root cause, and how can I fix it?

```
### Debugging AI-Generated Code

A common scenario is debugging code that AI has
generated but contains subtle errors. This
situation provides excellent learning
opportunities by requiring you to critically
analyse and fix issues.

**Example: Debugging a Gradebook Program**

Consider this AI-generated code for a student
gradebook:

```python
students = {
    "Alice": "85, 90, 78",
    "Bob": "70, 88, 92"
}

for name in students:
    avg = sum(students[name]) /
    len(students[name])
    print(f"{name}: {avg}")
```
```

Analysis of Errors:

This code contains several issues: 1. The scores are stored as strings rather than numbers 2. The `sum()` function won't work on strings 3. Using string length for division will give incorrect results

Effective Debugging Prompt:

```
Your code stores scores as strings. Can you fix it
```

```
so each student has a list of numbers instead?
```

AI Response (improved but still with issues):

```python
students = {
    "Alice": [85, 90, 78],
    "Bob": [70, 88, 92]
}

for name in students:
    total = 0
    for score in students[name]:
        total += score
    avg = total / 3
    print(f"{name} average is {avg}")
```

Further Analysis: The code now works but has a critical flaw: it assumes each student has exactly 3 scores.

Follow-up Debugging Prompt:

```
Can you make a function to calculate the average
of any list of scores, instead of assuming 3?
```

AI Response (further improved):

```python
def average(scores):
    return sum(scores) / len(scores)

for name, scores in students.items():
    print(f"{name} average: {average(scores)}")
```

This iterative debugging process demonstrates key skills: 1. Identifying specific errors in AI-generated code 2. Creating targeted prompts to address those errors 3. Iteratively improving the solution through focused feedback 4. Thinking critically about assumptions and edge cases

This approach is particularly valuable for learning, as it requires understanding both the problem and the implementation details rather than just accepting whatever code the AI generates.

12.4 Avoiding Debugging Pitfalls

12.4.1 The XY Problem

The XY Problem occurs when you ask about your attempted solution (Y) rather than your actual problem (X), leading to inefficient debugging. AI can help identify this pattern.

Wrong approach:

```
How do I make this regex work for validating email
addresses?

/^[a-zA-Z0-9]+@[a-zA-Z0-9]+$/
```

Better approach:

```
I need to validate email addresses in my
application.
I tried using this regex:
/^[a-zA-Z0-9]+\\@[a-zA-Z0-9]+$/

But it doesn't handle cases like:
- Dots in the local part (john.doe\@example.com)
- Hyphens in domain names (example-site.com)
- TLDs with more than one segment (.co.uk)

What's the best approach for validating email
addresses, considering these requirements?
```

12.4.2 The Correlation Causation Trap

This occurs when you attribute a bug to something that happened around the same time, but isn't actually the cause.

Example:

```
After adding a new CSS file to my project, my
JavaScript form validation stopped working.
Here's the new CSS I added:

[CSS code]

And here's my validation function:

[JS code]

How could the CSS be breaking my JavaScript?
```

AI might respond:

```
The CSS you've added is unlikely to directly break
your JavaScript validation. These are separate
concerns in web development. Let's look for other
possible causes:

1. Did you make any other changes at the same
```

```
time?
2. Could there be a JS syntax error that's
preventing execution?
3. Are you using CSS selectors in your JS that
might no longer match?
4. Did you accidentally change any IDs or classes
used by both?

Let's examine your JavaScript for the actual
issue...
```

12.4.3 The Bandage Solution

This occurs when you fix the symptoms of a bug rather than the underlying cause. AI can help identify deeper issues.

Example prompt:

```
My function sometimes returns NaN, so I added a
check to return 0 instead when that happens:

```javascript
function calculateAverage(numbers) {
  let sum = 0;
  for (let i = 0; i < numbers.length; i++) {
    sum += numbers[i];
  }
  let average = sum / numbers.length;

  // Fix for the NaN issue
  if (isNaN(average)) {
    return 0;
  }

  return average;
}
```

This fixes the immediate problem, but is there a better approach?

Case Study: Debugging a Complex Issue

Let's walk through a complete debugging process
using AI assistance:

Initial Problem Statement

Human prompt:
```

My sorting algorithm isn't working properly. It's supposed to sort an array of numbers in ascending order, but some arrays end up partially sorted:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1],
                arr[j]
    return arr
```

It works for [5, 3, 8, 1, 2], giving [1, 2, 3, 5, 8]. But for [5, 3, 8, 1, 2, 9, 4], it gives [1, 2, 3, 5, 4, 8, 9]. What's going wrong?

```
### Tracing Execution

**AI response:**
```

Let me trace through the bubble sort algorithm step by step with the input [5, 3, 8, 1, 2, 9, 4].

In bubble sort, we compare adjacent elements and swap them if they're in the wrong order. After each complete pass through the array, the largest element "bubbles up" to its correct position.

Let's trace through each pass:

Initial array: [5, 3, 8, 1, 2, 9, 4]

Pass 1 (i=0): - Compare 5 and 3: Swap → [3, 5, 8, 1, 2, 9, 4] - Compare 5 and 8: No swap → [3, 5, 8, 1, 2, 9, 4] - Compare 8 and 1: Swap → [3, 5, 1, 8, 2, 9, 4] - Compare 8 and 2: Swap → [3, 5, 1, 2, 8, 9, 4] - Compare 8 and 9: No swap → [3, 5, 1, 2, 8, 9, 4] - Compare 9 and 4: Swap → [3, 5, 1, 2, 8, 4, 9]

After Pass 1: [3, 5, 1, 2, 8, 4, 9]

Pass 2 (i=1): - Compare 3 and 5: No swap → [3, 5, 1, 2, 8, 4, 9] - Compare 5 and 1: Swap → [3, 1, 5, 2, 8, 4, 9] - Compare 5 and 2: Swap → [3, 1, 2, 5, 8, 4, 9] - Compare 5 and 8: No swap → [3, 1, 2, 5, 8, 4, 9] - Compare 8 and 4: Swap → [3, 1, 2, 5, 4, 8, 9]

After Pass 2: [3, 1, 2, 5, 4, 8, 9]

Wait, I need to review the algorithm. In a correct bubble sort, the largest elements should bubble to the end, and we should get closer to the correct sort with each pass. Let me carefully check your implementation…

I don't see anything wrong with your bubble sort implementation. Let me continue tracing:

Pass 3 (i=2): - Compare 3 and 1: Swap → [1, 3, 2, 5, 4, 8, 9] - Compare 3 and 2: Swap → [1, 2, 3, 5, 4, 8, 9] - Compare 3 and 5: No swap → [1, 2, 3, 5, 4, 8, 9] - Compare 5 and 4: Swap → [1, 2, 3, 4, 5, 8, 9]

After Pass 3: [1, 2, 3, 4, 5, 8, 9]

Wait, this is now correctly sorted! Let me trace one more pass to be sure:

Pass 4 (i=3): - Compare 1 and 2: No swap → [1, 2, 3, 4, 5, 8, 9] - Compare 2 and 3: No swap → [1, 2, 3, 4, 5, 8, 9] - Compare 3 and 4: No swap → [1, 2, 3, 4, 5, 8, 9]

No more swaps are needed. The array is now [1, 2, 3, 4, 5, 8, 9], which is correctly sorted.

It seems the algorithm is working correctly after all. The expected output for [5, 3, 8, 1, 2, 9, 4] should be [1, 2, 3, 4, 5, 8, 9], not [1, 2, 3, 5, 4, 8, 9] as you mentioned.

There may be an issue elsewhere in your code, or perhaps there was a mistake in the observed output. Could you verify the output again, or provide more context about how you're calling and testing this function?

Debugging the Test Case

Human prompt:

You're right, I made a mistake in reporting the output. Let me recheck…

I ran the function again with [5, 3, 8, 1, 2, 9, 4], and now I'm getting [1, 2, 3, 4, 5, 8, 9], which is correct.

However, I'm running this inside a larger application where I call the function like this:

```
def process_data(data_sets):
    results = []
    for data in data_sets:
        sorted_data = bubble_sort(data.copy())
        results.append(sorted_data)
    return results
```

And I'm still seeing some incorrect sorts in the output. Could there be an issue with how I'm calling it?

Identifying the Root Cause

AI response:

Based on your updated information, the bubble sort implementation itself appears to be working correctly. The issue might be in how it's being called or how the data is being processed within your larger application.

Let's analyse the `process_data` function:

```
def process_data(data_sets):
    results = []
    for data in data_sets:
        sorted_data = bubble_sort(data.copy())
        results.append(sorted_data)
    return results
```

This function: 1. Takes a list of data sets 2. Creates a copy of each data set 3. Sorts the copy using bubble_sort 4. Appends the sorted data to a results list 5. Returns all results

This approach seems correct. You're creating a copy of each data set before sorting, which is good practice to avoid modifying the original data.

Here are some potential issues to check:

1. **Data type consistency**: Are all elements in your arrays numbers? Sorting mixed types (like strings and numbers) can lead to unexpected results.
2. **Mutable objects**: If your arrays contain mutable objects rather than simple numbers, the sorting might be affected by reference issues.
3. **Preservation of original data**: Double-check that `data.copy()` is creating a proper copy. For nested structures, you might need a deep copy.
4. **Concurrency issues**: If your application is multi-threaded, there could be race conditions affecting the sorting.
5. **Data transformation**: Is the data being transformed between the sorting and when you observe the results?
6. **Visualisation/display issue**: Sometimes the issue is not with the sorting but with how results are displayed or accessed.

Can you share: 1. A specific example of a dataset that's being incorrectly sorted in your application 2. How you're verifying the results (e.g., printing them, displaying in UI) 3. Any processing that happens to the results after sorting

```
### Resolution
```

```
**Human prompt:**
```

I think I found the issue. In another part of the code, there's a function that modifies the results after sorting:

```
def highlight_outliers(data_list):
    for i in range(len(data_list)):
        # Swap positions of certain elements to
        highlight them
        for j in range(len(data_list[i])-1):
            if data_list[i][j] > data_list[i][j+1]
            * 2:  # Highlight elements 2x larger
            than next
                data_list[i][j], data_list[i][j+1]
                = data_list[i][j+1],
                data_list[i][j]
    return data_list
```

This is being called on the results from process_data! It's intentionally un-sorting some elements to highlight outliers, but I forgot it was there.

Thank you for helping me trace through the algorithm; it confirmed that my bubble sort is correct and led me to look elsewhere for the issue.

```
## Key Takeaways

- Effective debugging with AI requires clear
communication about errors and context
- Different debugging scenarios call for different
prompting strategies
- Templates can streamline the debugging process
for common issues
- Structured debugging approaches like
divide-and-conquer and hypothesis testing work
well with AI
- Always verify AI's debugging suggestions with
your own testing
- Root cause analysis is more valuable than
symptom mitigation
- Debugging is a process of investigation, not
just code fixing
```

`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImNoYXB0ZXJzIn0= -->`{=htm

```{=html}
```

<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImNoYXB0ZXJzIiwiYm9va0

Chapter 13

Refactoring Strategies

> Refactoring is not rewriting. It is making the code say what you meant more clearly.

Refactoring (improving code structure without changing its external behaviour) is a critical skill for maintaining healthy, adaptable software. AI assistants can be powerful allies in identifying refactoring opportunities and implementing improvements. This chapter explores intentional approaches to refactoring with AI assistance.

13.1 When and Why to Refactor

13.1.1 recognising Refactoring Opportunities

Refactoring is most valuable in specific situations:

1. **Code smells** - Patterns in code that indicate deeper problems
2. **Technical debt** - Accumulated design or implementation shortcuts
3. **Changing requirements** - Evolving needs that strain existing designs
4. **Performance bottlenecks** - Areas where optimisation is needed
5. **Duplication** - Repeated code that could be consolidated
6. **Complexity** - Overly complicated logic that's difficult to understand

AI assistants excel at identifying these opportunities, especially when prompted to look for specific issues.

Example prompt:

```
Here's a section of our codebase that's becoming
difficult to maintain:
```

```python
def process_customer_data(customer):
    # Calculate base price
    if customer.tier == 'premium':
        base_price = 99.99
    elif customer.tier == 'standard':
        base_price = 49.99
    elif customer.tier == 'basic':
        base_price = 19.99
    else:
        base_price = 29.99

    # Apply discounts
    if customer.years > 5:
        discount = 0.15
    elif customer.years > 2:
        discount = 0.10
    elif customer.is_first_responder:
        discount = 0.20
    elif customer.is_teacher:
        discount = 0.10
    else:
        discount = 0

    # Calculate final price
    final_price = base_price * (1 - discount)

    # Generate output data
    result = {
        'customer_id': customer.id,
        'name': customer.name,
        'email': customer.email,
        'price': final_price,
        'discount': discount,
        'tier': customer.tier,
    }

    return result
```

Can you identify refactoring opportunities in this code? What code smells do you notice?

```
### The Business Case for Refactoring

Refactoring is sometimes seen as unnecessary by
non-technical stakeholders. AI can help articulate
```

```
the business value:

**Example prompt:**
```

I need to make a case to my manager for refactoring our payment processing module. The current code works, but it's difficult to maintain and extend.

How can I effectively communicate the business value of this refactoring effort? What specific metrics or outcomes should I highlight?

```
## AI-Assisted Code Improvements

### Identifying Refactoring Targets

AI can analyse code to identify specific
improvement opportunities:

**Example prompt:**
```

Could you analyse this function and suggest potential refactoring improvements? Focus on maintainability, readability, and adherence to best practices.

```
public List<Transaction> getTransactions(String
userId, Date startDate, Date endDate,
                                        String
                                        category,
                                        String
                                        merchantName,
                                        Double
                                        minAmount,
                                        Double
                                        maxAmount,
                                        boolean
                                        includeDeclined,
                                        String
                                        sortBy,
                                        String
                                        sortDirection,
                                        int
                                        pageSize,
                                        int
                                        pageNum) {
    List<Transaction> results = new ArrayList<>();
    Connection conn = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
```

```
try {
    conn = dataSource.getConnection();
    StringBuilder sql = new
    StringBuilder("SELECT * FROM transactions
    WHERE user_id = ?");

    List<Object> params = new ArrayList<>();
    params.add(userId);

    if (startDate != null) {
        sql.append(" AND transaction_date >=
        ?");
        params.add(startDate);
    }

    if (endDate != null) {
        sql.append(" AND transaction_date <=
        ?");
        params.add(endDate);
    }

    // 30+ more lines of similar parameter
    handling...

    // Pagination and sorting logic
    // More database handling code...

    while (rs.next()) {
        // Transform ResultSet to Transaction
        objects
        // 20+ lines of mapping code...
        results.add(transaction);
    }
} catch (SQLException e) {
    logger.error("Database error", e);
} finally {
    // Close resources
    if (rs != null) {
        try { rs.close(); } catch
        (SQLException e) { logger.error("Error
        closing ResultSet", e); }
    }
    if (stmt != null) {
        try { stmt.close(); } catch
```

```
            (SQLException e) { logger.error("Error
            closing Statement", e); }
        }
        if (conn != null) {
            try { conn.close(); } catch
            (SQLException e) { logger.error("Error
            closing Connection", e); }
        }
    }

    return results;
}
```

Suggesting Improved Designs

AI can propose architectural improvements:

Example prompt:

I have a monolithic class that handles user authentication, profile management, and notification preferences. It's become unwieldy at 500+ lines.

Based on the SOLID principles, how should I refactor this into more focused classes? What would the new class structure look like?

Implementing Specific Refactorings

AI can implement common refactoring patterns:

Example prompt:

I'd like to apply the "Extract Method" refactoring to this code:

```
function calculateTotalPrice(items, customer) {
  let subtotal = 0;
  for (const item of items) {
    subtotal += item.price * item.quantity;
  }

  let tax = 0;
  if (customer.state === 'CA') {
    tax = subtotal * 0.0725;
  } else if (customer.state === 'NY') {
    tax = subtotal * 0.045;
  } else if (customer.state === 'TX') {
    tax = subtotal * 0.0625;
```

```
  } else {
    tax = subtotal * 0.05;
  }

  let shipping = 0;
  if (subtotal >= 100) {
    shipping = 0;
  } else if (customer.isPremium) {
    shipping = 5.99;
  } else {
    shipping = 10.99;
  }

  return subtotal + tax + shipping;
}
```

Please extract methods for calculating the subtotal, tax, and shipping.

```
## Measuring Impact of Refactoring

Refactoring should produce measurable
improvements. AI can help identify metrics and
assess results.

### Quantitative Metrics

**Example prompt:**
```

I'm planning to refactor our API request handling code. What quantitative metrics should I measure before and after refactoring to demonstrate the impact?

```
AI might suggest:

- **Performance metrics**: Response time,
throughput, resource utilisation
- **Code metrics**: Cyclomatic complexity, lines
of code, method length
- **Testing metrics**: Test coverage, test
execution time
- **Maintenance metrics**: Time to implement new
features, bug fix duration
- **Error rates**: Exceptions, crashes, incorrect
results

### Qualitative Assessment
```

```
**Example prompt:**
```

Beyond quantitative metrics, what qualitative factors should I consider when evaluating the success of my refactoring efforts?

```
AI might suggest:

- **Developer feedback**: Team assessment of code
clarity and maintainability
- **Onboarding impact**: How quickly new team
members understand the code
- **Flexibility**: Ease of implementing new
requirements
- **Documentation needs**: Reduction in necessary
explanation
- **Knowledge distribution**: Less reliance on
specific team members

## Maintaining Functionality During Refactoring

One of the most critical aspects of refactoring is
preserving existing behaviour.

### Test-Driven Refactoring

**Example prompt:**
```

I want to refactor this payment processing function, but I need to ensure I don't break existing functionality. What testing approach would you recommend?

Here's the current function:

```
def process_payment(order_id, card_details,
amount):
    # Implementation details...
```

What types of tests should I write before refactoring, and how should I structure the refactoring process to minimise risk?

```
### Incremental Refactoring Approaches

**Example prompt:**
```

I need to refactor a large legacy class (800+ lines) that handles multiple responsibilities. What's the most effective step-by-step approach to break this down safely?

AI might suggest:

1. **Create a comprehensive test suite** first
2. **Identify clear boundaries** between responsibilities
3. **Extract one responsibility at a time**, starting with the most independent
4. **Introduce interfaces** to define clear contracts
5. **Refactor clients** to use the new interfaces
6. **Run tests after each incremental change**

Case Studies

Case Study 1: Simplifying Complex Conditionals

Original Code:

```javascript
function calculateDiscount(customer, order) {
  let discount = 0;

  if (customer.type === 'regular') {
    if (order.total > 100) {
      if (customer.years > 2) {
        discount = 0.15;
      } else {
        discount = 0.10;
      }
    } else if (order.total > 50) {
      if (customer.years > 2) {
        discount = 0.10;
      } else {
        discount = 0.05;
      }
    }
  } else if (customer.type === 'premium') {
    if (order.total > 100) {
      discount = 0.20;
    } else if (order.total > 50) {
      discount = 0.15;
    } else {
      discount = 0.10;
    }
  } else if (customer.type === 'vip') {
```

```
    discount = 0.25;
  }

  if (order.items.length > 10) {
    discount += 0.05;
  }

  return discount;
}
```

Refactoring Request:

Human prompt:

```
This discount calculation function has become
difficult to understand and maintain due to nested
conditionals. How would you refactor it to improve
readability and maintainability?
```

AI response:

```
I'll refactor this function to simplify the nested
conditionals using a few strategies:

1. Use early returns for special cases
2. Extract specialised functions for different
customer types
3. Use lookup tables for rules-based logic
4. Implement a more declarative approach

Here's the refactored version:

```javascript
function calculateDiscount(customer, order) {
  // Start with base discount and add volume
  discount later if applicable
  const baseDiscount =
  calculateBaseDiscount(customer, order);
  const volumeDiscount = order.items.length > 10 ?
  0.05 : 0;

  return baseDiscount + volumeDiscount;
}

function calculateBaseDiscount(customer, order) {
  // Special case for VIP customers
  if (customer.type === 'vip') {
```
```

```
    return 0.25;
  }

  // Use lookup table for discount rules
  const discountRules = {
    regular: {
      tier1: { threshold: 100, loyal: 0.15, new:
      0.10 },
      tier2: { threshold: 50, loyal: 0.10, new:
      0.05 },
      tier3: { threshold: 0, loyal: 0, new: 0 }
    },
    premium: {
      tier1: { threshold: 100, discount: 0.20 },
      tier2: { threshold: 50, discount: 0.15 },
      tier3: { threshold: 0, discount: 0.10 }
    }
  };

  // Handle regular customers
  if (customer.type === 'regular') {
    const isLoyal = customer.years > 2;
    const customerRates = discountRules.regular;

    if (order.total >
    customerRates.tier1.threshold) {
      return isLoyal ? customerRates.tier1.loyal :
      customerRates.tier1.new;
    } else if (order.total >
    customerRates.tier2.threshold) {
      return isLoyal ? customerRates.tier2.loyal :
      customerRates.tier2.new;
    }
    return 0;
  }

  // Handle premium customers
  if (customer.type === 'premium') {
    const premiumRates = discountRules.premium;

    if (order.total >
    premiumRates.tier1.threshold) {
      return premiumRates.tier1.discount;
    } else if (order.total >
    premiumRates.tier2.threshold) {
```

```
      return premiumRates.tier2.discount;
    }
    return premiumRates.tier3.discount;
  }

  // Default case - no discount
  return 0;
}
```

Benefits of this refactoring:

1. **Improved readability**: The main function is now much shorter and clearer
2. **Separation of concerns**: Different discount calculations are separated
3. **Configurability**: Discount rules are organised in a data structure that could easily be moved to a configuration file
4. **Maintainability**: Adding new customer types or rules is more straightforward
5. **Testability**: Each function can be tested independently

For even further improvement, you could refactor this to use a more object-oriented approach with strategy patterns, or use a rules engine for complex discount logic.

```
### Case Study 2: Reducing Duplication

**Original Code:**

```python
class UserService:
    def get_user_by_id(self, user_id):
        connection = database.connect()
        cursor = connection.cursor()
        cursor.execute("SELECT * FROM users WHERE
        id = %s", (user_id,))
        result = cursor.fetchone()
        cursor.close()
        connection.close()

        if result:
            return {
                'id': result[0],
                'username': result[1],
                'email': result[2],
                'created_at': result[3]
            }
```
```

```
        return None

    def get_user_by_email(self, email):
        connection = database.connect()
        cursor = connection.cursor()
        cursor.execute("SELECT * FROM users WHERE
        email = %s", (email,))
        result = cursor.fetchone()
        cursor.close()
        connection.close()

        if result:
            return {
                'id': result[0],
                'username': result[1],
                'email': result[2],
                'created_at': result[3]
            }
        return None

    def get_user_by_username(self, username):
        connection = database.connect()
        cursor = connection.cursor()
        cursor.execute("SELECT * FROM users WHERE
        username = %s", (username,))
        result = cursor.fetchone()
        cursor.close()
        connection.close()

        if result:
            return {
                'id': result[0],
                'username': result[1],
                'email': result[2],
                'created_at': result[3]
            }
        return None
```

Refactoring Request:

Human prompt:

```
This UserService class has significant duplication
in its methods. How would you refactor it to
reduce repetition while maintaining the same
functionality?
```

AI response:

```
There's significant duplication in this
`UserService` class. The three methods follow the
same pattern but just differ in the query
condition. Let's refactor to eliminate this
duplication:

```python
class UserService:
    def __get_user_by_query(self, query, params):
        """
        Private helper method to execute a query
        and return user data.

        Args:
            query: SQL query with placeholders
            params: Parameters to substitute in
            the query

        Returns:
            User dict or None if not found
        """
        connection = None
        cursor = None

        try:
            connection = database.connect()
            cursor = connection.cursor()
            cursor.execute(query, params)
            result = cursor.fetchone()

            if result:
                return {
                    'id': result[0],
                    'username': result[1],
                    'email': result[2],
                    'created_at': result[3]
                }
            return None

        finally:
            # Ensure resources are closed even if
            an exception occurs
            if cursor:
```
```

```
            cursor.close()
        if connection:
            connection.close()

    def get_user_by_id(self, user_id):
        """Get a user by their ID."""
        return self.__get_user_by_query(
            "SELECT * FROM users WHERE id = %s",
            (user_id,)
        )

    def get_user_by_email(self, email):
        """Get a user by their email address."""
        return self.__get_user_by_query(
            "SELECT * FROM users WHERE email =
            %s",
            (email,)
        )

    def get_user_by_username(self, username):
        """Get a user by their username."""
        return self.__get_user_by_query(
            "SELECT * FROM users WHERE username =
            %s",
            (username,)
        )
```

Key improvements in this refactoring:

1. **Extracted common code** into a private helper method `__get_user_by_query`
2. **Improved error handling** with a try/finally block to ensure resources are properly closed
3. **Added docstrings** to improve code documentation
4. **Made public methods concise** and focused on their specific responsibilities
5. **Reduced duplication** significantly - the code is now much more maintainable

Further improvements could include:

- Creating a User model class instead of returning dictionaries
- Using a context manager or connection pool for database connections
- Adding proper exception handling and logging
- Implementing a data access layer or repository pattern

This refactoring maintains exactly the same functionality while making

the code much more maintainable.

```
## Key Takeaways

- Refactoring is about improving code structure
without changing functionality
- AI assistants excel at identifying refactoring
opportunities and suggesting improvements
- Measure the impact of refactoring with both
quantitative and qualitative metrics
- Preserve functionality during refactoring
through comprehensive testing
- Common refactoring patterns like extracting
methods and simplifying conditionals can be
effectively implemented with AI assistance
- Incremental, test-driven approaches minimise
risk during refactoring
```

`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImNoYXB0ZXJzIn0= -->`

```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImNoYXB0ZXJzIiwiYm9va0
```

Chapter 14

Case Studies

> Every real problem is messier than the textbook version. That is where methodology earns its value.

In this chapter, we'll explore comprehensive case studies that demonstrate intentional prompting principles applied to real-world programming scenarios. These case studies illustrate how the six-step methodology and intentional prompting techniques come together to solve complex problems effectively.

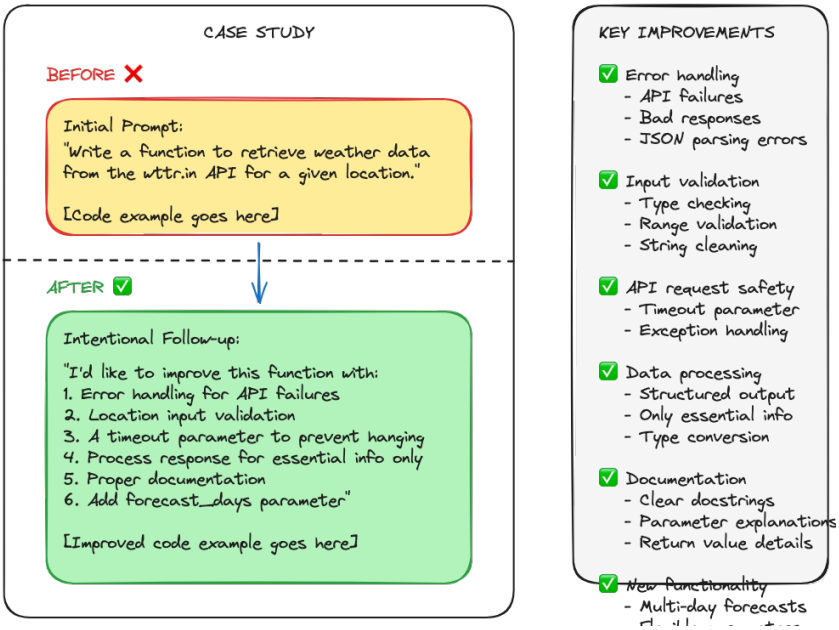

Figure 14.1: Before and After Case Study

14.1 Case Study 1: Building a Data Analysis Pipeline

14.1.1 The Problem

A research team needs to analyse a large dataset of customer transactions to identify purchasing patterns. They have a CSV file with millions of records containing transaction data including:

- Transaction ID
- Customer ID
- Date and time
- Product categories
- Purchase amount
- Payment method
- Store location

The team needs to: 1. Clean and preprocess the data 2. Identify the most frequently purchased product combinations 3. analyse how purchasing patterns change over time 4. Visualise the results

14.1.2 The Intentional Prompting Approach

14.1.2.1 Step 1: Restate the Problem

Human prompt:

```
I need to build a data analysis pipeline to
process customer transaction data. Let me restate
the problem:

We have a large CSV file (millions of rows) with
transaction data including transaction ID,
customer ID, date/time, product categories,
purchase amount, payment method, and store
location.

We need to:
1. Clean the data (handle missing values,
standardize formats)
2. Find frequently purchased product combinations
3. analyse how purchasing patterns change over
time
4. Create visualizations of the results

Is this understanding correct, and are there any
important aspects I've missed?
```

AI response:

```
Your understanding is correct! Your restatement
captures the main requirements of building a data
analysis pipeline for transaction data. Here are a
few additional considerations that might be
important:

1. **Data volume handling**: With millions of
rows, you'll need efficient processing techniques
that can handle large datasets. This might involve
streaming processing, chunking the data, or using
distributed computing frameworks.

2. **Performance considerations**: Some operations
like finding product combinations might be
computationally expensive. You might need
optimisation strategies.

3. **Data privacy**: Transaction data may contain
```

```
sensitive information. You might need to anonymize
or handle the data according to relevant privacy
regulations.

4. **Output format**: It's worth considering what
format the final results should be in (static
reports, interactive dashboards, raw data for
further analysis, etc.).

5. **Repeatability**: Will this be a one-time
analysis or something that needs to be run
regularly? If it's the latter, the pipeline should
be designed for automation and reproducibility.

6. **Data quality assessment**: Beyond basic
cleaning, you might want to include statistical
analysis to identify outliers or anomalies that
could skew the results.

With these considerations in mind, your
restatement effectively captures the core of the
problem.
```

14.1.2.2 Step 2: Identify Input and Output

Human prompt:

```
Let me define the specific inputs and outputs for
this pipeline:

INPUTS:
- A CSV file with the following columns:
  * transaction_id (string): Unique identifier for
  each transaction
  * customer_id (string): Identifier for the
  customer
  * timestamp (datetime): Date and time of the
  transaction
  * product_categories (string): Comma-separated
  list of purchased product categories
  * amount (float): Purchase amount in dollars
  * payment_method (string): Method of payment
  * store_location (string): Location identifier
  for the store

OUTPUTS:
```

```
1. Cleaned dataset (CSV or Parquet)
2. Top 20 most frequently purchased product
combinations (CSV)
3. Monthly purchasing trends by product category
(CSV)
4. Visualisations:
   * Bar chart of top product combinations
   * Line chart showing purchasing trends over
   time
   * Heatmap of purchase patterns by day of week
   and hour

Do these specifications adequately cover what's
needed for the pipeline?
```

AI response:

```
Your input and output specifications provide a
clear framework for the pipeline. They're
comprehensive and specific, which is excellent for
guiding the implementation. Here are a few
additional points to consider:

**For inputs:**

1. **File format details**: You might want to
specify if there are any header rows, quoting
patterns, or delimiter specifications for the CSV.

2. **Expected data sizes**: Approximate size of
the file (GB) and typical row count can help plan
for resource requirements.

3. **Data quality expectations**: Are there known
issues with the data? (e.g., are missing values
expected in certain columns, are there known
inconsistencies in formatting)

**For outputs:**

1. **Definition of "product combinations"**:
Clarify whether this means products purchased
together in a single transaction, or products
frequently purchased by the same customer across
different transactions.
```

```
2. **Granularity of time analysis**: You specified
monthly trends, but you might also want daily or
weekly analysis depending on the business needs.

3. **Additional metrics**: Consider adding:
   - Average basket size (number of items and
   monetary value)
   - Payment method distribution
   - Store location performance comparison

4. **Output file specifications**: For the CSV
outputs, specify whether headers should be
included and what the column structure should be.

5. **Visualisation formats**: Specify the file
formats for visualisations (PNG, PDF, interactive
HTML, etc.)

Overall, your specifications are solid and provide
a good foundation for the implementation. These
additional considerations can help make the
requirements even more precise.
```

14.1.2.3 Step 3: Work by Hand

In this step, we'd manually process a small sample of the data to understand the analysis process. For brevity, we'll focus on a simple example of finding product combinations.

Human work:

Sample data (5 transactions):

```
transaction_id,customer_id,timestamp,product_categories,amount,payment_met
T1001,C1,2023-01-15
14:30:00,"electronics,accessories",120.50,credit,store1
T1002,C2,2023-01-15
15:45:00,"groceries,dairy,produce",65.75,debit,store2
T1003,C1,2023-01-17
10:15:00,"electronics,videogames",210.99,credit,store1
T1004,C3,2023-01-18
09:30:00,"dairy,produce,meat",95.30,cash,store2
T1005,C2,2023-01-20
16:20:00,"groceries,dairy,bakery",45.60,debit,store2
```

To find product combinations: 1. Split each transaction's product_categories field 2. Generate all pair combinations 3. Count frequency of each

pair

For T1001: ["electronics", "accessories"] Pairs: [("electronics", "accessories")]

For T1002: ["groceries", "dairy", "produce"] Pairs: [("groceries", "dairy"), ("groceries", "produce"), ("dairy", "produce")]

...and so on for all transactions.

Counting frequencies: - ("dairy", "produce"): 2 - ("groceries", "dairy"): 2 - All others: 1

Most frequent combinations: ("dairy", "produce") and ("groceries", "dairy")

Human prompt:

```
I've worked through a small sample of the data
manually to understand the process of finding
product combinations. Here's what I did:

[The manual work shown above]

From this manual process, I can see that we need
to:
1. Parse the comma-separated product categories
2. Generate all possible pairs from each
transaction
3. Count occurrences of each pair across all
transactions
4. Sort by frequency to find the most common
combinations

This approach works for pairs, but for larger
combinations, we might need association rule
mining algorithms like Apriori. Does this manual
approach seem correct, and are there any important
considerations I'm missing?
```

AI response:

```
Your manual approach for finding product
combinations is sound! You've correctly identified
the key steps for analysing product pairs. Here
are some important considerations and extensions:

**Validations and extensions:**
```

```
1. **Multiple levels of combinations**: As you
noted, beyond pairs, you might want to find
triplets or larger combinations. Apriori or
FP-Growth algorithms would indeed be appropriate
for this.

2. **Association metrics**: Beyond just frequency
count, consider calculating:
   - Support (frequency of the combination
   relative to total transactions)
   - Confidence (likelihood of Y when X is
   purchased)
   - Lift (how much more likely Y is purchased
   when X is purchased, compared to normal)

3. **Performance for large datasets**: The naive
approach of generating all combinations can be
computationally expensive for transactions with
many items. For the full dataset:
   - Consider using optimised libraries like
   MLxtend or efficient implementations of Apriori
   - You might need to set a minimum support
   threshold to filter out rare combinations early

4. **Handling duplicates**: If a customer buys
multiple items in the same category in one
transaction (e.g., two electronics items), your
current approach correctly treats "electronics" as
appearing once in that transaction.

5. **Time-based analysis**: You might want to see
if popular combinations change over time (seasonal
patterns, etc.)

Your manual process demonstrates a good
understanding of the approach. For implementation
at scale, you'll want to leverage existing
libraries rather than building the combination
generator from scratch, especially as you move
beyond pairs to larger combinations.
```

14.1.2.4 Step 4: Write Pseudocode

Based on the manual exploration, we can now outline the solution in pseudocode.

Human prompt:

```
Based on my manual exploration, let me write
pseudocode for the data analysis pipeline:
```

FUNCTION clean_data(input_file): data = READ_CSV(input_file)

```
# Handle missing values
FILL_MISSING(data.product_categories, "unknown")
FILL_MISSING(data.payment_method, "unknown")
FILL_MISSING(data.store_location, "unknown")
DROP_ROWS_WHERE(data.transaction_id IS NULL OR data.timestamp IS NULL

# Standardize formats
data.timestamp = CONVERT_TO_DATETIME(data.timestamp)
data.amount = CONVERT_TO_FLOAT(data.amount)

RETURN data
```

FUNCTION find_product_combinations(data, min_support=0.001): # Initialize counter for product combinations combination_counts = EMPTY_COUNTER() total_transactions = COUNT_ROWS(data)

```
FOR each row IN data:
    products = SPLIT(row.product_categories, ",")
    product_combinations = GENERATE_COMBINATIONS(products, size=2)

    FOR each combination IN product_combinations:
        INCREMENT combination_counts[combination]

# Convert counts to support values
combination_support = EMPTY_DICT()
FOR combination, count IN combination_counts:
    combination_support[combination] = count / total_transactions

# Filter by minimum support
frequent_combinations = FILTER(combination_support >= min_support)

# Sort by support (descending)
frequent_combinations = SORT_DESCENDING(frequent_combinations)

RETURN frequent_combinations
```

FUNCTION analyze_time_patterns(data): # Add temporal features data['month'] = EXTRACT_MONTH(data.timestamp) data['year'] = EXTRACT_YEAR(data.timestamp) data['day_of_week'] = EXTRACT_WEEKDAY(data.timestamp) data['hour'] = EXTRACT_HOUR(data.timestamp)

```
# Group by month and product categories
monthly_trends = GROUP_BY(data, ['year', 'month', 'product_categories'],
                        AGGREGATE(COUNT(transaction_id), SUM(amount)))

# Group by day of week and hour
hourly_patterns = GROUP_BY(data, ['day_of_week', 'hour', 'product_categori
                         AGGREGATE(COUNT(transaction_id)))

RETURN monthly_trends, hourly_patterns
```

FUNCTION create_visualizations(product_combinations, monthly_trends, hourly_patterns): # Create bar chart of top combinations bar_chart = PLOT_BAR_CHART(TOP_N(product_combinations, 20))

```
# Create line chart of monthly trends
line_chart = PLOT_LINE_CHART(monthly_trends)

# Create heatmap of purchase patterns
heatmap = PLOT_HEATMAP(hourly_patterns)

RETURN bar_chart, line_chart, heatmap
```

FUNCTION main(input_file, output_dir): # Clean data cleaned_data = clean_data(input_file) SAVE_CSV(cleaned_data, output_dir + "/cleaned_data.csv")

```
# Find frequent product combinations
product_combinations = find_product_combinations(cleaned_data)
SAVE_CSV(TOP_N(product_combinations, 20), output_dir + "/top_combinations.

# analyse time patterns
monthly_trends, hourly_patterns = analyze_time_patterns(cleaned_data)
SAVE_CSV(monthly_trends, output_dir + "/monthly_trends.csv")

# Create visualizations
bar_chart, line_chart, heatmap = create_visualizations(
    product_combinations, monthly_trends, hourly_patterns)
SAVE_PNG(bar_chart, output_dir + "/top_combinations.png")
SAVE_PNG(line_chart, output_dir + "/monthly_trends.png")
SAVE_PNG(heatmap, output_dir + "/hourly_patterns.png")
```

```
Does this pseudocode cover all the necessary steps
for our data analysis pipeline? Are there any
inefficiencies or improvements you would suggest?
```

AI response:

```
Your pseudocode provides a good framework for the
data analysis pipeline! It covers the main steps
and follows a logical flow. Here are some
suggestions for improvements and considerations:

**Efficiency improvements:**

1. **Chunked processing**: For millions of rows,
you might want to process the data in chunks to
avoid memory issues:
```

FUNCTION clean_data(input_file): initialize empty dataframe for cleaned_data FOR each chunk IN READ_CSV_IN_CHUNKS(input_file, chunk_size=100000): # Apply cleaning operations to chunk cleaned_chunk = CLEAN_CHUNK(chunk) APPEND(cleaned_data, cleaned_chunk) RETURN cleaned_data

```
2. **optimised product combination generation**:
The current approach might be inefficient for
transactions with many products:
```

Chapter 15

Add a maximum combination size parameter

FUNCTION find_product_combinations(data, min_support=0.001, max_combination_size=2): # Use an efficient algorithm like Apriori instead of manual combination generation frequent_itemsets = APRIORI_ALGORITHM(data.product_categories, min_support=min_support, max_length=max_combination_size) RETURN frequent_itemsets

```
**Additional considerations:**

1. **Data validation**: Add checks for data
quality:
```

FUNCTION clean_data(input_file): # …existing code…

```
# Add data validation
LOG_STATS(COUNT_MISSING_VALUES(data))
LOG_STATS(COUNT_UNIQUE_VALUES(data.product_categories))
VALIDATE_NUMERIC_RANGE(data.amount, min=0)

# ...rest of function...

2. **Error handling**: Add error handling for
robust processing:
```

FUNCTION main(input_file, output_dir): TRY: # …existing code… CATCH FileNotFoundError: LOG_ERROR("Input file not found")

CATCH MemoryError: LOG_ERROR("Insufficient memory, try reducing chunk size") FINALLY: CLEAN_UP_TEMP_FILES()

```
3. **Parallelization**: For large datasets,
consider parallel processing:
```

FUNCTION find_product_combinations(data, min_support=0.001): # Split data into partitions partitions = SPLIT_DATA(data, num_partitions=NUM_CORES)

```
# Process partitions in parallel
partial_results = PARALLEL_MAP(PROCESS_PARTITION, partitions)

# Combine results
combined_results = COMBINE_PARTIAL_RESULTS(partial_results)
RETURN combined_results
```

```
4. **Progress tracking**: For long-running
operations, add progress tracking:
```

FUNCTION clean_data(input_file): total_chunks = ESTIMATE_TOTAL_CHUNKS(input_file, chunk_size) FOR i, chunk IN ENUMERATE(READ_CSV_IN_CHUNKS(input_file)): LOG_PROGRESS(f"Processing chunk {i}/{total_chunks}") # ...process chunk...

```
These improvements would make your pipeline more
robust, efficient, and suitable for large-scale
data processing. Your current pseudocode is a
great starting point, and these suggestions help
address some common challenges in real-world data
processing.
```

15.0.0.1 Step 5: Convert to Code

Now let's implement the solution in Python, focusing on the product combination analysis.

Human prompt:

```
Let's convert the pseudocode for finding product
combinations into Python code using pandas and
mlxtend for the Apriori algorithm:

```python
import pandas as pd
from mlxtend.frequent_patterns import apriori,
association_rules
from mlxtend.preprocessing import
```
```

```
TransactionEncoder
import numpy as np

def find_product_combinations(data,
min_support=0.001, max_len=3):
    """
    Find frequent product combinations using the
    Apriori algorithm.

    Args:
        data (pd.DataFrame): DataFrame containing
        transaction data
        min_support (float): Minimum support
        threshold (default: 0.001)
        max_len (int): Maximum combination size
        (default: 3)

    Returns:
        pd.DataFrame: DataFrame with frequent
        itemsets and their support
    """
    # Extract product categories and split into
    lists
    transactions =
    data['product_categories'].str.split(',').tolist()

    # Convert transactions to one-hot encoded
    format
    te = TransactionEncoder()
    te_ary = te.fit_transform(transactions)
    df_encoded = pd.DataFrame(te_ary,
    columns=te.columns_)

    # Apply Apriori algorithm
    frequent_itemsets = apriori(
        df_encoded,
        min_support=min_support,
        max_len=max_len,
        use_colnames=True
    )

    # Filter itemsets with more than one item
    frequent_itemsets =
    frequent_itemsets[frequent_itemsets['itemsets'].apply(lambda
    x: len(x) > 1)]
```

```
    # Sort by support (descending)
    frequent_itemsets =
    frequent_itemsets.sort_values('support',
    ascending=False)

    # Format the output
    result = pd.DataFrame({
        'combination':
        frequent_itemsets['itemsets'].apply(lambda
        x: ', '.join(list(x))),
        'support': frequent_itemsets['support'],
        'transaction_count':
        (frequent_itemsets['support'] *
        len(data)).astype(int)
    })

    return result

def clean_data(file_path, chunk_size=100000):
    """
    Clean and preprocess the transaction data.

    Args:
        file_path (str): Path to the CSV file
        chunk_size (int): Number of rows to
        process at once

    Returns:
        pd.DataFrame: Cleaned and preprocessed
        data
    """
    # Initialize an empty list to store chunks
    chunks = []

    # Process file in chunks
    for chunk in pd.read_csv(file_path,
    chunksize=chunk_size):
        # Fill missing values
        chunk['product_categories'] =
        chunk['product_categories'].fillna('unknown')
        chunk['payment_method'] =
        chunk['payment_method'].fillna('unknown')
        chunk['store_location'] =
        chunk['store_location'].fillna('unknown')
```

```
        # Drop rows with missing essential data
        chunk =
        chunk.dropna(subset=['transaction_id',
        'timestamp'])

        # Standardize formats
        chunk['timestamp'] =
        pd.to_datetime(chunk['timestamp'])
        chunk['amount'] =
        pd.to_numeric(chunk['amount'],
        errors='coerce')

        # Append the cleaned chunk
        chunks.append(chunk)

    # Combine all chunks
    cleaned_data = pd.concat(chunks,
    ignore_index=True)

    return cleaned_data
```

Is this implementation correct and efficient? Are there any issues or optimisations I should consider?

`**AI response:**`

Your implementation is solid and follows good practices for processing large datasets! Here are some thoughts and potential optimisations:

Strengths:

1. Using `chunksize` for reading large CSV files is excellent
2. The `mlxtend` library is a good choice for implementing Apriori efficiently
3. The code is well-structured with clear documentation
4. You've added appropriate data cleaning steps

Potential improvements:

1. **Memory optimisation for very large datasets:**

```
def find_product_combinations(data,
min_support=0.001, max_len=3):
    # For extremely large datasets, we might want
    to sample first
    if len(data) > 10_000_000:  # Arbitrary
    threshold
        data = data.sample(n=10_000_000,
```

```
        random_state=42)
        print(f"Data sampled to {len(data)} rows
        due to size")

    # Rest of your function...
```

2. **Error handling and logging:**

```
def clean_data(file_path, chunk_size=100000):
    chunks = []
    total_rows = 0
    invalid_rows = 0

    try:
        for i, chunk in
        enumerate(pd.read_csv(file_path,
        chunksize=chunk_size)):
            initial_rows = len(chunk)
            total_rows += initial_rows

            # Your cleaning code...

            invalid_rows += initial_rows -
            len(chunk)
            chunks.append(chunk)

            if i % 10 == 0:
                print(f"Processed {i+1} chunks
                ({total_rows} rows)...")

        print(f"Cleaning complete. Processed
        {total_rows} rows, removed {invalid_rows}
        invalid rows.")
    except Exception as e:
        print(f"Error during data cleaning:
        {str(e)}")
        raise
```

3. **Performance optimisation for Apriori on sparse data:**

```
# If your dataset has many distinct products but
each transaction has few,
# use sparse representation for better performance
from scipy.sparse import csr_matrix

te_ary = te.fit_transform(transactions)
```

```
sparse_te_ary = csr_matrix(te_ary)

# Then use the sparse matrix with apriori if
mlxtend supports it
# Otherwise, for very sparse data, consider
alternative implementations
```

4. **Data validation before processing:**

```
def find_product_combinations(data,
min_support=0.001, max_len=3):
    # Validate input data
    if 'product_categories' not in data.columns:
        raise ValueError("DataFrame must contain
        'product_categories' column")

    # Check for empty values post-cleaning
    empty_cats =
    data['product_categories'].isin(['',
    'unknown']).sum()
    if empty_cats > 0:
        print(f"Warning: {empty_cats} rows have
        empty or unknown product categories")

    # Rest of your function...
```

5. **Consider adding time-based analysis:**

```
def find_seasonal_combinations(data,
time_period='month'):
    """Find how product combinations vary over
    time."""
    # Add time period column
    if time_period == 'month':
        data['period'] =
        data['timestamp'].dt.to_period('M')
    elif time_period == 'quarter':
        data['period'] =
        data['timestamp'].dt.to_period('Q')

    periods = data['period'].unique()
    results = {}

    for period in periods:
        period_data = data[data['period'] ==
        period]
```

```
        period_combos =
        find_product_combinations(period_data)
        results[period] = period_combos.head(10)
        # Top 10 for each period

    return results
```

Overall, your implementation is well-designed for the task at hand. These suggestions are refinements that might be helpful in specific scenarios, but your core approach is sound!

```
#### Step 6: Test with Data

For this step, we would test our implementation
with real data, starting with a small sample and
progressively increasing to the full dataset.

**Human prompt:**
```

Let's test our implementation with a small sample dataset first:

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# Create sample data
np.random.seed(42)
n_samples = 1000

# Generate transaction IDs
transaction_ids = [f'T{i+1:04d}' for i in
range(n_samples)]

# Generate customer IDs (100 unique customers)
customer_ids = [f'C{np.random.randint(1,
101):03d}' for _ in range(n_samples)]

# Generate timestamps (last 90 days)
base_date = datetime.now() - timedelta(days=90)
timestamps = [base_date +
timedelta(days=np.random.randint(0, 90),

hours=np.random.randint(0,
                                    24),

minutes=np.random.randint(0,
```

```
                                    60))
             for _ in range(n_samples)]

# Product categories
categories = ['electronics', 'accessories',
'clothing', 'shoes', 'books',
              'groceries', 'dairy', 'produce',
              'meat', 'bakery', 'beverages',
              'snacks', 'beauty', 'health',
              'kitchen', 'toys', 'sports',
              'automotive', 'garden', 'furniture']

# Generate product categories (1-4 categories per
transaction)
product_categories = []
for _ in range(n_samples):
    n_cats = np.random.randint(1, 5)
    cats = np.random.choice(categories,
    size=n_cats, replace=False)
    product_categories.append(','.join(cats))

# Generate amounts
amounts = np.random.uniform(10, 500,
n_samples).round(2)

# Payment methods
payment_methods = np.random.choice(['credit',
'debit', 'cash', 'mobile'],
                                   size=n_samples,
                                   p=[0.4, 0.3,
                                   0.2, 0.1])

# Store locations
store_locations = np.random.choice(['store1',
'store2', 'store3', 'online'],
                                  size=n_samples,
                                  p=[0.3, 0.3,
                                  0.2, 0.2])

# Create DataFrame
sample_data = pd.DataFrame({
    'transaction_id': transaction_ids,
    'customer_id': customer_ids,
    'timestamp': timestamps,
    'product_categories': product_categories,
```

```
    'amount': amounts,
    'payment_method': payment_methods,
    'store_location': store_locations
})

# Save to CSV
sample_data.to_csv('sample_transactions.csv',
index=False)

# Test our implementation
# 1. Find product combinations
results = find_product_combinations(sample_data,
min_support=0.01, max_len=3)
print(f"Found {len(results)} frequent product
combinations")
print("\nTop 10 product combinations:")
print(results.head(10))

# 2. Check for issues or unexpected results
print("\nData statistics:")
print(f"Total transactions: {len(sample_data)}")
print(f"Unique product categories:
{len(set(','.join(product_categories).split(',')))}")
unique_combos = set()
for cats in product_categories:
    cat_list = cats.split(',')
    for i in range(len(cat_list)):
        for j in range(i+1, len(cat_list)):
            unique_combos.add(tuple(sorted([cat_list[i],
            cat_list[j]])))
print(f"Unique product pairs in data:
{len(unique_combos)}")
```

What issues might we encounter when scaling to the full dataset, and how should we address them?

`**AI response:**`

Your test approach with synthetic data is excellent for validating the implementation! Here are the potential issues you might encounter when scaling to the full dataset and strategies to address them:

Scaling Issues and Solutions:

1. **Memory Consumption**
    - **Issue**: The TransactionEncoder creates a one-hot encoded matrix which can be extremely memory-intensive for datasets

with millions of transactions and many unique products.
- **Solution**:
  - Process in batches based on time periods (e.g., analyse one month at a time)
  - Use sampling techniques for initial analysis
  - Implement a more memory-efficient version of Apriori or consider alternatives like FP-Growth
  - Use sparse matrix representations where possible

2. **Computation Time**
   - **Issue**: Apriori's performance degrades significantly with low support thresholds on large datasets.
   - **Solution**:
     - Start with higher minimum support values and gradually decrease
     - Use parallel processing libraries (e.g., Dask, PySpark)
     - Implement an incremental approach that processes data in chunks and merges results
     - Consider using GPU-accelerated implementations for very large datasets
3. **Long Tail of Infrequent Items**
   - **Issue**: With millions of transactions, you might get thousands of combinations with low support.
   - **Solution**:
     - Focus on combinations with higher lift or confidence scores
     - Implement filtering for statistically significant associations
     - Group similar products into categories for higher-level analysis
4. **Data Quality at Scale**
   - **Issue**: Large datasets often have more varied quality issues that become apparent only at scale.
   - **Solution**:
     - Implement more robust data cleaning steps
     - Add comprehensive logging of anomalies
     - Create data quality dashboards to monitor issues
5. **Time-Based Changes**
   - **Issue**: Product associations may change over time, and analysing everything together might obscure patterns.
   - **Solution**:
     - Implement time-window analysis
     - Compare associations across different time periods
     - Look for emerging or declining associations

Implementation Improvements for Scale:

```
# For very large datasets, consider this approach:
```

```
def analyze_large_dataset(file_path, output_dir):
    # 1. First pass: Get data statistics and
    sample
    print("First pass: Collecting statistics...")
    stats = collect_data_stats(file_path)

    # 2. Determine optimal processing approach
    based on stats
    if stats['total_rows'] > 10_000_000:
        print("Very large dataset detected. Using
        distributed processing...")
        # Initialize Dask or Spark
        import dask.dataframe as dd
        ddf = dd.read_csv(file_path)
        # Process with distributed framework
    else:
        print("Processing with pandas...")
        # Process with your current approach

    # 3. Use progressive sampling for quick
    insights
    sample_sizes = [10000, 100000, 1000000]
    for size in sample_sizes:
        if size > stats['total_rows']:
            continue
        print(f"analysing sample of {size}
        transactions...")
        sample = get_stratified_sample(file_path,
        size)
        # Run analysis on progressively larger
        samples
        results =
        find_product_combinations(sample)
        save_results(results,
        f"{output_dir}/sample_{size}_results.csv")

    # 4. Full analysis with optimised parameters
    print("Running full analysis with optimised
    parameters...")
    # Use parameters learned from sample analysis
    full_results =
    process_in_time_windows(file_path)

    return full_results
```

Your test setup is well-designed. When scaling to millions of rows, implementing these strategies will help manage both memory consumption and computation time while still extracting valuable insights from the data.

```
### Lessons Learned

This case study demonstrates several key
intentional prompting principles:

1. **Thorough problem restatement** helped
identify additional considerations beyond the
basic requirements
2. **Detailed input/output specification**
clarified the exact needs of the pipeline
3. **Manual exploration** of a small data sample
provided crucial insights into the required
algorithms
4. **Pseudocode development** allowed for
structural planning before implementation
5. **Implementation with AI assistance**
incorporated best practices and optimisations
6. **Testing with synthetic data** verified the
approach before scaling to the full dataset

The intentional prompting methodology led to a
solution that was:
- Efficient for large-scale data processing
- Robust to data quality issues
- Flexible enough to handle various analysis
requirements
- Well-tested before deployment to the full
dataset

## Case Study 2: Building a Weather Dashboard

### The Problem

A developer needs to build a weather dashboard
application in Python. The application should:

1. utilise a weather data module to retrieve
current and forecast information
2. Process and display weather data in a
user-friendly interface
3. Allow users to search for different locations
```

```
4. Provide visualisations of weather trends

### The Intentional Prompting Approach

#### Step 1: Restate the Problem

**Human prompt:**
```

I need to create a weather dashboard in Python that uses a provided module to access weather data and displays it in a user-friendly interface. Can you help me restate this problem more clearly to ensure I understand all requirements?

```
**AI response:**
```

This problem involves creating a Python application that: 1. Utilises an existing weather data module to retrieve weather information 2. Processes that data appropriately 3. Presents the information in a clear, user-friendly interface 4. Allows users to interact with the dashboard to view different weather metrics

The key components you'll need to develop are: - Functions to retrieve and process data from the provided module - A display system (likely console-based) to show weather information - User interaction components to allow navigation and selection - Visualisation elements for presenting weather data graphically

```
**Human follow-up prompt (clarification):**
```

That's helpful. Could you help me understand what types of weather data are typically available through weather APIs, and what visualisations might be most useful for users? This will help me plan my dashboard layout.

```
#### Step 2: Identify Input and Output
Requirements

**Human prompt:**
```

Based on our discussion, let me identify the specific inputs and outputs for my weather dashboard:

Inputs: - Location information (city name or coordinates) - Type of weather data requested (current, forecast) - Time period for forecast data

Outputs: - Current weather conditions display - Visual representations of temperature, humidity, etc. - Forecast for upcoming days - User navigation menu

Does this cover the essential inputs and outputs? Is there anything important I'm missing?

AI response:

Your input/output identification is strong! A few additional considerations:

Additional inputs to consider: - User preferences for temperature units (Celsius/Fahrenheit) - Display preferences (text-only vs. graphical visualisation) - Refresh rate for data updates

Additional outputs to consider: - Error messages for handling unavailable data or connection issues - Status indicators for data freshness - Historical data comparisons (if available) - Weather alerts or warnings

Step 3: Work Problem by Hand

Human prompt:

Before we start coding, I want to sketch out the user interaction flow for the dashboard:

1. User starts program
2. Program displays welcome and prompts for location
3. User enters location
4. Program fetches current weather data
5. Program displays main dashboard with:
    - Current temperature and conditions
    - Today's high/low
    - Humidity and wind
    - Simple 3-day forecast
6. Program shows menu options:
    - View detailed forecast
    - View temperature graph
    - Change location
    - Exit
7. User selects option
8. Program displays selected information
9. Return to menu (step 6)

Does this flow make sense? Would you structure the user experience differently?

Step 4: Write Pseudocode

Human prompt:

Now that we have a clear understanding of the problem and user flow, could you help me write pseudocode for this weather dashboard? I want

to focus on creating a modular design with separate functions for data retrieval, display, and user interaction.

```
**AI response:**
```python
# Main program structure
function main():
    display_welcome()
    location = get_location_from_user()
    units = get_preferred_units()

    while True:
        weather_data =
        fetch_weather_data(location, units)
        display_dashboard(weather_data)
        choice = display_menu()

        if choice == "forecast":
            display_detailed_forecast(weather_data)
        else if choice == "graph":
            display_temperature_graph(weather_data)
        else if choice == "location":
            location = get_location_from_user()
        else if choice == "refresh":
            continue  # Will loop back and refresh
            data
        else if choice == "exit":
            break

# Data functions
function fetch_weather_data(location, units):
    # Use provided weather module to get data
    current_weather =
    weather_module.get_current(location, units)
    forecast =
    weather_module.get_forecast(location, units)
    return {current: current_weather, forecast:
    forecast}

# Display functions
function display_dashboard(weather_data):
    clear_screen()
    display_current_conditions(weather_data["current"])
    display_simple_forecast(weather_data["forecast"])
```
```

```
function display_detailed_forecast(weather_data):
    clear_screen()
    for day in weather_data["forecast"]:
        display_day_forecast(day)
    wait_for_user()

function display_temperature_graph(weather_data):
    clear_screen()
    temperatures =
    extract_temperatures(weather_data["forecast"])
    dates =
    extract_dates(weather_data["forecast"])
    create_line_graph(dates, temperatures)
    wait_for_user()

# User interaction functions
function display_menu():
    print menu options
    return get_valid_choice()

function get_location_from_user():
    prompt for location
    validate location
    return location
```

Follow-up prompt (challenge):

```
The pseudocode looks good overall. I noticed that
error handling isn't explicitly included. Could
you expand the pseudocode to include error
handling for:

1. Invalid location input
2. Weather service connection failures
3. Handling missing data in the weather response
```

15.0.0.2 Step 5: Convert to Python Code

The developer starts implementing the solution based on the pseudocode, focusing on modularity, error handling, and user experience.

15.0.0.3 Step 6: Test with Data

Human prompt:

```
Now I want to test this code with some sample
data. Can you provide a sample data structure that
```

```
would be returned by:

1. weather_api.get_current()
2. weather_api.get_forecast()

Also, what test cases should I consider to ensure
my dashboard handles all possible scenarios
correctly?
```

AI response:

```
Here are sample data structures that might be
returned by the weather API:

```python
# Sample current weather data
sample_current = {
    'location': 'New York',
    'timestamp': '2023-09-15 14:30',
    'temperature': 22,
    'feels_like': 24,
    'condition': 'Partly Cloudy',
    'humidity': 65,
    'wind_speed': 8,
    'wind_direction': 'NE',
    'pressure': 1012,
    'visibility': 10,
    'uv_index': 5,
    'units': 'metric'
}

# Sample forecast data (list of days)
sample_forecast = [
    {
        'date': '2023-09-15',
        'day_of_week': 'Friday',
        'high': 24,
        'low': 18,
        'condition': 'Partly Cloudy',
        'precipitation_chance': 20,
        'humidity': 65,
        'wind_speed': 8,
        'wind_direction': 'NE',
        'units': 'metric'
    },
    # Additional days would follow...
```
```

```
]
```

Test cases to consider:

1. **Happy path**:
   - Valid location with complete weather data
   - Multiple days of forecast
2. **Data issues**:
   - Missing fields in current weather
   - Partial forecast data available
   - Empty forecast list
3. **Connection issues**:
   - API timeout
   - Service unavailable
4. **User input scenarios**:
   - Invalid location name
   - Empty input
   - Special characters in location name ““

15.0.1 Before and After Example: Improving a Key Function

One of the critical functions in the application is the data retrieval function. Let's examine how intentional prompting improved this function:

15.0.1.1 Before: Initial Data Retrieval Function

```python
def fetch_weather_data(location, units):
    """Fetch weather data from API."""
    current_weather =
    weather_api.get_current(location, units)
    forecast = weather_api.get_forecast(location,
    units, days=5)
    return {"current": current_weather,
    "forecast": forecast}
```

15.0.1.2 Refinement Process Prompt

```
The fetch_weather_data() function needs
improvement. It doesn't handle errors or validate
the data. How can I make it more robust to:

1. Handle connection errors
2. Validate returned data structure
3. Handle missing or incomplete data
```

4. Provide meaningful error messages to the user

15.0.1.3 After: Improved Data Retrieval Function

```
def fetch_weather_data(location, units):
    """
    Fetch weather data from API with comprehensive
    error handling.

    Args:
        location: City name or location
        units: 'metric' or 'imperial'

    Returns:
        Dictionary containing current weather and
        forecast

    Raises:
        ConnectionError: If unable to connect to
        weather service
        ValueError: If location is invalid or data
        is incomplete
    """
    try:
        # Attempt to get current weather
        current_weather =
        weather_api.get_current(location, units)

        # Validate current weather data
        required_current_fields = ['temperature',
        'condition', 'humidity', 'wind_speed']
        missing_fields = [field for field in
        required_current_fields if field not in
        current_weather]

        if missing_fields:
            # Handle missing fields by providing
            defaults
            for field in missing_fields:
                current_weather[field] = "N/A"

            # Log the issue but don't fail
            completely
            print(f"Warning: Some current weather
            data is unavailable: {',
```

```
            '.join(missing_fields)}")

        # Attempt to get forecast with retry
        mechanism
        max_retries = 3
        retry_count = 0
        forecast = None

        while retry_count < max_retries and
        forecast is None:
            try:
                forecast =
                weather_api.get_forecast(location,
                units, days=5)
                break
            except Exception as e:
                retry_count += 1
                if retry_count >= max_retries:
                    print(f"Warning: Could not
                    retrieve forecast data. Using
                    current conditions only.")
                    forecast = []  # Empty
                    forecast as fallback
                else:
                    time.sleep(1)  # Wait before
                    retrying

        # Ensure forecast is a list (even if
        empty)
        if not isinstance(forecast, list):
            forecast = []

        return {
            "current": current_weather,
            "forecast": forecast,
            "location": location,
            "units": units,
            "timestamp": time.strftime("%Y-%m-%d
            %H:%M")
        }

    except ConnectionError:
        raise ConnectionError("Failed to connect
        to weather service. Please check your
        internet connection.")
```

```
    except ValueError as e:
        raise ValueError(f"Invalid location:
        {location}. Please check spelling and try
        again.")
    except Exception as e:
        # Convert any other exceptions from the
        API to something we can handle
        raise Exception(f"Error fetching weather
        data: {str(e)}")
```

15.0.2 Lessons Learned

This case study demonstrates several key aspects of intentional prompting:

1. **Thorough problem exploration** - The developer took time to understand requirements and data structures before coding
2. **Modular design** - The solution was broken down into focused functions with clear responsibilities
3. **Proactive error handling** - Multiple failure scenarios were identified and handled gracefully
4. **Iterative refinement** - The initial simple implementation was systematically improved based on specific requirements
5. **Test scenario preparation** - Sample data and test cases were identified before testing began

The intentional approach resulted in a robust, maintainable application that properly handles edge cases and provides a good user experience, even when dealing with external data sources that may be unreliable.

15.1 Case Study 3: Debugging a Complex Bug

For our final case study, we'll look at a debugging scenario where intentional prompting techniques help solve a challenging issue.

15.1.1 The Problem

A team is experiencing intermittent failures in their e-commerce checkout system. The checkout process occasionally fails with a generic error message, but only for certain users and at seemingly random times.

15.1.2 The Intentional Prompting Approach

15.1.2.1 Step 1: Restate the Problem

Human prompt:

```
We're experiencing intermittent failures in our
checkout system. Let me restate the problem:

Users occasionally receive a generic "Unable to
complete checkout" error when attempting to
purchase items. The issue:
- Only affects approximately 5% of checkout
attempts
- Seems more common during peak traffic hours
- Doesn't consistently affect the same users
- Doesn't correlate with specific products or
payment methods
- Leaves limited information in our standard logs

Our goal is to identify the root cause and
implement a fix to eliminate these checkout
failures.

Is this an appropriate restatement of the
debugging challenge?
```

15.1.2.2 Step 2: Identify Information Needed

Human prompt:

```
To debug this issue effectively, we need to gather
specific information:

INPUTS (Information sources):
- Application logs with enhanced logging enabled
- Database transaction logs
- Server performance metrics during failures
- Network traffic patterns
- User session data for failed checkouts
- Payment processor API logs

OUTPUTS (What we need to determine):
- Exact point of failure in the checkout process
- Conditions that trigger the failure
- Root cause of the issue
- Potential fix or mitigation strategy
```

```
Are there other information sources or outputs we
should consider to effectively debug this
intermittent issue?
```

The debugging case study would continue through a systematic investigation process, demonstrating how intentional prompting techniques help narrow down the cause of complex bugs.

15.2 Key Takeaways

From these case studies, we can extract several important lessons about intentional prompting:

1. **Structured methodology yields better results**
   - Following all six steps produces more robust solutions than jumping straight to implementation
   - Each step builds on the previous one, creating a solid foundation
2. **Problem exploration is time well spent**
   - Restating the problem and identifying inputs/outputs reveals critical requirements
   - Manual exploration uncovers edge cases that might otherwise be missed
3. **AI assistance enhances human capability**
   - AI helps identify potential issues and optimisations
   - The human developer maintains control over the approach and design decisions
4. **Testing is essential**
   - Starting with small, synthetic datasets verifies the approach before scaling
   - Comprehensive testing reveals potential issues early
5. **Real-world complexity requires adaptability**
   - Initial solutions often need refinement as scale and complexity increase
   - The methodology provides a framework for iterative improvement

Part V

Advanced Topics

Chapter 16

Scaling to Complex Projects

> The practices that work for a single function do not automatically scale. That is the challenge.

So far, we've primarily focused on applying intentional prompting techniques to isolated functions and small-scale problems. In real-world development, however, you'll often work on complex projects with multiple components, larger codebases, and teams of developers. This chapter explores strategies for scaling intentional prompting practices to handle this increased complexity.

16.1 Beyond Single-File Programs

16.1.1 The Challenges of Scale

As projects grow in size and complexity, several challenges emerge:

1. **Architecture becomes more important** - The high-level organisation of code has a greater impact on maintainability
2. **Component interactions** introduce new failure modes and dependencies
3. **Consistency across the codebase** becomes harder to maintain
4. **Knowledge distribution** among team members becomes uneven
5. **Contextual understanding** becomes more difficult as no single person knows every detail

These challenges require adaptations to our intentional prompting approach.

16.1.2 Intentional Architecture Design

For larger projects, architectural planning becomes an essential first step. AI can be a valuable partner in this process:

Human prompt:

```
I'm designing a microservices architecture for an
e-commerce platform. The main functions include:
- Product catalogue
- User accounts
- Cart management
- Order processing
- Payment handling
- Shipping and fulfillment
- Analytics

What would be a sensible way to divide these into
microservices? What are the key considerations for
service boundaries, data ownership, and
interaction patterns?
```

AI response:

```
Here's a sensible approach to dividing your
e-commerce platform into microservices, along with
key considerations:

### Proposed Microservice Architecture

1. **Product Service**
   - Product catalogue and management
   - Categories and taxonomies
   - Product search and filtering capabilities
   - Inventory and stock information

2. **User Service**
   - User accounts and profiles
   - Authentication and authorization
   - User preferences and settings
   - Address book management

3. **Cart Service**
   - Shopping cart management
   - Cart item operations (add, remove, update)
   - Cart pricing calculations
   - Saved/wishlist items
```

```
4. **Order Service**
   - Order creation and management
   - Order status tracking
   - Order history
   - Returns and cancellations

5. **Payment Service**
   - Payment processing
   - Payment method management
   - Refund handling
   - Fraud detection (or integration with fraud
   services)

6. **Fulfillment Service**
   - Shipping and delivery management
   - Carrier integrations
   - Tracking information
   - Warehouse/inventory coordination

7. **Analytics Service**
   - Data collection and processing
   - Reporting and dashboards
   - Business intelligence
   - Customer behaviour analysis

### Key Considerations

**Service Boundaries:**

1. **Business Capability Alignment**: Each service
should align with a specific business capability
and have a clearly defined responsibility.

2. **Domain-Driven Design (DDD)**: Consider using
DDD principles to identify bounded contexts that
form natural service boundaries.

3. **Autonomy**: Services should be able to
function independently as much as possible, with
their own databases and minimal dependencies.

4. **Size Balance**: Neither too large (becoming a
monolith) nor too small (creating excessive
inter-service communication).
```

```
**Data Ownership:**

1. **Single Source of Truth**: Each type of data
should be owned by exactly one service that serves
as the authority for that data.

2. **Data Duplication vs. Service Calls**: Decide
when to duplicate data across services (for
performance/autonomy) versus making
service-to-service calls.

3. **Consistency Models**: Determine where strong
consistency is needed (typically within a service)
and where eventual consistency is acceptable
(typically across services).

4. **Data Evolution**: Plan for how data schemas
can evolve while maintaining compatibility.

**Interaction Patterns:**

1. **Synchronous vs. Asynchronous**: Choose
between direct API calls (synchronous) and
message/event-based communication (asynchronous)
based on the needs of each interaction.

2. **Event-Driven Architecture**: Consider an
event bus for broadcasting state changes, which
can help decouple services.

3. **API Gateway**: Implement an API gateway for
client applications to simplify their interaction
with multiple backend services.

4. **Service Discovery**: Implement a service
registry so services can find each other
dynamically.

5. **Circuit Breakers**: Implement circuit
breakers to prevent cascading failures when
dependent services are unavailable.

### Additional Considerations
```

```
- **Authentication/Authorization**: This could be
handled by the User Service or a dedicated Auth
Service, potentially using an identity provider.

- **Notification Service**: Consider adding a
dedicated service for handling all communications
(email, SMS, push notifications).

- **Review/Rating Service**: For product reviews
and ratings, which crosses concerns between
products and users.

- **Search Service**: For complex search
functionality across multiple domains, consider a
dedicated service using specialised search
technology.

This architecture provides a balanced approach
that separates concerns while maintaining logical
cohesion within each service.
```

16.1.3 Component-Level Intentional Prompting

For complex projects, apply the six-step methodology at the component level:

1. **Restate the component's purpose and responsibilities**
2. **Identify component interfaces** (inputs and outputs)
3. **Work through component interactions** with sequence diagrams or similar tools
4. **Write pseudocode for key interfaces and interactions**
5. **Implement the component** while maintaining clear boundaries
6. **Test the component** both in isolation and integrated with others

This approach ensures that each component is well-defined and properly integrated into the larger system.

16.2 Managing AI in Larger Codebases

16.2.1 Context Limitations

A key challenge when using AI with large codebases is providing sufficient context. Most AI assistants have limitations on how much code they can analyse at once.

Strategies for managing context limitations:

1. **Hierarchical exploration** - Start with high-level architecture and progressively drill down
2. **Component-focused prompting** - Focus inquiries on specific components rather than the entire system
3. **Interface-first approach** - Prioritise understanding component interfaces over implementation details
4. **Strategic code sampling** - Select representative portions of the codebase for AI analysis

Example prompt for hierarchical exploration:

```
I'm working on a large e-commerce platform. Let me
first describe the high-level architecture:

[High-level description of system components and
their interactions]

I'd like to focus on improving the cart service.
Here's the interface for this component:

[Cart service interface definition]

Based on this architectural context, what are the
key considerations I should keep in mind when
implementing a new feature for bundle discounts in
the cart service?
```

16.2.2 Maintaining Consistency

In larger projects, maintaining consistent patterns and approaches becomes critical. AI can help enforce these standards:

Human prompt:

```
Our team has established these coding standards
for our Java backend services:

1. Controller-Service-Repository pattern for all
API endpoints
2. Immutable data objects using Lombok
3. Detailed error handling with custom exception
classes
4. Comprehensive logging with structured log
format
5. Unit tests for all business logic

Here's an example of a controller that follows our
```

```
standards:

[Example controller code]

I need to create a new endpoint for managing
product bundles. Can you help me implement this
following our established patterns?
```

16.2.3 Modular Prompting Strategies

For complex tasks, break down your prompts into modular components:

1. **Context establishment** - Provide the necessary background information
2. **Specific task definition** - Clearly state what you're trying to accomplish
3. **Constraints and requirements** - Outline any limitations or must-have features
4. **Expected output format** - Specify how you want the response structured

Example of modular prompting:

```
CONTEXT:
I'm working on an analytics dashboard for our
e-commerce platform. The dashboard uses React on
the frontend with a Node.js/Express backend. We
fetch data from multiple microservices and
aggregate it for display.

TASK:
I need to implement a caching layer for API
responses to improve dashboard loading time.

CONSTRAINTS:
- Cache must invalidate after 5 minutes
- Cache should be per-user (different users see
different data)
- Solution must work with our existing
authentication system (JWT-based)
- Must handle error cases gracefully

REQUESTED OUTPUT:
Please provide:
1. A recommended caching approach
2. Pseudocode for the implementation
3. Any considerations for scaling this solution
```

16.3 Intentional System Design with Prompting

16.3.1 Designing for Evolution

Complex systems evolve over time. Intentional prompting at scale means designing with this evolution in mind:

Human prompt:

```
I'm designing a payment processing system that
currently needs to support credit cards and
PayPal. In the future, we anticipate adding:
- Additional payment providers
- Cryptocurrency payments
- Subscription billing
- Regional payment methods

How should I design the system to make these
future additions as straightforward as possible?
What patterns would help maintain a clean
architecture as the system grows?
```

AI response:

```
Designing a payment processing system with future
extensibility in mind requires careful
architectural planning. Here's a comprehensive
approach:

### Core Architectural Patterns

1. **Strategy Pattern** for Payment Methods
   - Create a common `PaymentProcessor` interface
   - Implement concrete strategies for each
   payment method (CreditCardProcessor,
   PayPalProcessor)
   - Use a factory to instantiate the appropriate
   processor based on payment type

2. **Adapter Pattern** for External Services
   - Create a consistent internal API for payment
   operations
   - Implement adapters for each external service
   (Stripe, PayPal, etc.)
   - This isolates external API changes from your
   core system
```

```
3. **Provider Model** for New Payment Types
   - Implement a plugin/provider registration
   system
   - New payment methods can be added as modules
   without modifying core code
   - Use dependency injection to load payment
   providers

### System Architecture Components

1. **Payment Gateway Interface**
   ```java
   public interface PaymentGateway {
       PaymentResult process(PaymentRequest
       request);
       RefundResult refund(RefundRequest request);
       PaymentStatus checkStatus(String
       paymentId);
       // Future methods can be added as needed
   }
```

2. **Abstract Payment Method**

```java
public abstract class PaymentMethod {
    protected PaymentGateway gateway;

    public abstract PaymentResponse
    authorize(Money amount, Customer customer);
    public abstract PaymentResponse
    capture(String authorizationId);
    public abstract PaymentResponse
    refund(String paymentId, Money amount);

    // Methods for subscription handling
    (future extension)
    public boolean supportsSubscriptions() {
    return false; }
    public PaymentResponse
    createSubscription(SubscriptionRequest
    request) {
        throw new
        UnsupportedOperationException();
    }
}
```
```

3. **Payment Provider Registry**

```
public class PaymentProviderRegistry {
    private Map<String, PaymentMethod>
    providers = new HashMap<>();

    public void registerProvider(String type,
    PaymentMethod provider) {
        providers.put(type, provider);
    }

    public PaymentMethod getProvider(String
    type) {
        if (!providers.containsKey(type)) {
            throw new
            UnsupportedPaymentTypeException(type);
        }
        return providers.get(type);
    }

    public List<String>
    getSupportedPaymentTypes() {
        return new
        ArrayList<>(providers.keySet());
    }
}
```

16.3.2 Design Principles to Follow

1. **Abstraction Layers**
   - Separate business logic from payment provider integration
   - Create clear boundaries between components
2. **Configuration Over Code**
   - Use configuration files for provider settings
   - Enable/disable payment methods without code changes
3. **Event-Driven Approach**
   - Implement payment events (initiated, authorized, captured, failed, refunded)
   - Allow other systems to react to payment events without tight coupling
4. **Idempotency**
   - Design all operations to be idempotent (can be safely retried)
   - Use idempotency keys for external API calls
5. **Generic Data Models**
   - Design data models that can accommodate various payment methods

- Use extensible property bags for method-specific data

16.3.3 Specific Extension Points

1. **For Cryptocurrency Support:**
   - Add a `CryptocurrencyPaymentMethod` class
   - Implement blockchain transaction monitoring
   - Consider address generation and validation
2. **For Subscription Billing:**
   - Extend the payment interface with subscription-specific methods
   - Add models for subscription plans, billing cycles, etc.
   - Implement subscription state management
3. **For Regional Payment Methods:**
   - Add metadata for region availability to payment methods
   - Implement region-specific validation and processing rules
   - Consider internationalization requirements

16.3.4 Implementation Example

Here's a skeleton implementation of the core components:

```
// The request/response models
public class PaymentRequest {
    private String paymentMethodType;
    private Money amount;
    private Customer customer;
    private Map<String, Object> methodSpecificData
    = new HashMap<>();
    // getters and setters
}

public class PaymentResponse {
    private boolean success;
    private String transactionId;
    private PaymentStatus status;
    private String errorCode;
    private String errorMessage;
    // getters and setters
}

// The payment service
public class PaymentService {
    private PaymentProviderRegistry
    providerRegistry;
```

```
    public PaymentResponse
    processPayment(PaymentRequest request) {
        PaymentMethod method =
        providerRegistry.getProvider(request.
getPaymentMethodType());
        return
        method.authorize(request.getAmount(),
        request.getCustomer());
    }

    // Additional methods for different payment
    operations
}
```

16.3.5 Testing Considerations

1. Create mock implementations of payment gateways for testing
2. Implement comprehensive integration tests for each payment provider
3. Design a test framework that can validate new payment methods

By following these patterns and principles, your payment system will be well-positioned to accommodate new payment methods and functionality with minimal changes to the core architecture.

```
### Cross-Cutting Concerns

In complex systems, certain concerns cut across
multiple components. Intentional design addresses
these systematically:

**Human prompt:**
```

In our microservices architecture, we're seeing the same patterns implemented inconsistently across services for: - Authentication and authorization - Logging and monitoring - Error handling and reporting - API request validation - Rate limiting

How can we standardize these cross-cutting concerns across our services while maintaining each team's autonomy over their service's core functionality?

```
## Documentation and Knowledge Transfer

### Intentional System Documentation

Documentation is particularly crucial for complex
```

```
systems. AI can help create and maintain effective
documentation:

**Human prompt:**
```

I've implemented a new authentication service with the following components: - OAuth2 provider integration - JWT token issuance and validation - Role-based access control - Multi-factor authentication support

I need to create comprehensive documentation for other developers who will integrate with this service. What should my documentation cover, and how should I structure it for maximum clarity?

```
**AI response:**
```

Chapter 17

Authentication Service Documentation Structure

Here's a comprehensive documentation structure for your authentication service that balances thoroughness with usability:

17.1 1. Overview

17.1.1 1.1 Purpose and Scope

- Brief description of the authentication service
- Core features (OAuth2, JWT, RBAC, MFA)
- Where it fits in the overall system architecture

17.1.2 1.2 High-Level Architecture

- Component diagram showing major parts of the auth service
- Integration points with other systems
- Data flow diagrams for key authentication paths

17.1.3 1.3. Terminology

- Clear definitions of domain-specific terms
- Explanation of OAuth2 concepts for those unfamiliar

17.2 2. Getting Started

17.2.1 2.1 Prerequisites

- Required credentials and access
- Development environment setup
- Dependencies

17.2.2 2.2 Quick Start Guide

- Basic implementation examples for common scenarios
- Sample code for authentication flows
- Minimal configuration example

17.2.3 2.3 Integration Checklist

- Step-by-step guide for service integration
- Security best practices checklist

17.3 3. Core Features

17.3.1 3.1 OAuth2 Provider Integration

- Supported OAuth providers
- Configuration parameters
- Authorization code flow implementation
- Refresh token handling
- Error scenarios and troubleshooting

17.3.2 3.2 JWT Token Management

- Token structure and payload
- Issuance process and endpoints
- Validation procedures
- Token lifecycle (expiration, refresh)
- Signature verification

17.3.3 3.3 Role-Based Access Control

- Role hierarchy and definitions
- Permission assignments
- Role checking API
- Integration with resource servers
- Custom permission logic

17.3.4 3.4 Multi-Factor Authentication

- Supported MFA methods
- Enrollment process
- Verification flow
- Fallback mechanisms
- End-user experience considerations

17.4 4. API Reference

17.4.1 4.1 Authentication Endpoints

- Complete API specification (OpenAPI/Swagger)
- Request/response formats
- Error codes and handling
- Rate limits

17.4.2 4.2 Client Libraries

- Available client libraries for different languages
- Installation instructions
- Basic usage patterns

17.4.3 4.3 Middleware Components

- Authentication middleware for different frameworks
- Configuration options
- Error handling

17.5 5. Implementation Guides

17.5.1 5.1 Backend Service Integration

- Step-by-step guide with code examples
- Validation middleware setup
- Error handling best practices

17.5.2 5.2 Frontend Application Integration

- Login implementation
- Token storage strategies
- Handling token expiration
- Logout procedures

17.5.3 5.3 Mobile Application Integration

- Native app authentication flows
- Biometric integration
- Token storage on mobile devices

17.6 6. Security Considerations

17.6.1 6.1 Security Best Practices

- HTTPS requirements
- Token storage recommendations
- CSRF protection
- XSS prevention

17.6.2 6.2 Common Vulnerabilities

- Known attack vectors
- Prevention strategies
- Security testing checklist

17.6.3 6.3 Compliance Information

- Relevant standards compliance (GDPR, SOC2, etc.)
- Audit logging capabilities

17.7 7. Operations

17.7.1 7.1 Monitoring and Alerting

- Available metrics
- Recommended alerts
- Performance benchmarks

17.7.2 7.2 Troubleshooting

- Common issues and resolutions
- Logging details
- Debugging tools

17.7.3 7.3 Disaster Recovery

- Failure scenarios
- Recovery procedures

17.8 8. Reference Implementations

17.8.1 8.1 Sample Applications

- Links to reference implementations
- Example projects demonstrating best practices

17.8.2 8.2 Common Patterns

- Authentication patterns for microservices
- Single sign-on implementation
- API gateway integration

17.9 9. FAQ and Support

17.9.1 9.1 Frequently Asked Questions

- Answers to common questions
- Clarification of complex concepts

17.9.2 9.2 Getting Help

- Support channels
- Issue reporting procedure
- SLAs and response times

17.10 Documentation Tips

1. **Use consistent terminology** throughout all documentation
2. **Include working code examples** that developers can copy and adapt
3. **Create diagrams for complex flows**, especially the OAuth2 and MFA processes
4. **Add sequence diagrams** showing the authentication process across systems
5. **Document error scenarios** just as thoroughly as success paths
6. **Version your documentation** to align with service versions
7. **Provide a searchable format** (ideally with a good search function)
8. **Include a changelog** tracking major documentation updates

This structure provides a comprehensive yet navigable documentation set that addresses the needs of different audiences - from developers getting started to those troubleshooting specific issues.

```
### AI-Enhanced Code Documentation

AI can also help create and maintain code-level
documentation:

**Human prompt:**
```

Here's a complex piece of code from our order processing system:

```
public class OrderProcessor {
    private final InventoryService
    inventoryService;
    private final PaymentService paymentService;
    private final ShippingService shippingService;
    private final NotificationService
    notificationService;
    private final OrderRepository orderRepository;

    // Constructor and other methods...

    \\@Transactional
    public OrderResult processOrder(Order order) {
        // Validate order
        if (order == null ||
        order.getItems().isEmpty()) {
            return OrderResult.failure("Order must
            contain at least one item");
        }

        // Check inventory
        List<String> unavailableItems = new
        ArrayList<>();
        for (OrderItem item : order.getItems()) {
            InventoryStatus status =

inventoryService.checkAvailability(item.getProductId(),
            item.getQuantity());
            if (status !=
            InventoryStatus.AVAILABLE) {
                unavailableItems.add(item.getProductId());
            }
        }
```

```
        if (!unavailableItems.isEmpty()) {
            return OrderResult.failure("Items not
            available: " + String.join(", ",
            unavailableItems));
        }

        // Reserve inventory
        try {
            for (OrderItem item :
            order.getItems()) {

inventoryService.reserveInventory(item.getProductId(),
                item.getQuantity(),
                order.getId());
            }
        } catch (InventoryException e) {
            return OrderResult.failure("Failed to
            reserve inventory: " +
            e.getMessage());
        }

        // Process payment
        PaymentResult paymentResult;
        try {
            paymentResult =

paymentService.processPayment(order.getPaymentDetails(),
            order.getTotalAmount());
            if (!paymentResult.isSuccessful()) {
                // Release inventory if payment
                fails
                for (OrderItem item :
                order.getItems()) {

inventoryService.releaseInventory(item.getProductId(),
                    item.getQuantity(),
                    order.getId());
                }
                return
                OrderResult.failure("Payment
                failed: " +
                paymentResult.getErrorMessage());
            }
        } catch (PaymentException e) {
            // Release inventory if payment throws
```

```
            exception
            for (OrderItem item :
            order.getItems()) {

inventoryService.releaseInventory(item.getProductId(),
                item.getQuantity(),
                order.getId());
            }
            return OrderResult.failure("Payment
            error: " + e.getMessage());
        }

        // Update order status
        order.setStatus(OrderStatus.PAID);

order.setPaymentId(paymentResult.getTransactionId());
        orderRepository.save(order);

        // Create shipment
        try {
            ShipmentResult shipmentResult =
            shippingService.createShipment(order);
            if (shipmentResult.isSuccessful()) {
                order.setStatus(OrderStatus.SHIPPED);

order.setTrackingNumber(shipmentResult.getTrackingNumber());
                orderRepository.save(order);
            } else {
                // Shipment creation failed but
                payment succeeded
                // Requires manual intervention

order.setStatus(OrderStatus.PAYMENT_RECEIVED);
                order.setNotes("Shipment creation
                failed: " +
                shipmentResult.getErrorMessage());
                orderRepository.save(order);

notificationService.notifyAdminOfShippingIssue(order);
            }
        } catch (ShippingException e) {
            // Shipment creation threw exception
            order.setStatus(OrderStatus.PAYMENT_RECEIVED);
            order.setNotes("Shipping error: " +
            e.getMessage());
```

```
            orderRepository.save(order);

notificationService.notifyAdminOfShippingIssue(order);
        }

        // Send confirmation
        notificationService.sendOrderConfirmation(order);

        return OrderResult.success(order);
    }

    // Other methods...
}
```

Can you provide comprehensive JavaDoc documentation for this class and the processOrder method, explaining its purpose, the process flow, error handling, and any considerations for extending or modifying it?

```
## Knowledge Distribution in Teams

### Collaborative Learning Patterns

Intentional prompting in team settings requires
effective knowledge sharing patterns:

**Human prompt:**
```

Our development team of 12 engineers is working on a complex system with multiple components. We're finding that knowledge is becoming siloed, with each developer understanding only their area deeply.

What collaboration patterns or team practices would help us maintain a shared understanding of the system while allowing for individual specialisation?

```
**AI response:**
```

Chapter 18

Team Knowledge Sharing Patterns for Complex Systems

To balance specialised expertise with shared understanding in your team of 12 engineers, consider implementing these complementary practices:

18.1 Regular Knowledge Exchange Practices

18.1.1 1. Rotation-Based Code Reviews

- **Approach**: Assign code reviews outside of a developer's primary area
- **Frequency**: Every developer reviews code outside their specialty at least once per sprint
- **Implementation**: Create a rotation schedule ensuring exposure to different components
- **Benefits**: Cross-pollination of ideas and gradual knowledge distribution

18.1.2 2. Architecture Decision Records (ADRs)

- **Approach**: Document significant architectural decisions with context and rationale
- **Content**: Problem statement, constraints, considered options, chosen solution, consequences
- **Storage**: Keep in version control alongside the code
- **Review**: Team-wide review of new ADRs in dedicated meetings

18.1.3 3. Component Demo Sessions

- **Approach**: Regular deep-dives into specific components
- **Format**: 30-45 minute sessions with code walkthroughs, followed by Q&A
- **Frequency**: Weekly or bi-weekly, rotating through components
- **Documentation**: Record sessions and summarise key points in a knowledge base

18.1.4 4. Pair Programming Rotations

- **Approach**: Scheduled pair programming sessions across component boundaries
- **Duration**: Half-day sessions
- **Frequency**: Bi-weekly for each developer
- **Pairing**: Deliberate pairings of specialists with non-specialists

18.2 Documentation Practices

18.2.1 5. Living Architecture Diagram

- **Approach**: Maintain an always-current system architecture visualisation
- **Ownership**: Rotating responsibility for updates
- **Reviews**: Monthly team reviews to ensure accuracy
- **Accessibility**: Prominently displayed in team spaces (physical and virtual)

18.2.2 6. Component Fact Sheets

- **Approach**: One-page summaries of each component's purpose, interfaces, and quirks
- **Content**: Core responsibilities, key interfaces, common issues, future plans
- **Updates**: Refresh quarterly or after major changes
- **Usage**: Required reading for onboarding to a new component

18.3 Team Structure Adaptations

18.3.1 7. Bounded Rotation Program

- **Approach**: Planned rotations of developers between components
- **Duration**: 4-8 weeks per rotation
- **Scope**: Limited to 1-2 developers rotating at once to maintain continuity

- **Knowledge Transfer**: Dedicated handoff sessions when starting/ending rotations

18.3.2 8. Component Ambassadors

- **Approach**: Each component has a primary owner and a secondary "ambassador"
- **Ambassador Role**: Learn enough to answer questions and represent the component
- **Rotation**: Ambassadors rotate every 3-6 months
- **Responsibilities**: Attend cross-component design meetings, review integration points

18.4 Meeting Structures

18.4.1 9. System-Wide Design Reviews

- **Approach**: Regular reviews of how components interact
- **Frequency**: Monthly
- **Format**: Each team presents recent changes and upcoming work
- **Focus**: Integration points and cross-cutting concerns

18.4.2 10. Technical Brown Bags

- **Approach**: Lunch-and-learn sessions on specific technical topics
- **Presenters**: Rotating responsibility among team members
- **Topics**: Both component-specific and general engineering practices
- **Schedule**: Weekly or bi-weekly

18.5 Tooling and Infrastructure

18.5.1 11. centralised Documentation Portal

- **Approach**: Single entry point for all system documentation
- **Content**: Component guides, architecture diagrams, ADRs, API docs
- **Maintenance**: Part of definition of done for features
- **Search**: Robust search functionality across all documentation

18.5.2 12. Cross-Component Integration Tests

- **Approach**: Shared ownership of integration test suite
- **Responsibility**: All developers contribute to integration tests
- **Reviews**: Cross-component review of test scenarios
- **Visibility**: Integration test results prominently displayed

18.6 Implementation Strategy

Rather than implementing all these patterns at once:

1. **Start with 3-4 high-impact practices** based on your team's specific needs
2. **Evaluate after 2-3 months** to assess effectiveness
3. **Add or adjust practices** based on team feedback
4. **Create explicit time allocations** for knowledge sharing activities

These practices will help create multiple knowledge bridges across component boundaries while still allowing for deep specialisation where needed.

```
### Documentation as Code

Treat documentation as a first-class development
artifact:

**Human prompt:**
```

We want to implement a "documentation as code" approach for our microservices architecture. This would mean: - Documentation lives alongside code in the repository - Documentation is reviewed and updated with code changes - Documentation is automatically generated and published

What tools, practices, and workflows would you recommend for implementing this approach effectively?

```
## Practical Exercise: Scaling an Application

Let's apply these principles to a practical
exercise scaling a simple application to a more
complex system.

### Initial Simple Application

**Human prompt:**
```

I have a simple note-taking application with these components: - A React frontend for creating and viewing notes - A Node.js/Express backend with RESTful APIs - MongoDB for data storage

Now I need to scale this to an enterprise collaboration platform with: - Team workspaces - Document sharing and editing - User permissions and roles - Activity tracking and notifications - Integration with other tools via webhooks

How should I approach scaling this application architecture? What components would I need to add or modify, and how can I ensure the

system remains maintainable as it grows?

```
## Key Takeaways

- Scaling intentional prompting to complex
projects requires architectural thinking
- Break down large systems into manageable
components with clear boundaries
- Use modular prompting strategies to help AI
understand complex contexts
- Address cross-cutting concerns systematically
across the system
- Invest in comprehensive documentation and
knowledge-sharing practices
- Treat documentation as a first-class artifact
that evolves with the code
- Use collaborative patterns to distribute
knowledge across the team
```

`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImNoYXB0ZXJzIn0= -->`

```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImNoYXB0ZXJzIiwiYm9va0
```

Chapter 19

Teaching and Learning

> Teaching someone to use AI well is teaching them to think well. The tool is secondary.

The advent of AI coding assistants has dramatically changed not just how we program, but also how we teach and learn programming. This chapter explores the application of intentional prompting principles in educational settings, offering guidance for both educators and students.

19.1 Pedagogical Applications of Intentional Prompting

19.1.1 Rethinking Programming Education

Traditional programming education has focused on teaching syntax, algorithms, and problem-solving techniques directly. With AI assistants readily available, the educational focus needs to shift:

Traditional Focus	Intentional Prompting Focus
Syntax memorization	Critical evaluation of AI-generated code
Writing code from scratch	Guiding AI toward better solutions
Debugging syntax errors	Understanding conceptual errors
Implementing standard algorithms	Adapting algorithms to specific contexts
Following prescribed solutions	Exploring multiple approaches

This shift doesn't mean abandoning the fundamentals. Rather, it means teaching them in a new context where AI collaboration is assumed.

19.1.2 The Educator's Role in the AI Era

Educators now have additional responsibilities:

1. **Teaching effective prompting** alongside programming concepts
2. **modelling critical evaluation** of AI-generated solutions
3. **Emphasising conceptual understanding** over implementation details
4. **Preparing students** for a future where AI collaboration is the norm
5. **Creating assessment methods** that work in an AI-assisted environment

19.1.3 Integrating AI into Curriculum Design

AI assistants can be integrated at different levels of the curriculum:

Level 1: AI as a Learning Resource - Students use AI to explain concepts they don't understand - AI provides additional examples and alternative explanations - Educators guide students on effective question-asking

Level 2: AI as a Programming Partner - Students learn to prompt AI for code suggestions - Assignments include evaluating and improving AI-generated code - Focus on understanding the "why" behind AI suggestions

Level 3: AI as a Feedback Mechanism - Students request AI feedback on their code - AI identifies potential improvements and best practices - Students learn to critically evaluate AI feedback

Level 4: AI as a Challenge Mechanism - Students intentionally guide AI toward errors, then identify them - Assignments include "debugging" AI-generated code - Focus on developing critical thinking skills

19.2 Designing Assignments in the AI Era

19.2.1 Beyond "AI-Proof" Assignments

Many educators initially responded to AI coding assistants by trying to create "AI-proof" assignments, ones that AI tools couldn't easily solve. This approach is ultimately unsustainable and misses the opportunity to teach intentional prompting skills.

Instead, design assignments that incorporate AI collaboration explicitly:

19.2.2 Prompting-Focused Assignments

These assignments emphasise the skill of effective prompting:

Example assignment:

```
Topic: Sorting Algorithms

Your task is to implement a merge sort algorithm
in Python. Instead of
coding it directly, you'll guide an AI assistant
through the implementation
process.

Requirements:
1. Start with a broad prompt asking the AI to
implement merge sort
2. Evaluate the initial implementation,
identifying at least three aspects to improve
3. Use targeted prompts to refine the
implementation
4. Document each prompt you used and the AI's
response
5. Submit your final code along with your
prompting journey and reflections
```

19.2.3 Step-by-Step Guided Exercise

This example from an educational setting shows how to guide students through intentional prompting:

Example: Building a Python Gradebook

```
**Task:** Create a gradebook program that:
- Uses a dictionary to store student names and
their list of scores
- Has a function to calculate average score
- Prints each student's name and average
- Handles edge cases (e.g., empty lists, invalid
inputs)
```

Step 1: Start Simple

```
"Can you help me write a Python program to store
student names and their test scores, and print
their average?"
```

The AI might provide a basic implementation:

```python
students = {
    "Alice": [85, 90, 78],
    "Bob": [70, 88, 92],
```

```
    "Charlie": [100, 100, 90]
}

for name, scores in students.items():
    avg = sum(scores) / len(scores)
    print(f"{name} has an average of {avg}")
```

Step 2: Add a Function

```
"Can you turn the average calculation into a
function?"
```

The AI improves the implementation:

```
def average(scores):
    return sum(scores) / len(scores)

for name, scores in students.items():
    print(f"{name} has an average of
    {average(scores)}")
```

Step 3: Improve Robustness

```
"What happens if a student has no scores? Can you
fix that?"
```

The AI addresses the edge case:

```
def average(scores):
    if not scores:
        return 0
    return sum(scores) / len(scores)
```

This guided approach teaches students to: - Start with a simple prompt - Identify limitations in the initial response - Use targeted follow-up prompts to improve specific aspects - Think critically about edge cases - Guide the AI through a step-by-step improvement process

19.2.4 Evaluation and Improvement Assignments

These assignments focus on critical evaluation of AI-generated code:

Example assignment:

```
Topic: Error Handling

You are provided with an AI-generated function
that processes user
input for a banking application. The code is
```

```
functional but has
issues with error handling, security, and
readability.

Requirements:
1. Evaluate the provided code, identifying all
potential issues
2. Create a prioritised list of improvements
needed
3. Implement the improvements
4. Document your reasoning for each change
5. Reflect on what the AI should have done
differently
```

19.2.5 Comparative Analysis Assignments

These assignments emphasise exploring multiple approaches:

Example assignment:

```
Topic: Data Structures

Use an AI assistant to implement three different
data structures for
storing and retrieving student records: a hash
table, a binary search
tree, and a balanced tree.

Requirements:
1. Guide the AI to implement each data structure
2. Create a test suite to measure performance
characteristics
3. analyse the strengths and weaknesses of each
approach
4. Select the most appropriate structure for the
given requirements
5. Justify your selection with empirical evidence
```

19.2.6 Process Documentation Assignments

These assignments focus on the development process rather than just the final code:

Example assignment:

```
Topic: API Development
```

```
Create a RESTful API for a library management
system, documenting your
entire development process.

Requirements:
1. Document your initial planning (endpoints, data
models)
2. Record all prompts used with AI assistants
3. Document key decisions and pivots during
development
4. Include a reflective analysis of where AI was
most/least helpful
5. Identify areas where your human expertise was
essential
```

19.3 Assessing Understanding vs. Output

19.3.1 Moving Beyond Code Assessment

In the AI era, the code students produce is no longer a reliable indicator of their understanding. Assessment strategies must evolve:

19.3.2 Process-Based Assessment

Evaluate the student's development process rather than just the final code:

- **Documentation of prompting strategy**
- **Reflection on AI collaboration**
- **Analysis of alternative approaches considered**
- **Justification of design decisions**

19.3.3 Concept Demonstration

Have students demonstrate conceptual understanding separate from code production:

- **Verbal explanations of how their code works**
- **Identification of potential edge cases**
- **Analysis of time and space complexity**
- **Prediction of how code would behave under different inputs**

19.3.4 Code Modification Challenges

Test understanding through targeted modification tasks:

- **Adding a new feature to existing code**

- **optimising for a different constraint**
- **Adapting the solution to a different context**
- **Fixing intentionally introduced bugs**

19.3.5 Prompt Engineering Assessment

Directly assess students' ability to effectively prompt AI tools:

- **Provide a difficult problem and evaluate prompting strategy**
- **Grade the quality and specificity of prompts**
- **Assess ability to refine prompts based on initial responses**
- **Evaluate critical thinking about AI-generated solutions**

19.4 Building a Learning Community

19.4.1 Collaborative Learning in the AI Era

AI assistants change the dynamics of collaborative learning:

Challenges: - Students may rely on AI rather than peers for help - Individual work becomes harder to distinguish - Knowledge gaps can be masked by AI assistance

Opportunities: - Groups can focus on higher-level design discussions - AI can serve as a common "baseline" for group projects - Students can learn from each other's prompting strategies

19.4.2 Peer Learning Strategies

Encourage intentional peer collaboration:

1. **Prompt-sharing sessions** where students compare effective prompts
2. **Code review workshops** focused on evaluating AI-generated code
3. **Paired prompting** where students collaborate on guiding AI
4. **Prompting competitions** to solve challenges with the most effective prompts

19.4.3 Creating a Supportive Environment

Foster an environment that supports learning with AI:

1. **Explicitly acknowledge** AI use as expected and valuable
2. **Establish clear guidelines** for appropriate AI collaboration
3. **Share success stories** of effective AI use
4. **Normalise discussions** about AI limitations and frustrations
5. **recognise prompting expertise** as a valuable skill

19.5 Case Studies in Educational Settings

19.5.1 Case Study 1: Introductory Programming Course

Context: An introductory Python programming course at a university, with students who have no prior programming experience.

Approach: The instructor redesigned the course around intentional prompting principles:

1. **Week 1-3: Foundations and AI Introduction**
    - Basic Python syntax and concepts taught traditionally
    - Introduction to AI assistants in week 3
    - Assignment: Compare hand-written and AI-generated solutions
2. **Week 4-8: Guided AI Collaboration**
    - Structured prompting templates provided
    - Focus on understanding and modifying AI-generated code
    - Assignments include documented prompting journey
3. **Week 9-15: Independent Problem-Solving**
    - Complex problems requiring multiple prompting iterations
    - Focus on proper problem decomposition
    - Final project includes reflection on AI collaboration process

Results: - Students showed stronger conceptual understanding compared to previous cohorts - More students attempted advanced topics and extensions - Course completion rate increased by 15% - Students reported higher confidence in their programming abilities

Key Takeaway: Integrating AI assistants from the beginning, with explicit instruction on effective usage, led to better learning outcomes than either banning AI or allowing it without guidance.

19.5.2 Case Study 2: Advanced Software Engineering Course

Context: A senior-level software engineering course focused on designing and implementing a complex system over a semester.

Approach: The instructor implemented a "documentation-first" intentional prompting methodology:

1. **Project Planning Phase**
    - Students created detailed specifications before any coding
    - AI was used to evaluate specifications for completeness
    - Students documented initial architectural decisions
2. **Implementation Phase**
    - Students used the six-step methodology for each component

- Weekly submissions included prompting strategies used
- Peer reviews focused on AI collaboration effectiveness

3. **Integration and Testing Phase**
   - AI used to generate comprehensive test cases
   - Students documented where AI testing missed edge cases
   - Final submission included reflective analysis

Results: - Projects showed more consistent architecture and documentation - Student teams reported fewer integration issues - Code quality metrics improved over previous cohorts - Students demonstrated better ability to explain architectural decisions

Key Takeaway: The intentional, documentation-first approach helped students maintain control over increasingly complex projects, even with significant AI assistance.

19.6 Practical Guide for Educators

19.6.1 Introducing Intentional Prompting in Your Curriculum

Step 1: Self-Education - Familiarize yourself with AI coding assistants - Practice the six-step methodology on your own projects - Develop your own prompting skills

Step 2: Curriculum Analysis - Identify which learning objectives remain essential - Determine where AI can enhance rather than replace learning - Design new objectives around intentional prompting

Step 3: Assignment Redesign - Convert existing assignments to incorporate AI collaboration - Create new assignments focused on prompting skills - Develop clear guidelines for acceptable AI use

Step 4: Assessment Adaptation - Design assessments that evaluate process, not just output - Create rubrics that include prompting quality - Implement verification mechanisms (explanations, modifications)

Step 5: Student Onboarding - Explicitly teach effective prompting techniques - Provide examples of good and poor AI collaboration - Set clear expectations for documentation

19.6.2 Sample Lesson Plan: Introduction to Intentional Prompting

Objective: Introduce students to intentional prompting for programming tasks

Materials: - Access to an AI coding assistant - Sample programming problem - Prompting template handout

Lesson Flow:

1. **Introduction (15 min)**
   - Discuss the limitations of traditional "solution-oriented" prompting
   - Introduce the concept of intentional prompting
   - Share examples of effective vs. ineffective prompts
2. **Demonstration (20 min)**
   - Instructor demonstrates solving a problem using intentional prompting
   - Highlight the iterative refinement process
   - Show how to evaluate and improve AI-generated code
3. **Guided Practice (30 min)**
   - Students work in pairs on a simple programming task
   - Using provided prompting templates
   - Instructor circulates to provide feedback
4. **Reflection and Discussion (15 min)**
   - Students share what worked and what didn't
   - Discuss patterns in effective prompts
   - Address common misconceptions
5. **Independent Practice (Homework)**
   - Students solve a new problem using intentional prompting
   - Document their prompting journey
   - Reflect on what they learned about effective prompting

19.7 Practical Guide for Students

19.7.1 maximising Learning with AI Assistance

Develop an Intentional Learning Mindset - Focus on understanding concepts, not just completing assignments - Use AI to explore alternative approaches, not just get answers - Reflect on what you're learning through AI interactions

Document Your Learning Journey - Keep a prompting journal recording effective prompts - Note concepts that were unclear and how AI helped clarify them - Track your progress in prompting effectiveness

Balance AI Assistance with Independent Work - Try solving problems independently before consulting AI - Use AI to verify your understanding, not replace it - Deliberately practice skills that AI can't provide (debugging, testing)

Leverage AI for Learning Acceleration - Use AI to explain concepts

in multiple ways - Ask AI to generate practice problems on topics you're struggling with - Have AI explain the "why" behind its recommendations

19.7.2 Self-Assessment Questions

Students can use these questions to gauge their intentional prompting skills:

1. Can I explain the code I submitted, including its logic and potential edge cases?
2. Do I understand why the AI suggested certain approaches over others?
3. Can I modify the solution to work under different constraints?
4. Am I able to identify limitations or potential improvements in AI-generated code?
5. Can I effectively guide AI through complex problems using systematic prompting?

19.8 Key Takeaways

- The educational focus should shift from syntax to intentional collaboration
- Assignments should incorporate AI explicitly rather than trying to be "AI-proof"
- Assessment should focus on process, understanding, and prompting skills
- Both educators and students need new strategies for effective learning with AI
- Intentional prompting principles provide a framework for teaching programming in the AI era
- Building learning communities around AI collaboration enhances educational outcomes

19.9 Moving Forward

In the final chapter, we'll explore the future of intentional prompting as AI technology continues to evolve, examining emerging trends and preparing for the next evolution of human-AI programming collaboration.

Chapter 20

Future Directions

> The tools will change. The discipline of thinking clearly will not.

As we reach the conclusion of this book, it's important to look ahead at how intentional prompting might evolve as AI capabilities continue to advance. This chapter explores emerging trends in AI-assisted development, potential shifts in programming paradigms, and strategies for maintaining the human element in an increasingly AI-powered development landscape.

20.1 Emerging AI Capabilities

20.1.1 Beyond Text-Based Interfaces

Current AI coding assistants primarily interact through text-based interfaces, but this is likely to change:

20.1.1.1 Multimodal Interactions

Future AI assistants will likely incorporate:

- **Visual programming interfaces** that blend graphical and text-based interaction
- **Voice-based coding** where spoken instructions translate to code
- **Diagram-to-code conversion** where architecture diagrams become implementation scaffolding
- **Code-to-visualisation generation** where code is automatically visualised for better understanding

These multimodal capabilities will require adaptations to intentional prompting practices, with greater emphasis on clear specification and

visualisation of intent.

20.1.1.2 Contextual Awareness

AI assistants are becoming increasingly aware of their operational context:

- **Project-wide understanding** beyond individual files
- **Development history awareness** including past decisions and their rationale
- **Developer preference modelling** that adapts to individual coding styles
- **Team convention adherence** that ensures consistency across contributors

For intentional prompting, this means prompting strategies will need to focus more on high-level goals and constraints, while the AI handles more of the implementation details.

20.1.2 Increasing Autonomy and Agentic Programming

AI systems are gradually moving toward greater autonomy in coding tasks, leading to what's known as "agentic programming," where AI agents can autonomously perform complex development tasks with minimal human intervention:

20.1.2.1 Autonomous Coding Capabilities

- **Self-directed exploration** of solution spaces
- **Automated testing and verification** of generated code
- **Self-debugging and refinement** of initial solutions
- **Proactive suggestion** of improvements and optimisations

This increased autonomy will shift intentional prompting toward a supervisory role, where human developers focus on problem definition, architectural decisions, and quality assurance rather than implementation details.

It is worth stating plainly: better models make the practices in this book more important, not less. When AI-generated code was obviously rough, developers naturally reviewed it. As models produce code that reads like something a competent developer wrote, the temptation to accept it without understanding increases, and so does the cost of the bugs you miss. The more convincing the output, the more critical it is that you can verify it, explain it, and test it. Intentional prompting is not a workaround for weak models. It is a discipline that becomes more valuable as models improve.

20.1.2.2 AI Pairing and Collaboration

Future development environments may feature:

- **Multiple specialised AIs** with different expertise areas
- **AI-to-AI collaboration** where different models handle different aspects of development
- **Continuous AI presence** during the entire development process
- **Persistent learning** from team interactions and feedback

Intentional prompting in this environment will involve orchestrating these AI collaborators effectively, ensuring they work toward cohesive solutions.

20.1.2.3 The Essential Role of Human Oversight in Agentic Systems

As AI development agents become more sophisticated, research has identified critical intervention points where human oversight remains essential:

Requirements Phase - *AI Role*: Convert user stories to technical specifications - *Human Role*: Validate business alignment and feasibility of proposed approaches

Design Phase - *AI Role*: Generate architecture diagrams from specifications - *Human Role*: Evaluate maintainability and technical debt implications

Implementation - *AI Role*: Auto-complete functions with context awareness - *Human Role*: Ensure compliance with security protocols and organisational standards

Deployment - *AI Role*: Auto-scale and configure infrastructure - *Human Role*: Final approval for production releases

Research shows that even the most advanced AI systems face fundamental limitations. They struggle with precision despite excellence in generalisation, encounter difficulties with large-scale tasks despite competence with smaller ones, and demonstrate incomplete understanding of physical and business realities. These limitations highlight why human oversight in agentic systems is not merely beneficial but essential.

Effective human oversight includes:

1. **Establishing clear protocols and roles** for when and how human intervention should occur
2. **Continuous training and development** for oversight personnel
3. **Fostering collaboration** between AI experts and domain specialists
4. **Implementing robust monitoring tools** and evaluation frameworks

The future of intentional prompting will likely involve more sophisticated techniques for providing this essential oversight, ensuring that increasingly autonomous AI systems remain aligned with human goals, values, and quality standards.

20.2 Evolving Programming Paradigms

20.2.1 From Code Writing to Code Curation

The fundamental activity of programming is shifting from writing code to curating it:

20.2.1.1 Specification-First Development

- **Detailed specifications** become the primary artifact
- **Automated implementation** derived from specifications
- **Human review and refinement** of generated code
- **Iterative specification improvement** based on implementation feedback

Intentional prompting in this paradigm emphasises clear articulation of requirements and constraints, with less focus on implementation syntax.

20.2.1.2 Quality-Oriented Development

- **Focus on correctness criteria** rather than implementation
- **Property-based specifications** that define behaviour boundaries
- **Automated verification against properties** rather than manual testing
- **Formal methods** becoming more accessible through AI interpretation

This shift requires developing skills in specifying quality attributes and verification criteria rather than algorithmic details.

20.2.2 New Programming Languages and Tools

AI capabilities are influencing the design of programming languages and development tools:

20.2.2.1 AI-Native Languages

- **Natural language programming** becoming more feasible
- **Intent-based syntax** that focuses on what, not how
- **Adaptable abstraction levels** that adjust to developer expertise
- **Bidirectional translation** between natural language and formal code

Intentional prompting practices will need to evolve to leverage these new language paradigms effectively.

20.2.2.2 Integrated AI Development Environments

- **Continuous AI collaboration** throughout the development workflow
- **Predictive development** that anticipates next steps
- **Automated refactoring and optimisation** based on semantic understanding
- **Intelligent knowledge integration** from documentation and community resources

These environments will require new skills in guiding AI assistance effectively throughout the development lifecycle.

20.3 Preparing for Future Changes

20.3.1 Skill Evolution for Developers

As AI capabilities advance, the most valuable developer skills will shift:

20.3.1.1 Emerging Critical Skills

1. **System design and architecture** - Defining cohesive, maintainable systems
2. **Problem formulation** - Precisely specifying what needs to be solved
3. **Solution evaluation** - Critically assessing generated solutions
4. **Adaptation and integration** - Combining AI-generated components effectively
5. **Domain modelling** - Translating real-world complexity into computational terms

Intentional prompting principles provide a foundation for developing these skills, with increasing emphasis on architecture and evaluation.

20.3.1.2 Continuous Learning Strategies

To remain effective as AI capabilities evolve:

1. **Focus on fundamentals** - Core concepts remain relevant even as implementation details change
2. **Practice intentional prompting** - Develop precise communication with AI systems
3. **Engage with new paradigms** - Experiment with emerging languages and methodologies

4. **Build cross-disciplinary knowledge** - Combine programming with domain expertise
5. **Cultivate creative problem-solving** - Develop skills that complement rather than compete with AI

20.3.2 Organisational Adaptation

organisations will need to adapt their development processes:

20.3.2.1 Evolving Team Structures

- **AI specialists** who optimise AI collaboration across teams
- **Prompt engineers** who develop effective prompting strategies
- **Integration architects** who ensure cohesive solutions
- **Quality assurance evolution** toward specification verification

20.3.2.2 Workflow Transformation

- **Specification-centric processes** replacing code-centric workflows
- **Continuous AI collaboration** throughout the development lifecycle
- **Balanced human-AI responsibility** with clear accountability
- **Knowledge management** becoming increasingly critical

Intentional prompting principles can guide these organisational transformations, ensuring that human expertise remains central despite increasing automation.

20.4 Maintaining the Human Element

20.4.1 Ethical Considerations

As AI takes on more programming responsibilities, ethical considerations become increasingly important:

20.4.1.1 Responsibility and Accountability

- **Clear attribution** of AI vs. human contributions
- **Transparency in AI assistance** for regulatory compliance
- **Ethical review processes** for AI-generated solutions
- **Professional responsibility** for AI-assisted outcomes

20.4.1.2 Bias and Fairness

- **Awareness of AI biases** in generated code
- **Proactive fairness assessment** of solutions

- **Diverse human oversight** to detect potential issues
- **Inclusive design principles** throughout development

Intentional prompting practices should incorporate explicit ethical review steps to address these concerns.

20.4.2 Human Comparative Advantages

Despite advancing AI capabilities, humans maintain several critical advantages:

20.4.2.1 Enduring Human Strengths

1. **Contextual understanding** - Grasping the broader purpose and impact of software
2. **Ethical judgment** - Evaluating moral implications of technical decisions
3. **Creative problem-solving** - Finding novel approaches to unprecedented challenges
4. **Empathetic design** - Deeply understanding user needs and experiences
5. **Strategic thinking** - Aligning technical decisions with long-term goals

Intentional prompting should increasingly focus on leveraging these uniquely human capabilities while delegating algorithmically definable tasks to AI.

20.4.2.2 Meaningful Human Contribution

To ensure programming remains a fulfilling human activity:

1. **Focus on high-impact decisions** where human judgment adds value
2. **Cultivate creativity in problem-solving** approaches
3. **Engage with ethical dimensions** of technical decisions
4. **Prioritise human-centred design** considerations
5. **Develop cross-disciplinary expertise** combining programming with domain knowledge

20.5 Vision for Intentional Prompting's Evolution

20.5.1 Short-Term Evolution (1-3 Years)

In the near future, intentional prompting will likely adapt to:

- More comprehensive code generation capabilities
- Better context awareness across codebases
- Improved natural language understanding
- Initial multimodal interaction capabilities

Key adaptations will include:

1. More refined prompting techniques for whole-component generation
2. Greater emphasis on specification and architectural design
3. Increased focus on evaluation and integration skills
4. Enhanced collaborative workflows combining human and AI strengths

20.5.2 Medium-Term Evolution (3-7 Years)

Looking further ahead, we may see:

- Natural language becoming a primary programming interface
- AI agents capable of autonomous implementation of components
- Sophisticated AI collaboration networks in development
- Formal verification becoming mainstream through AI assistance

Intentional prompting will likely evolve toward:

1. Architecture and system design as primary human activities
2. Specification languages replacing traditional programming languages
3. Orchestration of multiple specialised AI collaborators
4. Quality assurance through formal specification rather than testing

20.5.3 Long-Term Vision (7+ Years)

Though speculative, longer-term developments might include:

- Programming primarily through intent specification
- AI systems capable of end-to-end application development
- Symbiotic human-AI development partnerships
- New programming paradigms built specifically for human-AI collaboration

The core principles of intentional prompting (maintaining human understanding and control while leveraging AI capabilities effectively) will remain relevant even as the specific practices evolve.

20.6 A Personal Development Roadmap

20.6.1 Building Your Intentional Prompting Skills

To prepare for the future of programming, consider this development roadmap:

20.6.1.1 Foundational Skills

1. **Master the six-step methodology** for intentional prompting
2. **Develop effective prompting techniques** for different programming tasks
3. **Practice critical evaluation** of AI-generated code
4. **Build architectural thinking skills** for component and system design
5. **Cultivate test-driven thinking** to ensure solution quality

20.6.1.2 Advancing Your Practice

1. **Experiment with emerging AI coding assistants** to understand their capabilities
2. **Develop domain-specific prompting strategies** for your areas of expertise
3. **Create personal libraries of effective prompts** for common tasks
4. **Incorporate intentional documentation** practices in your workflow
5. **Contribute to open standards** for human-AI development collaboration

20.6.1.3 Leading the Evolution

1. **Explore new programming paradigms** that leverage AI capabilities
2. **Develop team practices** for effective AI collaboration
3. **Establish ethical guidelines** for responsible AI use in development
4. **Share your experiences** through writing, speaking, or teaching
5. **Participate in shaping** the future of human-AI programming

20.6.2 Continuous Adaptation

The most important skill for the future will be adaptability. By grounding yourself in intentional prompting principles while remaining open to evolving techniques, you'll be well-positioned to thrive in the changing landscape of software development.

20.7 Key Takeaways

- AI capabilities will continue to evolve, with increased contextual understanding, multimodal interactions, and autonomy
- Programming paradigms are shifting from code writing to code curation and verification

- Developer skills are evolving toward architecture, specification, and solution evaluation
- organisations need to adapt team structures and workflows for effective AI collaboration
- Maintaining the human element requires focus on our unique capabilities and ethical considerations
- Intentional prompting principles provide a foundation for navigating these changes effectively

20.8 Intentional Prompting Beyond Programming

While this book has focused primarily on applying intentional prompting principles to programming tasks, the core concepts of intentional prompting have far broader applications across various domains and activities.

20.8.1 Universal Principles of AI Collaboration

The fundamental principles of intentional prompting that we've explored (maintaining human agency, focusing on understanding, and prioritising process over output) apply universally to any human-AI collaboration:

20.8.1.1 In Creative Fields

- **Writing**: Using AI to brainstorm, refine, and edit written content while maintaining a unique voice
- **Design**: Directing AI to explore visual concepts while preserving artistic intent and coherence
- **Music**: Collaborating with AI on composition while maintaining musical identity and expression
- **Film and Animation**: Using AI for technical aspects while humans direct narrative and emotional elements

20.8.1.2 In Knowledge Work

- **Research**: Employing AI to analyse data and literature while humans guide inquiry and interpret significance
- **Law**: Using AI for precedent research and document preparation while humans handle judgment and argumentation
- **Medicine**: Applying AI for analysis and pattern recognition while physicians maintain diagnostic responsibility
- **Education**: utilising AI to customise learning materials while educators focus on motivation and conceptual guidance

20.8.1.3 In Business and Management

- **Strategic Planning**: Using AI to analyse trends and options while humans make value-based decisions
- **Project Management**: Employing AI for scheduling and resource allocation while humans handle stakeholder relationships
- **Marketing**: utilising AI for data analysis and content generation while humans direct brand voice and strategy
- **Customer Service**: Using AI for routine interactions while humans handle complex or sensitive situations

20.8.2 The Five Principles in Any Domain

The five core principles of intentional AI collaboration transcend specific methodologies and can be applied in any context:

1. **Human-led workflows**: Regardless of domain, humans should establish goals, define parameters, and make final decisions
2. **Critical evaluation**: All AI outputs should be thoughtfully assessed based on domain expertise and context
3. **Iterative collaboration**: The process of refinement through feedback applies across all creative and analytical activities
4. **Domain-appropriate trust**: Every field has aspects where AI excels and where human judgment is essential
5. **Skill complementarity**: The most effective collaborations leverage both AI and human strengths appropriately

20.8.3 The Future of Human-AI Partnership

As AI capabilities continue to evolve, these principles will only become more important. Future systems may offer:

- **More seamless multimodal interaction** across text, voice, images, and other media
- **Greater domain-specific expertise** in specialised fields
- **Enhanced contextual awareness** of projects, goals, and constraints
- **Increased autonomy** for well-defined tasks

However, the need for intentional human direction will remain essential. The most successful individuals and organisations will be those who develop effective practices for intentional collaboration with increasingly capable AI systems while maintaining human purpose, creativity, and ethical judgment.

20.9 Conclusion

As we conclude this exploration of intentional prompting, it's clear that the fundamental relationship between human developers and AI assistants will continue to evolve. What won't change is the need for a thoughtful, structured approach to leveraging AI capabilities while maintaining human understanding and control.

The six-step methodology and intentional prompting techniques presented in this book provide a foundation that can adapt to emerging capabilities and paradigms. By focusing on process over output, understanding over implementation, and human judgment over automation, intentional prompting offers a path forward that embraces AI assistance while preserving the essence of what makes programming a uniquely human creative endeavour.

Importantly, the methodology addresses many common objections to using LLMs in programming. For those concerned about the ambiguity of natural language, intentional prompting provides structured frameworks and progressive disambiguation techniques that bridge natural language and formal code. For those worried about the non-deterministic nature of LLMs, the methodology establishes clear evaluation criteria and testing practices that ensure reliability despite variability in outputs. And for those sceptical of AI's impact on learning and skill development, intentional prompting's focus on understanding over copying ensures that developers continue to build expertise rather than becoming dependent on AI.

The power of intentional prompting comes from recognising that LLMs have inherited human problem-solving patterns through their training data. Our methodology doesn't fight against this trait but leverages it by providing procedural frameworks that activate latent knowledge in both the human and the AI. This collaborative approach represents a middle ground between uncritical acceptance and outright rejection of AI assistance: a balanced path that maximises benefits while minimising risks.

As we move toward increasingly autonomous agentic programming systems, intentional prompting becomes not less but more essential. These systems excel at approximation and pattern matching but struggle with precision. They demonstrate competence with small-scale tasks while faltering with larger ones. They reveal an incomplete understanding of both physical and business realities. All these limitations highlight why human oversight through structured methodologies remains necessary.

The six-step methodology provides a framework not just for individual programmers but for organisations implementing AI oversight. It offers clear intervention points where human judgment and domain knowledge can guide AI agents toward optimal solutions. By maintaining this

human-in-the-loop approach, organisations can harness the efficiency of AI automation while mitigating its risks through informed, intentional human direction.

While we've focused primarily on programming, the principles of intentional collaboration with AI will extend across all domains of human creativity and problem-solving. By developing these skills now, you're preparing not just for current AI tools, but for the next generation of human-AI partnership in programming and beyond.

Acknowledgments

This book began as a set of Jupyter notebooks for teaching computational thinking. The students who worked through those notebooks — sometimes struggling, sometimes surprising me with approaches I had not considered — shaped the methodology that became intentional prompting. Their willingness to experiment with AI tools when the ground rules were still being written made this book possible.

Colleagues who teach programming in various contexts provided feedback that kept the ideas practical. The question "would this actually work in a first-year unit?" came up often enough to become a design principle.

The methodology in this book draws on *Conversation, Not Delegation*, and the two were developed in parallel. Ideas that emerged in the programming context were generalised for the companion book; principles from the companion book were tested against code. Each made the other sharper.

The open source community behind Python, Quarto, GitHub, and the broader ecosystem of developer tools made both the writing and the publishing possible. The entire toolchain — from drafting to building to hosting — is free and open, and that matters.

AI tools were used throughout the writing process. Claude (Anthropic) served as a conversation partner for drafting, iterating, and refining both text and code examples. The process was exactly what the book advocates: intentional, directed, and critically evaluated. The author made the decisions. The AI made the work faster. Every sentence and every line of code reflects the author's judgement.

About the Author

Michael Borck is a software developer and educator passionate about the intersection of human expertise and artificial intelligence. He developed the Intentional Prompting methodology to help programmers maintain agency and deepen their understanding while leveraging AI tools effectively.

Michael believes that the future of programming lies not in delegating to AI, but in conversing with it—treating AI as a collaborative partner that enhances human capability rather than replacing human understanding.

When not writing about AI collaboration, Michael works on practical applications of these principles across software development, education, and creative projects. He creates educational software and resources, and explores the 80/20 principle in learning and productivity.

Connect

- michaelborck.dev (https://michaelborck.dev) — Professional work and projects
- michaelborck.education (https://michaelborck.education) — Educational software and resources
- 8020workshop.com (https://8020workshop.com) — Passion projects and workshops

- LinkedIn (https://linkedin.com/in/michaelborck)

Other Books in This Series

Foundational Methodology:

- *Conversation, Not Delegation: Your Expertise + AI's Breadth = Amplified Thinking*

Python Track:

- *Think Python, Direct AI: Computational Thinking for Beginners*
- *Code Python, Consult AI: Python Fundamentals for the AI Era*
- *Ship Python, Orchestrate AI: Professional Python in the AI Era*

Web Track:

- *Build Web, Guide AI: Business Web Development with AI*

For Educators:

- *Partner, Don't Police: AI in the Business Classroom*

Appendix A

Common Pitfalls and Solutions

When working with AI coding assistants, certain patterns of challenges tend to emerge. This appendix catalogues common pitfalls in the intentional prompting process and provides practical strategies for avoiding or addressing them.

A.1 Over-Reliance on AI

A.1.1 Symptoms

- Accepting AI-generated code without critical evaluation
- Inability to explain how your code works
- Discomfort when working without AI assistance
- Decreasing ability to solve problems independently

A.1.2 Root Causes

- Convenience of immediate solutions
- Pressure to deliver quickly
- Gradual atrophy of independent problem-solving skills
- Incomplete understanding of the problem domain

A.1.3 Solutions

A.1.3.1 Intentional Skill Building

- Schedule regular "AI-free" coding sessions to maintain core skills

- Implement a personal rule to solve problems manually first, then verify with AI
- Create deliberate learning challenges in unfamiliar areas

A.1.3.2 Critical Evaluation Process

- Establish a personal checklist for evaluating AI-generated code
- Always trace through at least one test case manually
- Ask "why" questions about design decisions in AI-generated code
- Look for potential edge cases that might not be handled

A.1.3.3 Balance Strategies

- Use the six-step methodology to ensure you understand the problem before seeking AI solutions
- Distinguish between areas where you need to build expertise (limit AI use) and areas where you can leverage AI more heavily
- Set clear boundaries for when and how you use AI assistance

A.2 Unclear Prompt Formulation

A.2.1 Symptoms

- Receiving irrelevant or incorrect solutions
- Needing many iterations to get useful responses
- Finding yourself frustrated with AI "not understanding" what you want
- Receiving overly generic or simplistic answers

A.2.2 Root Causes

- Ambiguous problem descriptions
- Missing context or requirements
- Unexpressed assumptions
- Imprecise terminology

A.2.3 Solutions

A.2.3.1 Structured Prompting Framework

- Use the templates from Appendix A as starting points
- Break complex requirements into clear, specific points
- Include explicit constraints and edge cases
- Specify the level of detail needed in the response

A.2.3.2 Context Enhancement

- Provide relevant background information
- Include code snippets that show the context
- Specify the programming language and framework
- Describe the broader system architecture when relevant

A.2.3.3 Iterative Refinement

- Start with a basic prompt and refine based on initial responses
- Use phrases like "Let me clarify…" to correct misunderstandings
- Build upon partial successes rather than completely changing approach
- Maintain a library of effective prompts for common tasks

A.3 Accepting Incorrect Solutions

A.3.1 Symptoms

- Discovering bugs after implementation that were present in the AI suggestion
- Receiving user reports of edge case failures
- Finding that AI-generated solutions don't integrate well with existing code
- Code reviews identifying fundamental flaws in approach

A.3.2 Root Causes

- Insufficient validation of proposed solutions
- Misplaced trust in AI capabilities
- Pressure to deliver quickly
- Gaps in your own understanding of requirements

A.3.3 Solutions

A.3.3.1 Systematic Verification

- Always test AI-generated code with multiple inputs, especially edge cases
- Review security implications, particularly for input handling and data access
- Check for performance issues with realistic data volumes
- Verify integration points with existing systems

A.3.3.2 Understanding Before Implementation

- Request explanations of unfamiliar patterns in generated code

- Step through complex algorithms manually with example data
- Compare against alternative approaches
- Ensure you can explain every part of the solution yourself

A.3.3.3 Incremental Adoption

- Integrate AI-generated code in small, testable chunks
- Start with non-critical components to build confidence
- Implement monitoring for new components to catch issues early
- Create comprehensive tests before implementing AI suggestions

A.4 Losing Track of the Big Picture

A.4.1 Symptoms

- Solutions that solve immediate problems but create architectural issues
- Inconsistent patterns across different parts of the codebase
- Difficulty explaining how components fit together
- Growing technical debt from expedient but uncoordinated decisions

A.4.2 Root Causes

- Focus on tactical code generation rather than strategic design
- AI's limited context window and understanding of full system
- Solving problems in isolation without system-wide perspective
- Missing architectural guidelines for AI collaboration

A.4.3 Solutions

A.4.3.1 Architectural Discipline

- Establish clear architectural principles to guide AI prompting
- Regularly review how individual components fit into the overall system
- Maintain up-to-date architecture diagrams to reference during development
- Use the six-step methodology to ensure each solution aligns with system goals

A.4.3.2 Documentation-First Approach

- Document design decisions before implementing them
- Create clear interface specifications before coding
- Maintain a "source of truth" for architectural patterns
- Implement an architecture decision record (ADR) process

A.4.3.3 System-Level Review

- Periodically step back to assess the entire system
- Schedule "architectural reviews" to identify drift from intended design
- Use visualisation tools to understand system evolution
- Consider how individual changes affect system properties (performance, security, maintainability)

A.5 Ethical and Legal Concerns

A.5.1 Symptoms

- Uncertainty about code ownership and attribution
- Potential licence violations from AI-suggested code
- Reuse of code with unclear provenance
- Privacy or security implications in generated solutions

A.5.2 Root Causes

- Unclear understanding of AI training data sources
- Limited transparency in how AI generates responses
- Evolving legal landscape around AI-generated content
- Lack of established ethics frameworks for AI coding assistants

A.5.3 Solutions

A.5.3.1 Responsible AI Usage

- Understand how your AI assistant was trained and what data it uses
- Review AI-generated code for potential intellectual property issues
- Follow your organisation's guidelines for AI use
- Consider the ethical implications of automated decision-making in your code

A.5.3.2 Documentation and Transparency

- Document when and how AI assistance was used
- Maintain clear attribution for code sources
- Keep records of significant prompts used in development
- Be transparent with stakeholders about AI's role in development

A.5.3.3 Verification Processes

- Implement review processes specifically for AI-generated code

- Check for known security vulnerabilities in suggested solutions
- Verify that solutions meet regulatory requirements
- Consider third-party audits for critical systems

A.6 Prompt Pattern Case Studies

A.6.1 Case Study 1: Debugging a Complex Algorithm

A.6.1.1 Problematic Approach

Initial prompt:

```
Fix this sorting algorithm, it's not working
correctly.

[code snippet]
```

Issues: - Vague problem description - No information about how it's failing - No context about requirements or constraints

A.6.1.2 Improved Approach

Better prompt:

```
I'm debugging this merge sort implementation that
fails when the input contains duplicate values.

[code snippet]

Specific issues:
1. When input contains duplicates (e.g.,
[3,1,3,2]), it produces [1,2,3] instead of
[1,2,3,3]
2. It works correctly for inputs without
duplicates

I suspect the issue is in the merging step, but
I'm not sure exactly what's wrong.
Can you help identify the bug and explain how to
fix it?
```

Improvements: - Specific algorithm identified (merge sort) - Clear description of the failure case - Example input and expected output provided - Indication of where the bug might be located

A.6.2 Case Study 2: Implementing a New Feature

A.6.2.1 Problematic Approach

Initial prompt:

```
Write a user authentication system for my web app.
```

Issues: - Extremely broad scope - No information about the technology stack - No specific requirements or constraints - No context about the existing application

A.6.2.2 Improved Approach

Better prompt:

```
I need to implement user authentication for a
React/Node.js web application.

Requirements:
- JWT-based authentication
- Support for email/password login
- Password reset functionality
- Role-based authorization (admin and regular
users)
- Integration with existing MongoDB user
collection

Current user schema:
```json
{
  "email": String,
  "name": String,
  "created_at": Date
}
```
```

I'd like to: 1. Update the user schema to support authentication 2. Implement the backend endpoints for login, registration, and password reset 3. Create a middleware for protecting routes based on authentication 4. Set up the frontend components for the login flow

Can you help me with the schema design and backend implementation first?

```
**Improvements:**
- Specific technology stack identified
- Clear, detailed requirements
- Existing schema provided for context
- Scope broken down into manageable parts
```

```
- Clear starting point specified

## Confusing Prompt Engineering with Intentional Prompting

### Symptoms

- Focusing exclusively on crafting the perfect prompt to get complete solu
- Neglecting the manual work and understanding phases of the methodology
- Measuring success solely by the quality of AI outputs rather than learni
- Minimal critical evaluation of AI-generated solutions
- Skipping the six-step methodology and moving directly to implementation

### Root Causes

- Misconception that intentional prompting is just about creating better p
- Prioritising short-term efficiency over long-
term understanding
- Not fully grasping the educational purpose of the intentional prompting
- Confusing the means (good prompts) with the end (developer understanding

### Solutions

#### Clarify the Distinction

- Remember that prompt engineering is a set of techniques while intentiona
- recognise that prompt engineering focuses on getting optimal outputs whi
- Understand that prompt engineering is a component of intentional prompti

#### Balance Output and Process

- Follow the complete six-step methodology even when it seems faster to sk
- Evaluate success based on both the quality of the solution AND your unde
- Set aside time specifically for the "Work by Hand" and "Test with Data"
- Document your learning insights separate from code outputs

#### Adopt a Learning Mindset

- Ask AI to explain its solutions rather than just accepting them
- Challenge yourself to modify and extend AI-generated code
- Practice explaining solutions to others without referring to AI explanat
- Identify gaps in your understanding and use AI to fill them deliberately

## Key Takeaways

- **Be intentional about balancing AI assistance with skill development**
```

```
- **Invest time in clear, detailed prompts** to save time in iterativ
- **Always verify AI-generated solutions** through testing and manual
- **Maintain system-level perspective** when implementing individual
- **Address ethical and legal concerns proactively** through document
- **Learn from failures** by analysing and improving your prompting a
- **Distinguish between prompt engineering techniques and the intenti

By recognising and addressing these common pitfalls, you can maximise

`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImFwcGVuZGljZXMifQ==

```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImFwcGVuZGljZXMiLCJib2
```
```

Appendix B

Glossary

This glossary provides definitions for key terms used throughout "Intentional Prompting."

B.1 AI-Related Terms

AI Coding Assistant: Software tools that use artificial intelligence to help programmers write, understand, and debug code. Examples include GitHub Copilot, Amazon CodeWhisperer, and conversational AI models like Claude and ChatGPT when used for programming tasks.

Chain-of-Thought Prompting: A technique that encourages AI to break down complex reasoning into intermediate steps, producing more comprehensive and accurate outputs.

Few-Shot Prompting: Providing an AI with examples of the desired input-output behaviour before presenting the actual task, helping guide the model's response format and approach.

Foundation Model: Large AI models trained on vast datasets that serve as the basis for more specialised applications, including coding assistants.

Large Language Model (LLM): AI systems trained on massive text datasets that can generate human-like text and code based on prompts.

Prompt Engineering: The practice of crafting effective instructions for AI systems to produce desired outputs, involving specific techniques and patterns.

Zero-Shot Prompting: Asking an AI to perform a task without providing examples, relying on the model's pre-existing knowledge.

B.2 Intentional Prompting Concepts

Intentional Prompting: A deliberate approach to programming that leverages AI tools while maintaining human control over the development process. It focuses on understanding, guiding AI through structured methodology, using AI as a learning tool, and maintaining the human developer as the architect and decision-maker.

Six-Step Methodology: The structured approach at the core of intentional prompting: 1. Restate the problem 2. Identify input and output 3. Work the problem by hand 4. Write pseudocode 5. Convert to code 6. Test with data

Challenge Prompts: Deliberately introducing programming challenges to test understanding and explore potential issues.

Clarification Prompts: Questions that refine requirements and explore edge cases before implementation.

Foundation Prompts: Initial prompts that establish baseline requirements for a programming task.

Refinement Prompts: Prompts that push for code improvements based on best practices and efficiency considerations.

Scaffolding Prompts: Prompts that support learning by breaking down complex concepts into understandable components.

B.3 Programming Concepts

Code Smell: Patterns in code that may indicate deeper problems or potential for improvement.

Edge Case: A problem or situation that occurs only at an extreme operating parameter, often requiring special handling in code.

Pseudocode: A simplified, high-level description of a computer program or algorithm, using structural conventions of programming languages but intended for human reading rather than machine reading.

Refactoring: The process of restructuring existing code without changing its external behaviour, typically to improve non-functional attributes like readability, reduce complexity, or improve maintainability.

Technical Debt: The implied cost of additional rework caused by choosing an easy (but limited) solution now instead of using a better approach that would take longer.

B.4 Teaching and Learning Terms

AI-Proof Assignment: Assignments designed to be difficult for AI coding assistants to solve directly, often focusing on conceptual understanding or novel problems.

Documentation-First Methodology: An approach where students create detailed specifications and documentation before writing any code, using AI to evaluate completeness and clarity.

Process-Based Assessment: Evaluating students based on their problem-solving approach and understanding rather than just the final code output.